



Faculty of Engineering and Technology

Department of Computer Science

Graduation Project

CSEC4300

**Secure Network Asset Discovery and Vulnerability Assessment
System.**

Student Name	Student ID
Tharwat Assi	1220826
Ansam Hamayel	1220463
Afnan Manasrah	1221150

Supervised By

Dr. Murad Njoum

*A Graduation Project submitted to the Department of Computer Science and
Cyber Security as part of the requirements for the B.Sc. degree in Cyber Security.*

29 June 2026

ABSTRACT

With the continuous rapid growth of internet-facing assets and dependence on the internet for all things, cyberattacks have increased significantly, particularly targeting small and medium-sized enterprises (SMEs). Due to limited financial and technical resources to protect themselves, many SMEs are unable to deploy full and high-cost security. Which increases their exposure to cyber threats. Attack Surface Management (ASM) is widely used to identify externally exposed assets and vulnerabilities; however, most existing ASM solutions provide static views of vulnerability exposure and fail to demonstrate how multiple vulnerabilities can be combined to form realistic attack paths and how to validate vulnerabilities, which limits effective risk assessment and prioritization.

To overcome these limitations, this project suggests an integrated approach that combines Attack Surface Management (ASM) with the Continuous Threat Exposure Management (CTEM) framework to enable continuous discovery and validation of exposed vulnerabilities while ASM is used to identify internet-facing assets and CTEM introduces structured stages (scoping, discovery, prioritization, validation, and mobilization) for validating detected vulnerabilities, reducing false positives, and ensuring that only exploitable and relevant security weaknesses are considered in further analysis. Considering the effectiveness of CTEM in vulnerability validation and prioritization, it doesn't provide sufficient visibility into how validation vulnerabilities can chain together to form realistic attack scenarios. To address this limitation, the suggested system includes attack path generation techniques to model potential attacker movements to access interconnected assets. Then the analyzed results are presented through an interactive dashboard and structured security reports, providing enhanced visibility into system exposure and supporting informed risk mitigation decisions.

Words Count: 20249.

DECLARATION

We hereby declare that our graduation report is entirely our work and genuine/original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade, and it may result in withdrawal of our bachelor's degree.

Group members:

Name

Signature

Tharwat Assi	
Ansam Hamayel	
Afnan Manasrah	

Supervisor Signature

ACKNOWLEDGMENT

We would like to sincerely thank to the project supervisor Dr. Murad Njoum, for his insightful advice, supports, and helpful criticism during the course of this graduation project. Also we would like to express our sincere gratitude to Birzeit University for providing an academic environment that supported this work.

Table of Contents

ABSTRACT.....	2
DECLARATION.....	3
ACKNOWLEDGMENT.....	4
CHAPTER 1	13
1.1 Introduction.....	13
1.2 Problem Domain	14
1.3 Problem statement.....	14
1.4. Proposed System:.....	15
1.4.1 Aims and Objectives:	15
1.4.2 Proposed System Features:	15
1.5 Methodology:	16
1.6 Resource Requirements	22
1.7 Report Layout	23
CAPTER 2	24
2.1 Introduction.....	24
2.2 Overview of existing projects	24
2.2.1 Attack Surface Management (ASM) - Based Solutions	24
2.2.2 Continuous Threat Exposure Management (CTEM) - Based Solutions.....	24
2.2.3 Attack Graph - Based Approaches	25
2.3 Limitations of Existing Projects.....	25
2.4 Innovations of this project	26
2.5 Comparison Table	26
CHAPTER 3	28
3.1 Introduction.....	28
3.1.1 Purpose.....	28
3.1.2 Document Conventions.....	28
3.1.3 Intended Audience and Reading Suggestions	28
3.1.4 Product Scope	29
3.2 System Description	29
3.2.1 Product Description	29

3.2.2 System Objective	29
3.2.3 Operating Environment.....	29
3.2.4 Design and Implementation Constraints.....	30
3.2.5 Design and implementation Dependencies	30
3.2.6 Workflows	31
3.3 Requirements Specifications.....	31
3.3.1 User Requirements.....	31
3.3.2 System Requirements.....	32
3.3.3 Security Requirements	39
3.4 Non-functional Requirements (NFR)	45
3.4.1 Software Quality Attributes	45
3.5 Requirements Analysis and Modelling	47
3.5.1 Actors analysis	47
3.6.2 Use case & Misuse case Model/Diagram Analysis & Modelling.....	47
3.6.3 Use-Case Specifications.....	48
3.6.4 Activity Diagram.....	53
CHAPTER 4	54
4.1 Design Overview	54
4.1.1 Introduction.....	54
4.1.2 Environment Overview	54
4.1.3 Constraints, Assumptions and Dependencies	55
4.2 System Design Models	55
4.2.1 Class Diagram.....	56
4.2.2 Class Diagram using Secure UML	57
4.2.3 Sequence Diagrams.....	58
4.2.4 State Diagram.....	61
4.3 System Architectural Models.....	61
4.3.1 Design Goal	61
4.3.2 Component diagram.....	62
4.3.3 Software Architecture	62
4.3.4 Architectural Risk Analysis and Threat Modelling.....	63

4.3.4 Deployment Diagram.....	70
4.4 Data Management and Data Stores	70
4.4.1 ERD Diagram.....	72
4.5 Security Design	72
4.5.1 Authentication	73
4.5.2 Authorization.....	73
4.5.3 Data Access	73
4.5.4 File Upload.....	73
4.6 Interfaces	73
4.6.1 User Interfaces	73
4.6.2 System Interfaces	76
4.6.3 Software Interfaces	76
CHAPTER 5	77
5.1 Introduction.....	77
5.2 Implementation approach.....	77
5.2.1 ASM Module Implementation	77
5.2.2 CTEM Validation Module.....	77
5.2.3 Attack Graph Module.....	78
5.3 Software Testing and Verification.....	78
5.3.1 Test Case generation and Testing	78
5.3.2 Static Application Security Testing (SAST)	80
5.3.3 Dynamic Application Security Testing (SAST).....	81
5.4 Software Evaluation.....	81
CHAPTER 6	83
Conclusion	83
Future Work	83
Glossary	83
CHAPTER 7	84
Appendix 1	84
Appendix 2.....	85
Appendix 3.....	87

Appendix 4.....	88
Appendix 5.....	90
Appendix 6.....	95
References	110

Table of Figure

Figure 1: Workflow of the ASM and CTEM-Based Attack Graph Generation and Analysis	31
Figure 2: Steps of the Lightweight SQUARE Approach	39
Figure 3: Integrated outline of use cases and abuses of system functions, potential security attacks and associated mitigation mechanisms	48
Figure 4: Activity diagram showing the core system workflow	53
Figure 5: High-level client–server architecture of the proposed system	54
Figure 6: High-level class diagram illustrating the structure of the suggested system.....	56
Figure 7: Secure UML class diagram illustrating roles, permissions, and protected system actions	57
Figure 8: Sequence diagram for the secure login use case	58
Figure 9: Sequence diagram for secure user account management	59
Figure 10: Sequence diagram showing the workflow of defining and confirming the assessment scope and executing the security assessment process.	60
Figure 11: State diagram illustrating the system behavior.....	61
Figure 12: The main components of the system with their own responsibilities and interactions.	62
Figure 13: High-level layered software architecture illustrating the interaction between UI, business, and infrastructure layers.	63
Figure 14: High-level system architecture showing major components, data flows, and critical assets	64
Figure 15: Application decomposition and trust boundary identification for threat modelling ...	64
Figure 16: Data Flow Diagram (DFD) of the suggested system showing main processes, data flows, data stores, external entities, and trust boundaries.....	65
Figure 17: STRIDE threat modeling stages	66
Figure 18: Attack Trees represent leak sensitive security data threats, denial of service threats, and gain of privilege action threats.....	68
Figure 19 : Attack Trees represent repudiation action threats, tampering with assessment data or results threats, and spoofing identity threats.....	69
Figure 20: Deployment diagram of the proposed system	70
Figure 21: The Overall Entity Relationship Diagram (ERD) is used to show the database structure, main entities, and system relationships.....	72
Figure 22: Basic dashboard.....	74
Figure 23: Assets dashboard	74
Figure 24: Vulnerabilities Analysis Dashboard	75
Figure 25: Attack Graph Visualization Interface	75
Figure 26: Secure Use Case Diagram for SOC Analyst.	84
Figure 27: Secure Use Case Diagram for System Administrator and OTP Service	85
Figure 28: first: detailed view of system classes and their attributes and operations	85
Figure 29: Second detailed view of system classes and their attributes and operations.....	86

Figure 30: Third detailed view of system classes and their attributes and operations.....	86
Figure 31: Fourth detailed view of system classes and their attributes and operations.....	87
Figure 32: First detailed Secure UML class diagram illustrating roles, permissions, and protected system actions	87
Figure 33: Second detailed Secure UML class diagram illustrating roles, permissions, and protected system actions	88
Figure 34: Attack tree representing spoofing identity threats and possible attack paths	88
Figure 35: Attack tree representing repudiation action threats and possible attack paths	89
Figure 36: Attack tree representing tampering with assessment data or results threats and possible attack paths.....	89
Figure 37: Attack tree representing leak sensitive security data threats and possible attack paths.	89
Figure 38: Attack tree representing case denial of service threats and possible attack paths.	90
Figure 39: Attack tree representing gain of privilege action threats and possible attack paths	90
Figure 40: First detailed ERD	90
Figure 41: Second detailed ERD.....	91
Figure 42: Third detailed ERD	92
Figure 43: Fourth detailed ERD.....	93
Figure 44: Fifth detailed ERD.....	94
Figure 45: Sixth detailed ERD	94
Figure 46: Login to the site	95
Figure 47: Two factor authentication	96
Figure 48: Redirect to dashboard.....	96
Figure 49: The scan process is underway	97
Figure 50: View discovered assets	97
Figure 51: Select Asset for vulnerability scanning	98
Figure 52: Starting vulnerability scanning process.....	98
Figure 53: The vulnerability scanning process is ongoing	98
Figure 54: Vulnerability scanning process completed	98
Figure 55: View discovered vulnerabilities	99
Figure 56: Send an urgent email to SOC Analyst	99
Figure 57: Graph Attack Offer after its completion.....	100
Figure 58: View all detected attack paths	100
Figure 59: View the first step and its information from the chosen attack path.	101
Figure 60: View the second step and its information from the chosen attack path.....	101
Figure 61: View the third step and its information from the chosen attack path.	101
Figure 62: Choose which sections to display in the report	102
Figure 63: Report Generated.....	102
Figure 64: Enter the required information for the scan.....	103
Figure 65: Scan Scheduled.....	103

Figure 66: The scanning process started at the time specified.....	103
Figure 67: Enter the required information for the scan table.....	103
Figure 68: Scan Scheduled.....	104
Figure 69: The scanning process started at the time specified.....	104
Figure 70: Trying to change the URL to access the admin interface from the SOC interface....	104
Figure 71: Block access and user log-out	104
Figure 72: Try SQL injection	105
Figure 73: Try XSS attack.....	105
Figure 74: Try invalid credential.....	106
Figure 75: Try make the input field empty	106
Figure 76: The test was completed and the vulnerabilities were exposed	106
Figure 77: SQL Injection false positive	107
Figure 78: Fixed CSRF on login page	107
Figure 79: Apply security headers to protect system sessions.	107
Figure 80: Apply security cookies parameters.....	107
Figure 81: Python codes static testing	108
Figure 82: Javascript codes static testing.....	108
Figure 83: PHP codes static testing.....	109

List of Tables

Table 1: Tools and technologies used in the system implementation	23
Table 2: Comparison between Existing Solutions and the suggest System.....	26
Table 3: entities Critical Assets for Threat Modeling and Risk Analysis	40
Table 4: Security Requirements Classification and Priority	44
Table 5: Identifies the main actors with the suggested system describes their use cases	47
Table 6: Detailed Use Case Specification for UR-02 – Secure login	49
Table 7: Detailed Use Case Specification for UR-03 -Define & Confirm Assessment Scope	50
Table 8: Detailed Use Case Specification for UR-09 -View Attack Graph	51
Table 9: STRIDE-based classification of identified security threats in the proposed system	66
Table 10: Documented STRIDE-based security threats for the proposed system	67
Table 11: DREAD-based Risk Assessment of Identified Threats	69
Table 12: Test Cases and Implementation Results	78
Table 13: Software evaluation tests and results	81

CHAPTER 1

INTRODUCTION

1.1 Introduction

Over the past few years, the rapid growth in Internet-connected assets such as web applications, cloud services, and externally exposed systems has significantly expanded the attack surface for cybercriminals. Small and medium-sized enterprises (SMEs) are particularly exposed, as they often lack sufficient resources to fully secure their Internet-facing assets [1].

Despite the effectiveness of common solutions such as Attack Surface Management (ASM), it is a security methodology that detects and monitors all previously known and undiscovered assets on the Internet, which may contain vulnerabilities that can be exploited to gain access to the system, and provides a clear view of the organization's external attack surface, reducing the likelihood of cyberattacks succeeding [2]. However, they usually provide a consistent, incomplete view of security risks and do not determine the importance of the asset, or the vulnerabilities it contains, and provide a limited analysis of how these vulnerabilities can be exploited in real-world attack scenarios [3], and they rely primarily on the scanning tools, their quality, and comprehensiveness. In addition, they only perform external scanning, and may result in false positives in some cases, such as duplicates of assets due to the variety of detection tools that may be similar in some results and differ in others[4][5].

To address these limitations, modern cybersecurity strategies have increasingly implemented Continuous Threat Exposure Management (CTEM), an evolving security framework that aims to overcome the limitations of traditional vulnerability-centric security approaches through a five-stage continuous strategy that helps in identifying, verifying, and addressing security vulnerabilities before attackers can exploit them [6] [7]. CTEM focuses on five domains: scoping, discovery, prioritization, validating, and mobilizing the mitigation actions based on real-world exploitability rather than theoretical vulnerability scores [6] [7]. However, it is important to note that while CTEM is effective in exposing weaknesses, most CTEM products lack the ability to provide organizations with a clear graphical representation of how different weaknesses in an organization's environment can potentially work together in order to provide an attack scenario. This limitation can be addressed by integrating CTEM with attack graph analysis, where multiple vulnerabilities can be linked together to visualize possible attack paths and provide organizations with a better understanding of chained attack scenarios within their environment [7].

To overcome the identified limitations of existing ASM and CTEM solutions, particularly the lack of contextual attack visualization and the challenges related to asset correlation and automation, this project suggests an effective ASM based system integrated with the CTEM framework and Attack Graph to deliver a dynamic and automated view of an organization's external exposure, with a particular focus on small and medium-sized environments. The proposed system continuously discovers Internet-facing assets, correlates vulnerabilities across validated CTEM stages, and dynamically generates attack graphs that reflect realistic attacker movements across

interconnected assets[3][6]. CTEM also adopts an ongoing methodology [6] that prioritizes and vets threat scenarios to support effective security decision-making. This makes ASM and CTEM particularly suitable for small and medium-sized enterprises (SMEs), which require practical and continuous techniques that combine visibility and prioritization to improve their overall security posture [1].

The system provides analysts with a clear and actionable view of the organization's security posture with an interactive dashboard that offers comprehensive data, using system reports, ranked attack paths, vulnerability severity levels, and asset risks. This approach leads to greater visibility, fewer false positives, and earlier detection of critical attack paths before they can be exploited. Consequently, the system supports SMEs in transitioning from vulnerability scanning to proactive security decision-making.

1.2 Problem Domain

Many challenges in modern organizations, especially in SME environments, stand out in understanding and managing the security risks resulting from the continuous expansion of dynamic environments, where assets and vulnerabilities can emerge or change at any time [3]. Due to financial constraints, the absence of specialized security teams and the limited resources available to them, it has become difficult to obtain a clear and comprehensive security view of exposed assets and associated vulnerabilities. Organizations also suffer from a lot of alerts and inaccurate results, which lead to wasting a lot of resources and time in addressing security risks that may not pose a real threat. There is an urgent need for lightweight, scalable, and easy-to-manage security tools that do not rely on complex infrastructures or costly manual processes [1].

1.3 Problem statement

Organizations, particularly small and medium-sized enterprises (SMEs), continue to have trouble accurately understanding their security risk [1], despite the growing use of Attack Surface Management (ASM) tools and the development of Continuous Threat Exposure Management (CTEM) frameworks. Most existing ASM and CTEM solutions focus primarily on asset discovery and vulnerability listing, offering limited insight into how these vulnerabilities can be exploited within realistic attack scenarios or combined by attackers [5][4] [6].

Hence, the security team is often overwhelmed with invalidated, non-prioritized, and false-positive data, thus increasing the difficulty in identifying low-impact vulnerabilities from high-impact vulnerabilities and critical risks [7]. Without an efficient correlation mechanism between assets, vulnerabilities, exploitability, and potential attackers, it is impossible to have an efficient decision-making process in security as well as an efficient utilization of limited security resources [8].

In addition, current ASM and CTEM implementations lack fully automated and dynamic capabilities, which limits their effectiveness in environments where assets and vulnerabilities regularly change [5]. Without integrated attack path analysis, organizations cannot clearly understand how attackers might navigate their infrastructure to reach sensitive resources.

Consequently, the primary challenge addressed by this project is the lack of an integrated, automated, and attacker-focused approach that combines ASM, CTEM, and attack graph analysis. This gap between asset and vulnerability visibility and a true understanding of real-world attack paths and criminal behavior points out the need for a unified security framework, one that treats ASM, CTEM, and attack graph analysis as interconnected components of a single, automated security system rather than as isolated processes.

1.4. Proposed System:

This section presents the suggested system, which integrates Attack Surface Management (ASM), Continuous Threat Exposure Management (CTEM), and Attack Graph.

1.4.1 Aims and Objectives:

The main objective of this project is to design a secure system that integrates ASM, CTEM, and Attack Graph methodologies. This system will be able to scan the organizational attack surface, identify potential risks, and create attack graphs that outline different potential paths for attackers. This system would be able to provide useful information for better decision-making.

The aims and objectives of the suggested system are outlined as follows:

Aims:

1. Develop a system capable of continuously managing, assessing, and exposing cybersecurity risks.
2. Integrate ASM, CTEM, and Attack Graph approaches to improve asset visibility, attack surface scanning, and overall security.
3. Help organizations to identify their exposures and, vulnerabilities to understand how to protect them.

Objectives:

1. Use open-source tools to conduct external ASM scanning of networks, websites, and perimeter assets.
2. To perform regular risk assessment, incorporate scanning results into a CTEM framework.
3. Create attack graphs with possible attack paths.
4. Provide a comprehensive, easy-to-use security and understandable dashboard for decision support.

1.4.2 Proposed System Features:

1. Detect externally exposed assets, collect complete identifying information and identify the possible vulnerabilities related with each asset, and store the data in a centralized repository.
2. Provide continuous security monitoring and vulnerability assessment using CTEM and ASM methodologies.
3. Generate attack graphs to show the possible paths that attackers may take.

4. Assess and rank the vulnerabilities according to the degree of risk.
5. Display the results through an interactive dashboard that displays the outcomes in a clear and structured way.
6. Develop a feature for security reports that gives a summary and overview of the security position's outcomes.

1.5 Methodology:

The suggested system follows the steps outlined below:

1. Define the Scope and Assets to Be Assessed:

These include all IP addresses, domains, subdomains, and internet-facing services.

2. External scan and data collection:

Scan and discover all externally exposed assets across the attack surface using multiple tools and supporting scripts, as no single tool can identify all assets.

Tools used in this part:

- **Nmap:** It is an open-source tool that scans networks and searches for assets such as devices, services, and even the type of operating system used. And performs basic and advanced scans with many options, can be used for compliance tests as well as asset and inventory management [9]. However, it does not give details of security vulnerabilities or identify the risk factor, and it may omit some assets [10]. The role of Nmap in this project is to be part of the scanning tools, and it is more accurate than MASSCAN and gives more detailed information but is relatively slower.
- **MASSCAN:** It is a tool that scans ports in large bands, characterized by its speed in scanning and its ability to scan large areas in a short time, but it gives limited details compared to Nmap. Its role in the project is to scan open ports of discovered assets [9].
- **Subfinder:** It is a tool that detects subdomains belonging to a specific domain, is characterized by its high data collection capacity, and can be easily integrated with other tools, but it relies on a large number of data sources and does not return the IP address of the subdomain. Its role in the project is to find subdomains [11].
- **DNSX:** It is a tool that resolves subdomain names, and returns the IP address of each domain found by the subfinder tool. Its role in the project is to validate domains and return IP addresses.
- **HTTPX:** It is a tool that checks whether a website is working or closed. Its role in the project is to filter the detected domains and ensure effective services.
- **Nuclei:** It is a tool that scans vulnerabilities and detects wrong settings and exposed panels, and its features include many templates ready to check for several types of vulnerabilities. Its role in the project is to detect gaps in the detected bands.
- **OpenVAS:** is an open-source system that assesses, detects, and manages vulnerabilities, has its own calculator for use in the common vulnerability assessment system (CVSS), and provides reports in several extensions, but only supports the Linux operating system,

consumes a lot of resources such as memory and CPU. Its role in the project is to detect vulnerabilities in the discovered assets [9].

- **CISA:** It is an official American security agency that publishes security information, one of its features is to provide information about real vulnerabilities that have been exploited and provide information about them, and to publish security alerts and threat reports. Its role in the project is to verify the possibility of exploiting vulnerabilities for all assets, and to distinguish between real and fake vulnerabilities.

3. Banners and metadata extraction:

The banners issued by the tools are collected, then extract the metadata and ensure its completeness. If it is not complete, the required data should be completed and searched for in external sources.

4. Prioritization:

The vulnerability results are classified based on their severity, exploitability, level of exposure, and role in potential attacks. High-impact results are filtered in this step for additional validation.

5. Validation (Exploitability Verification):

Instead of depending only on theoretical severity scores, the CTEM framework concentrates on verifying whether identified exposures can be practically exploited in real-world attack scenarios during the validation stage [8]. This stage employs safe and controlled validation techniques, such as adversary emulation, and exposure validation to ascertain how an attacker might actually exploit a vulnerability in the environment [12], and distinguish between theoretical vulnerabilities and exposures that can be practically exploited by attackers (reducing false positives) so that subsequent CTEM stages focus on exposures that demonstrate realistic attack potential [6].

6. Mobilization (Actionable Remediation):

Validated exposures are transformed into actionable remediation steps, to support the decision-making and guide remediation efforts, based on likely attacker paths and potential impact. It helps to understand how attackers might traverse interconnected systems, validated exposures are examined in relation to one another, assisting in the creation of the attack paths and attack graph representations.

7. Attack Graph Generation:

The representation of the attack graph is defined as a directed weighted graph $G = (V, E)$, where the vertices are represented by V and the directed edges between vertices are represented by E . The vertices correspond to different security states that include an external attacker, discovered or compromised asset, subdomain, and attacker capability.

The directed edges show various attacks that an attacker may take in transitioning from vertex v_i to vertex v_j .

Each edge is associated with a number of attributes that will be used to analyze the attack paths, including a priority rating, cost, impact rating, confidence rating, exploitability rating, vulnerability ID, CVE ID, precondition, and postcondition. Therefore, an edge can be defined as follows: $e_{ij} = (v_i, v_j, type, priority, resistance, impact, confidence, precondition, postcondition)$.

a. Initial Sampling Phase

The attack graph generation process starts with an initial sampling phase, where the initial nodes of the graph are established. These nodes include two main categories: external asset nodes, which represent externally exposed assets or services, and an attacker node, which represents the initial state of the attacker and serves as the starting point for possible transitions based on exposed vulnerabilities. The transition probability utilized in the case of the controlled random walk depends on the weight associated with each outgoing edge from the current node. The formula for calculating the probability of going from a current node v_i to a succeeding node v_j can be given as $P(v_i \text{ to } v_j) = W_{ij} / \sum W_{ik}$ where W_{ij} is the calculated priority weight of the edge from v_i to v_j , and $\sum W_{ik}$ is the sum of the weights of all valid outgoing candidate edges from v_i . This ensures that higher priority, lower resistance, and more impactful transitions are more likely to be explored, while still allowing alternative attack paths to be sampled.

b. Asset-Based vulnerability modeling

Once the set of external assets and validated vulnerabilities has been gathered, the attack graph engine makes correlations among gathered information and creates an acceptable model of possible attacker movements in the analyzed environment.

At first, each validated vulnerability is linked to its corresponding asset and exposed services through CVE ID and gathered data on assets. Then, the engine takes into effect the properties of the identified vulnerability, like severity, exploitability, validation confidence, target service, and network exposure, in order to establish whether this vulnerability can actually cause a proper exploit transition.

In contrast to the conventional approach to vulnerability analysis, in which each identified vulnerability is viewed independently from others, the proposed approach converts each validated vulnerability into an exploit transition between two security states represented in the attack graph. Such an exploit transition involves not only specific properties of the vulnerability but also its preconditions and postconditions, which define the initial state and the final state of the attack graph. With each new asset that becomes reachable following a successful compromise, the attack graph will have to account for new states that are reachable and perform vulnerability assessments on those assets. The result is an attack graph that is more realistic, as the process of correlation will make the model capable

of considering lateral movements through interconnected assets and not just specific vulnerabilities. Therefore, the resulting attack graph will have a set of realistic attack paths.

c. Initial random walk execution

The creation of the attack graph and the attack path explorations are performed by the system using a controlled weighted random walk method. In contrast to exhaustive search methods that try to explore every single attack path, this approach focuses on a probabilistic exploration of the graph while controlling the whole exploration process with an adjustable set of parameters.

The exploration process begins at the attacker node and continues by selecting one of the outgoing transitions according to its calculated transition probability. The probability of selecting a transition is proportional to its priority weight, which is derived from vulnerability severity, exploitability, validation confidence, impact, and CTEM-based prioritization. Consequently, transitions representing more realistic attack possibilities are explored more frequently, while alternative paths remain discoverable.

- To ensure that uncontrolled exploration does not occur, the following configurable parameters are used to control the exploration process:
- Maximum Walk Depth: restricts the number of transitions that can be made within an attack path.
- Maximum Number of Walks: sets the number of iterations of random walk processes that will be conducted per graph exploration phase.
- Convergence Patience: dictates the number of successive iterations where no new attack paths are discovered before ending the exploration process.
- Path Signature Check: unique identifiers of discovered paths prevent duplicate paths from being stored and explored.

The use of these configurable parameters allows the process of attack graph generation to strike a balance between exploration and computation efficiency.

d. Early Attack path discovery

One of the most important benefits that comes with the suggested approach is the capability to identify attack paths even before the attack graph construction has finished. If a vulnerability creates a valid connection between two nodes correctly, an edge will be created, and therefore, either a partial or full attack path will be identified immediately and will be placed within the Attack Path Repository.

e. Progressive graph expansion

As additional vulnerabilities are discovered over time in the assets, the attack graph is expanded incrementally. With each vulnerability, its corresponding preconditions and postconditions get represented in the CVE model, and thus new edges of exploitation get introduced, relating specific system states together. After that, further sampling runs get

triggered from these new nodes, thus enabling the exploration of new potential attack paths constantly according to these changes.

f. Convergence monitoring

To improve computational efficiency, the proposed system continuously monitors the attack graph generation process to determine when sufficient exploration has been achieved. During each progressive generation batch, the system compares the current graph structure with the previous one while tracking the number of newly discovered unique attack paths. If consecutive batches produce no significant structural changes or new attack paths, the graph is considered to have converged. To eliminate redundant exploration, each attack path is assigned a unique signature, preventing duplicate paths from being stored or processed. Once the predefined convergence conditions are satisfied, the graph generation process terminates automatically, ensuring efficient exploration while reducing unnecessary computational overhead.

g. Attack path ranking

After convergence is achieved, all discovered attack paths are extracted and ranked according to their overall risk. The ranking process considers multiple factors, including vulnerability severity, exploitability, validation confidence, transition priority, path length, and normalized risk score. Complete attack paths are prioritized over partial paths, while lower-resistance paths receive higher importance during analysis. This ranking enables security analysts to focus on the most critical and realistic attack scenarios, supporting effective risk prioritization and remediation planning.

In order to supplement the random walk search procedure, the proposed framework uses the Dijkstra's shortest path algorithm to compute the minimum resistance attack paths starting from the attack node to reach the compromised nodes. This is done by computing edge resistance costs in the network to determine the attack paths that are most likely from the point of view of the attacker.

In order to enhance the understanding of the generated attack paths, the proposed framework will assign every step of an attack path to the MITRE ATT&CK framework and also assign it to the specific cyber kill chain phase. Each of the transitions made by the attacker is assessed using the vulnerability information, the service metadata, and other context information in order to assign the most appropriate ATT&CK tactics and technique along with the specific phase of the attack with a confidence value assigned.

h. Final Outputs

The result of this phase will be the fully generated attack graph along with the attack paths discovered, the critical attack paths ranked, and convergence metrics showing the stability of the attack graph.

Tools used in attack graph model:

- **Python:** it is a popular high-level programming language, characterized by its simple and readable syntax and has a large collection of libraries for statistical modeling, data processing, and graph analysis. However, it has some drawbacks such as poor support for large-scale concurrent computation without the need for extra frameworks. Python was selected because it is highly adaptable also it enables quick development when used with the security analysis library, and generate attack graph logic [13].
- **NetworkX:** it's a Python package used to construct and examine intricate graph structures. It is used to model the attack graph [14]. It offers advanced graph modeling and traversal functionalities, easily integrates with python-based data processing pipelines, and supports multiple graph algorithms relevant to attack path analysis. However, it has some drawbacks such as its scalability is limited for very large graphs and it's not designed for real-time or high-performance graph processing. NetworkX was chosen because it offers enough flexibility and analytical capability to analyze and model attack graphs, which is compatible with the suggested system design.
- **CVE Dataset:** the Common Vulnerabilities and Exposures (CVE) community provides a dataset of standardized vulnerability identification numbers and explanations of the publicly known vulnerabilities [15]. In the system, the CVEs serve as the principal way of vulnerability identification and the construction of the exploit transition in the attack graph. It is characterized by its globally standardized vulnerability identifiers, it's also widely accepted and continuously updated, and suitable with security scoring such as CVSS. However, it does not ensure that every vulnerability on the list has an exploit available. Because it offers a dependable and consistent basis for vulnerability identification, the CVE dataset was chosen to the suggested system.
- **Random Walk Algorithms:** These are simulation techniques for traversing graphs, used to explore paths within large and complex graphs. Weighted random walking is used to reduce the repetitive exploration of the same paths, while finding a variety of attack paths efficiently [12]. This algorithm is characterized by its effective exploration of large graphs without the need for a comprehensive census, its lack of reliance on machine learning techniques, and its ability to identify varied and unclear attack paths. However, the potential outcomes may vary from execution to execution, and it is not guaranteed to cover all potential attack paths. Random walking was chosen in this project because it provides random walk sampling as an effective way to explore large attack graphs, while detecting various potential attack paths at a lower computational cost and the system can detect different attack paths during the early stages of graph construction.

In terms of technology used in the design of the Attack Graph engine, a mixed backend technology stack, including Python and PHP, has been considered to combine the best features of both platforms. Specifically, Python will be used for graph building, vulnerability linking, controlled random walk attacks, Dijkstra graph path searches, and MITRE ATT&CK mapping with the use of the NetworkX library. The built graphs, attack

paths, and analyzed data will be saved in a centralized MySQL database. Meanwhile, PHP will take care of the web application, API security, user authentication, and SOC dashboards, which will display the attack graph and ranked attack paths.

8. Interactive Dashboard Generation:

An integrated interactive interface in which all outputs from the beginning of the scan process to the end of the attack paths extraction process are displayed, and includes all the statistics of assets, vulnerabilities, and the most dangerous attack paths and their statistics.

System design:

- Frontend: HTML, CSS, and JavaScript
- Backend: PHP and python.
- Database: MySQL.

Technologies used in this stage:

- **HTML:** HTML was chosen because it is the base technology for structuring web-based user interfaces and provides a solid foundation on which to develop the dashboard and the administrative panels [16].
- **CSS:** In order to improve the dashboard interface's usability and visual clarity and guarantee a user-responsive and intuitive experience, CSS was selected [17]
- **JavaScript:** Was chosen to offer dynamism in the dashboard by enabling users to navigate security data and analysis results interactively [18].
- **PHP:** The reason why the backend management tasks in the application are done using PHP is mainly due to ease of usage, popularity, and web-related functionalities [19].
- **MySQL:** MySQL's popularity, reliability, compatibility with the system's web architecture, and capacity to handle structured security data all played a role in the decision [20] .

1.6 Resource Requirements

- **Human Resources: -**
 1. Full stack developer: To develop system logic, connect scanning tools, analyze vulnerabilities, build an attack graph, design a dashboard, display results and manage the admin panel.
 2. Security Analyst: Interpret the results, correlate, analyze, and evaluate exposures and security gaps and minimize false positives and false negatives.
- **Hardware Requirements: -**
 1. Server (Virtual Machine):
 - CPU: 2-4 cores.
 - RAM: 4-8GB.

- Storage: 50-100 GB.
2. Development machine: Laptops.

System development and testing activities were conducted using standard laptop computers. The development machines were outfitted with Intel 64-bit processors and operating on Microsoft Windows 11 (64-bit). Memory configurations of 12 GB and 16 GB of RAM were available across the development devices. All laptops provided sufficient computational resources for asset discovery, vulnerability analysis, and attack graph generation tasks required by the suggested system.

- **Software Requirements: -**

Table 1 summarize the tools and technologies utilized for the suggested system.

Table 1: Tools and technologies used in the system implementation

Category	Tools, technologies	Objective
ASM tools	Nmap, Masscan, Subfinder, HTTPX, DNSX and OpenVAS.	Discovery of assets, services, and subdomains
CTEM tools	CVE, CISA, Nuclei and OpenVAS	Vulnerability scanning and validation
Frontend development	HTML, CSS and JavaScript	UI and dashboard implementation
Backend development	PHP and Python	Business logic and functionality of the system
Database	MySQL	System data storage and management
Attack graph building tools	Python, NetworkX, CVE Dataset, Random Walk Algorithms, Vulnerability Metadata, Statistical Analysis Techniques.	Attack graph generation and attack path analysis
Operating Environment	Kali Linux	System deployment and security tools execution
Version Control	GitHub	Source code management and tracking

1.7 Report Layout

This report consists of four chapters that present the project in a clear and structured manner. The first chapter presents the problem addressed by the project, the project objectives, and the methodology followed. The second chapter reviews related work and similar projects, highlighting their limitations and the improvements suggested in this project. Building on this foundation, the third chapter focuses on the project scope, user requirements, system requirements, system features, and functional, non-functional, and security requirements. Finally, the fourth chapter presents the system design and architecture, explaining how the suggested solution is structured to achieve its intended security objectives.

CAPTER 2

RELATED WORK

2.1 Introduction

This chapter provides a review of the related work that is relevant to the project. It looks at the different approaches that have been used, their drawbacks, and the ways in which our suggested system overcomes these limitations to improve and extend existing tools.

2.2 Overview of existing projects

Current cybersecurity research and technological solutions propose multiple approaches to manage and assess organizational security risks. However, most existing projects define Attack Surface Management (ASM), Continuous Threat Exposure Management (CTEM), and attack graph analysis as separate techniques rather than integrated components of a single security system. The following section reviews existing approaches by describing their fundamental methods, achieved results, advantages, disadvantages, and key limitations.

2.2.1 Attack Surface Management (ASM) - Based Solutions

It's focus on discovering and monitoring externally exposed assets, including IP addresses, domains, subdomains, open ports, cloud services, and publicly accessible systems [2] [3]. These systems depend mainly on automated scanning techniques and vulnerability databases such as the Common Vulnerabilities and Exposures (CVE) repository to identify known weaknesses associated with discovered assets [4] [8], in doing so, security teams are able to maintain an up-to-date set of internet-connected components and reduce blind spots, especially in dynamic environments with low operational costs and can therefore easily meet the needs of organizations with limited security resources by identifying exposed assets and mapping known vulnerabilities [2] [3].

However, a major drawback is that most ASM tools remain asset-centric and focused on vulnerability lists. They typically focus on identifying “what is exposed” rather than explaining “how an attacker could exploit these exposures” in actual attack scenarios [5]. That is, they do not verify exploitability, lack specific correlations between vulnerabilities, and do not simulate attacker behavior or attack paths. Consequently, they provide an incomplete risk assessment and limited decision-making support [5][3].

2.2.2 Continuous Threat Exposure Management (CTEM) - Based Solutions

Continuous Threat Exposure Management approaches extend beyond traditional vulnerability management processes by introducing a continuous lifecycle that includes scoping, discovery, validation, prioritization, and mobilization of security exposures [10] [8]. CTEM puts focus on risk-driven decision-making processes that involve dangers most likely to be attacked along with those having high business impact. Thus, vulnerability prioritization has been improved through

vulnerability verification and the inclusion of exploitability data, thus moving from basic vulnerability identification to exposure risk management [8] [21].

However, the CTEM methodology has its limitations. Most of these solutions rely on abstract risk assessments and do not provide useful and understandable graphical representations of how proven vulnerabilities are linked and exploited by attackers. Therefore, frameworks of this type may not be very effective in illustrating how attacks occur [6]. They also rely heavily on input data, and this data must include accurate external assets, in addition to information about the vulnerabilities [6].

2.2.3 Attack Graph - Based Approaches

Attack Graph techniques aim to model the attacker's behavior specifically by representing systems as graphs, where nodes represent system assets, and edges represent exploit transitions [6] [10]. These approaches mostly focus on finding, viewing, and analyzing possible attack paths that an attacker may take to penetrate critical assets when initiating an attack from an external entry point, thus clearly modeling the dependencies between vulnerabilities and system states and allowing for an understanding of how individual vulnerabilities are integrated into entire attack chains [12]. These technologies enable advanced security analytics, such as identifying critical attack paths and assessing the impacts of individual vulnerabilities within broader attack scenarios.

However, traditional attack graph generation tends to suffer from poor scalability and high complexity, often need an expensive computation cost in a large-scale or dynamically changing environment [22], and these limitations create difficulties for the practical implementation of attack graphs in operational security environment, as they require expensive preprocessing, deep system knowledge, or manual modelling. It is hard to deploy these in real-time or resource constrained SME environments [6].

2.3 Limitations of Existing Projects

There are many limitations in current solutions, as studies [4] primarily focus on discovering internet-connected assets and identifying vulnerabilities using multiple scanning tools. While these approaches improve coverage, they do not provide clear methods for prioritizing risks or verifying their exploitability. Furthermore, they do not explain how vulnerabilities are correlated to form real attack trails, thus limiting their practical value in security analysis.

Studies [7] and [6], related to CTEM, offer effective conceptual and organizational frameworks for CTEM and risk verification. However, they focus more on strategic implementation and theory rather than technical application. They do not offer automated systems that integrate asset discovery, practical vulnerability verification, and attack trail analysis into a single platform, neither do they include clear actual performance metrics.

for attack path generation, studies [12] propose advanced and scalable techniques for early attack trail detection using statistical sampling and machine learning-guided methods. These methodologies have the potential to greatly enhance efficiency and scalability of the operations but on the other hand machine learning, based methods demand high quality training data, lead to more complex systems and may not be easily transferable to different environments. Statistical methods, on the other hand, still rely heavily on CVE and may produce invalid or overvalued paths.

Generally, current research treats asset management, vulnerability management, and attack scheme generation as separate issues and does not offer an integrated system that combines continuous asset discovery, vulnerability verification, risk prioritization, and dynamic attack scheme generation in a single and automated manner, particularly for small and medium-sized enterprises (SMEs).

2.4 Innovations of this project

This suggested system introduces a major innovation by producing an existing system that integrates ASM & CTEM & Attack Graph methodologies, as opposed to current solutions, so that the system starts by discovering assets, collecting and analyzing data, linking them to vulnerabilities and plotting attack paths, displaying results on the interactive dashboard, and generating reports. This allows the risk to be visualized and minimized, and the false positive and false negative are minimized, to clearly highlight the differences between existing approaches and the suggested system.

2.5 Comparison Table

The following *Table 2* summarizes the main differences between existing approaches and the suggested system.

Table 2: Comparison between Existing Solutions and the suggest System

Comparison Aspect	Existing Solutions (ASM, CTEM, Attack Graph)	Suggested system
Overall Concept	ASM, CTEM, and Attack Graph methodologies have evolved as separate solutions [7][23][4].	A centralized system that integrates all methodologies.
Security Visibility Provided	It identifies known weaknesses, exposed assets, and a limited understanding of attack scenarios [8] [23].	Provides contextual visibility by showing exposed assets with possible attack paths.

Data Completeness and Input Dependency	CTEM frameworks rely heavily on the quality and completeness of input data, which is often limited by incomplete asset discovery [6] [12].	Continuous external asset discovery improves data completeness before validation and analysis stages.
Exploitability Verification	Traditional ASM tools list vulnerabilities without confirming whether they are practically exploitable [4][8].	Includes a validation stage that verifies exploitability before vulnerabilities are used in attack path construction.
Correlation of Vulnerabilities	It does not explain how vulnerabilities are incorporated into real-world attacks [4] [8].	Vulnerabilities are linked and transformed into structured exploit chains.
Attack Path Modeling	Separate from continuous asset discovery and exposure management [4][12].	Dynamically generates attack graphs, directly from validated CTEM findings and the discovered assets.
Level of Integration	Limited integration due to the independence of security methodologies and their components [12][8][4].	A comprehensive integration from asset discovery to attack graph generation.
Automation Level	Partial automation, especially in CTEM-related work [12] [8].	Full automation, including discovery, verification, correlation, and visualization.
Scalability and Practicality	Attack plans face many challenges [22].	Uses progressive graph generation and sampling techniques to remain practical and scalable for small environments.
Suitability for SMEs	Complex, expensive, or designed for large corporations [5] [8] [24].	Specifically designed to be lightweight, and suitable for SMEs.
Final Outputs	Outputs are often limited to vulnerability lists, conceptual risk reports, or complex technical graphs.	Provides an interactive dashboard including asset inventory, attack graphs, and clear security reports.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Introduction

This chapter outlines the Software Requirements Specification (SRS) for the suggested system. The SRS serves as a detailed and organized specification of system requirements to serve as a reference point to ensure secure development and evaluation of the suggested system. The chapter provides a platform for a detailed understanding of system scope, constraints, and functionality.

3.1.1 Purpose

This section presents the Software Requirements Specification (SRS) for the suggested system. The SRS provides a complete and structured view of the system's scope, limitations, and requirements, including user requirements, functional requirements, non-functional requirements, security requirements, and critical system constraints. These requirements form the basis needed to support the design, development, implementation, and secure validation of the suggested system, and promote a common understanding of the system among all project stakeholders, including academic supervisors, developers, and reviewers. It remains the central guidance document for the entire Secure Software Development Lifecycle (SSDLC).

3.1.2 Document Conventions

This report is written in the usual academic and technical way to maintain clarity and consistency. The section and subtitle headers are numbered systematically to mirror the Software Requirements Specification document structure. All software requirements are written in plain and specific natural-language form so that they can be easily understood by everyone involved with the project. The requirements are given a unique identification and are numbered in a serial manner (e.g., UR-1, SR-1). Citations are marked by numbering inside square brackets (e.g., [1], [2]) and refer to the list of references provided at the end of this document. The tables, figures, and other graphic elements are numbered in a continuous manner with captions that are easy to refer to in the document.

3.1.3 Intended Audience and Reading Suggestions

The document is specifically for all stakeholder's use of the suggested system, which includes the system developer and administrators who need to understand the detailed requirements of the system in order to design, implement, and maintain the system.

The document targets the academic supervisors and evaluators who need to review the system requirements, design decisions, and overall project quality. Additional readers include security analysts and testers interested in validation of security requirements and review of the system's overall security posture. Those readers who want a high-level overview of the system's

functionality and business services should instead read Chapter 3, Software Requirements Specification. Those readers interested in system architecture and design decisions should review Chapter 4, Software Design. Those interested in security analysis and threat modeling and the definition of security requirements will be most interested in Section 3.4.3 Security Requirements, of this chapter.

3.1.4 Product Scope

The suggested system aims to develop a web-based cybersecurity application, specifically designed for small and medium-sized enterprises, to assist them in their security analysis and risk assessment processes. The suggested system provides different types of user accounts, enabling access, management, and analysis of security data, and includes an administrative dashboard used to configure, manage, and control the application, and an interactive dashboard that displays security findings, including network asset discovery, vulnerability analysis, and attack path analysis, in a clear and visual way to support informed security decision-making.

3.2 System Description

This section provides an overview of the system before technical details. The suggested system integrates Attack Surface Management (ASM), Continuous Threat Exposure Management (CTEM), and attack graph analysis into an integrated system that aims to help SMEs scan the external surface exposed on the Internet, discover all related assets, identify the threats they face and the vulnerabilities contained in each asset, and map potential attack paths.

3.2.1 Product Description

The suggested system is a new standalone web-based application that provides an integrated solution for security analysis by combining multiple cybersecurity concepts into a single framework, including Attack Surface Management (ASM), Continuous Threat Exposure Management (CTEM), and Attack Graph analysis. The suggested system interacts with the open-source scanning tools and publicly available vulnerability databases to gather information about network assets and their associated vulnerabilities.

3.2.2 System Objective

The main goal of the suggested system is to support and improve the state of security in Small and Medium Enterprises (SMEs) and help them to maintain visibility of their external attack surface. The system would support effectively discovering and evaluating potential security risks via asset discovery, vulnerability identification, and attack path analyses through ASM, CTEM and Attack Graph, also it helps security analysts prioritize and make reliable security decisions based on risk-driven insights.

3.2.3 Operating Environment

The suggested system follows a client–server architecture that allows authorized users to access it via a web browser without requiring additional software installation on the client side. Throughout the development process, the project team uses personal computers running

the Microsoft Windows operating system as the primary development environment. The actual project environment, however, runs on a Linux-based virtual machine (VM) that supports the backend services and security scanning tools used in the system. The users will interact with the system through the common web browsers like Google Chrome, Mozilla Firefox, and Microsoft Edge. For the efficient operation of the system, easy access to the core functionalities related to the discovery of assets, vulnerability analysis, attack graph generation, and visualization on an interactive dashboard requires a stable internal network connection.

It is designed to be compatible with other cybersecurity and monitoring tools that could be used in the user environment, such as vulnerability scanning tools, Intrusions detection and prevention tools, and other collection tools, without causing operational conflicts.

3.2.4 Design and Implementation Constraints

There exist various constraints for designing and implementing the system that affect the design and architecture. First, the system is developed within the time frame specified for graduation projects, which is limited to the functionality to be implemented rather than enterprise-level functionality. Secondly, the system is developed and runs as a web-based system with an academic hosting environment on a local server. Therefore, it is constrained in terms of its scalability and high availability settings typically found on commercial systems. Third, the project does not incur any costs, as all the development elements of the project are dependent on open-source solutions such as programming languages, database management, and cyber analysis tools, among others. This will directly affect the technologies used in the project since advanced technologies are usually offered for a fee.

Further, the choice of languages and concepts, frameworks, and development tools is also based on the project team's area of expertise and academic requirements. Thus, ensuring project duration and maintainability. These four requirements must be satisfied while meeting additional requirements for cybersecurity best practices for a system supporting user authentication, access controls on roles, and protection for processing sensitive information, which may add constraints on system design choices.

3.2.5 Design and implementation Dependencies

The effectiveness of the implementation of the suggested system would depend upon the stability and availability of the open-source security tools and vulnerability databases that would be used for the purpose of asset identification and vulnerability analysis. In addition to the previous factors, the system would also depend upon the backend processing environment and the database that would store the results for the identification of the discovered assets, identified vulnerabilities, and analysis results.

3.2.6 Workflows

Figure 1 presents the workflow, which starts with defining the assessment scope by an authorized SOC user. Where the security analyst identifies the target assets to be evaluated using domains or IP ranges. Once the scope is defined, the system initiates the scanning process to identify all assets present on the network. After listing the assets, the system extracts the banners and metadata from the discovered assets, including service information and software details, followed by verification to make sure that all the required metadata has been collected; if there is missing information, the system proceeds to extract the remaining metadata. Once the data is complete, the system classifies assets based on their type and functionality [5].

The classified assets are then prioritized according to their exposure level and potential risk. Next, the system validates the identified vulnerabilities to confirm exploitability and reduce false positives. The validated data is then prepared during the mobilization phase for attack modeling [8]. Following this, the output from this stage becomes the entrance to the Attack Graph, where the system builds the attack graph, performs the extraction of the early attack path without the need to complete the environmental graph, and applies immediate monitoring to track the stability of the graph. The identified attack paths are then evaluated for risk. Finally, all findings and recommendations are displayed in the dashboard for analyst review [21].

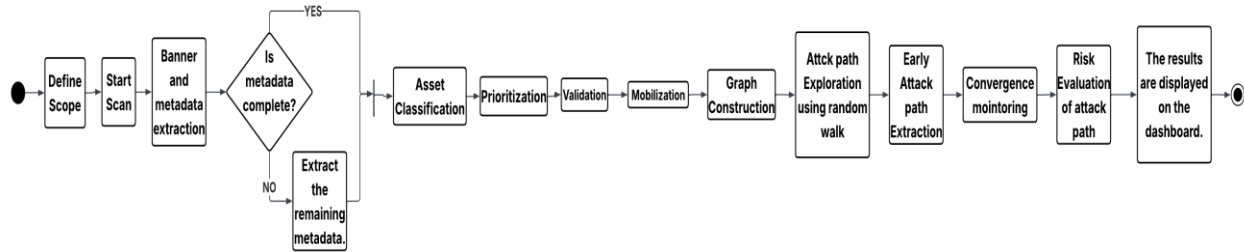


Figure 1: Workflow of the ASM and CTEM-Based Attack Graph Generation and Analysis

3.3 Requirements Specifications

This section presents the detailed requirements of the suggested system that define its expected functionality, security, and quality. These requirements provide a clear basis for system design, implementation, and validation.

3.3.1 User Requirements

In this section present the user requirements that provide a high-level description of the system expected functionalities from the perspective of end users and system administrators. These requirements are writing using natural language that understandable by the customer and define what the system should allow users to do, without describing how the system actually implements these functionalities.

- **UR 1.0:** The admin shall be able to create user accounts.
- **UR 2.0:** The admin shall be able to manage user accounts. Such as updating account information, assigning roles and permissions for the user and deactivating an account when necessary.
- **UR 3.0:** The user shall be able to securely login into the system, by using valid authentication credentials.
- **UR 4.0:** The user shall be able to define an authorized and accurate network scope before performing any security scan.
- **UR 5.0:** The user shall be able to view the defined assessment scope, then confirm that only authorized assets are included in the analysis.
- **UR 6.0:** The user shall be able to perform authorized external scanning activities within the approved assessment scope.
- **UR 7.0:** The user shall be able to view all discovered assets along with their associated metadata, such as asset type, services, and exposure details.
- **UR 8.0:** The user shall be able to view vulnerabilities identified that relate to each discovered asset.
- **UR 9.0:** The user shall be able to view attack paths that have been generated showing how vulnerabilities can be chained to compromise assets.
- **UR 10.0:** The user shall be able to view attack graphs that visually represent the possible attacker paths across interconnected assets.
- **UR 11.0:** The user shall be able to analyze risk evaluation results that reflected to the overall system risk level based on identified assets and vulnerabilities.
- **UR 12.0:** The user shall be able to see the vulnerabilities and attack paths prioritized, according to their severity levels, exploitability, and potential impact assessments.
- **UR 13.0:** The authorized user shall be able to access a centralized visualization dashboard, showing a comprehensive and support understandable representation of system security results.
- **UR 14.0:** The user shall be able to create security analysis reports summarizing system findings and results.
- **UR 15.0:** The user and the system administrator shall be able to access help information, demos, and usage guidelines with help information explaining how to use the system.
- **UR 16.0:** The user should be able to give feedback to the system that would support maintenance and overall system enhancement.
- **UR 17.0:** The user shall be able to securely log out of the system to terminate the active session.

3.3.2 System Requirements

In this section. The system requirements are explained, as a detailed and structured translation of the user requirements, into more specific technical behaviors and functionalities that the

system must perform. The system requirements include the description of how the system must work in order to satisfy the user expectations.

UR 1.0: The admin shall be able to create user accounts.

- SR 1.1: The system shall provide an administrative interface that would enable the administrative user to create user accounts by entering the necessary details such as the full name, and organizational email for the user.
- SR 1.2: The system shall validate the correctness, and the completeness of account creation, before saving the user account information.
- SR 1.3: The system shall enforce role-based access control to ensure that users can access system functionalities according to their assigned roles and permissions.
- SR 1.4: The system shall prevent duplicate account creation using the same organization email address.
- SR 1.5: The system shall first send an account creation confirmation email to the administrator's organizational email address before activating.
- SR 1.6: The system shall activate user accounts only after successful administrator approval through a secure confirmation process.
- SR 1.7: After administrator approval, the system shall send informational email to the user's organizational email address containing a temporary password.
- SR 1.8: The system shall protect all email communications and data transmissions between the system and users using TLS encryption.
- SR 1.9: The system shall require users to change their password when their first successful login.
- SR 1.10: The system shall record all account management actions for auditing and accountability purposes.

UR 2.0: The admin shall be able to manage user accounts. Such as updating account information, assigning roles and permissions for the user and deactivating an account when necessary.

- SR 2.1: The system shall provide a feature for administrators to assign or change the role or permission of a user account.
- SR 2.2: The system shall allow the administrator to deactivate or disable of user accounts when required.
- SR 2.3: The system shall allow administrators to update existing user account information.

UR 3.0: The user shall be able to securely login into the system, by using valid authentication credentials.

- SR 3.1: The system shall provide an interface for the user to log in with email and password input fields.

- SR 3.2: The system shall validate submitted login credentials against the system before access is granted.
- SR 3.3: The system shall deny access when invalid login credentials are provided.
- SR 3.4: The system shall require the user to complete an additional verification step in the form of entering a one-time code verification sent to the registered email address before completing the login process.
- SR 3.5: The system shall create a user session upon successful authentication.
- SR 3.6: The system shall grant access to the system functionalities to authenticated users only and authorized to access.
- SR 3.7: The system shall terminate the active session upon logout of the user from the system.
- SR 3.8: The system shall automatically terminate the active session after ten minutes of user inactivity.
- SR 3.9: The system shall offer a password reset function that must be approved by an administrator before allowing a password reset.
- SR 3.10: After approved by the administrator, a verification code shall be sent by the system to the user's registered email for password reset activation.
- SR 3.11: The system shall record login and password reset activities for monitoring and system management purposes.

UR 4.0: The user shall be able to define an authorized and accurate network scope before performing any security scan.

- SR 4.1: The system shall provide an interface that allows the authorized users to define the assessment scope, by specifying network targets such as IP addresses, IP ranges, or domain names.
- SR 4.2: The system shall validate the format, and structure of the specified scope inputs before accepting them.
- SR 4.3: The system shall allow the user to review defined assessment scope before confirming it.
- SR 4.4: The system shall prevent the initiation of any security scan unless an assessment scope has been successfully defined and confirmed.

UR 5.0: The user shall be able to view the defined assessment scope, then confirm that only authorized assets are included in the analysis.

- SR 5.1: The system shall display the defined assessment scope, including all specified network targets and assets to the user.
- SR 5.2: The system shall allow the user to review the assessment scope details, then beginning any analysis or perform scanning activities.
- SR 5.3: The system shall allow the user to confirm the assessment scope after review.

- SR 5.4: The system shall record the user's confirmation of the assessment scope, for supporting traceability and accountability.

UR 6.0: The user shall be able to perform authorized external scanning activities within the approved assessment scope.

- SR 6.1: The system shall allow the user to start external scanning activities on the approved assessment scope.
- SR 6.2: The system shall restrict scanning activities to targets that are included within the confirmed assessment scope only.
- SR 6.3: The system shall present the status of the external scanning process to the authorized user during execution.
- SR 6.4: The system shall store the results of the external scanning process for using in analysis and reporting stages.

UR 7.0: The user shall be able to view all discovered assets along with their associated metadata. Including asset type, services, and exposure details.

- SR 7.1: The system shall display a listing of all discovered assets, and findings within the authorized and approved assessment scope.
- SR 7.2: The system shall display the metadata that associated with each asset, including asset type, IP address, domain or hostname, open ports and exposed services, asset exposure level, and discovery timestamp.
- SR 7.3: The system shall validate the completeness of the collected asset data ensuring that all required asset attributes and metadata are fully captured.
- SR 7.4: The system shall automatically trigger an additional scanning process if the collected asset data is identified as incomplete or missing required attributes.
- SR 7.5: The system shall display the discovered assets by using a clear, structured, and searchable user interface.
- SR 7.6: The system shall provide the ability for the user to filter, sort, and search assets by attributes such as asset type, exposure level, IP address, or risk classification.
- SR 7.7: The system shall provide detailed information that related to an asset when selected from the asset list by the user.
- SR 7.8: The system shall update the displayed asset information as new assets are discovered or existing asset data changes.

UR 8.0: The user shall be able to view vulnerabilities identified that relate to each discovered asset.

- SR 8.1: The system shall be capable of detecting the vulnerabilities that have been found for each asset and displaying them.

- SR 8.2: The system shall display information for each vulnerability, including vulnerability identifier, vulnerability name, level of severity, description, affected asset, and potential impact.
- SR 8.3: The system shall display the respective value for CVSS for each vulnerability found.
- SR 8.4: The system shall support a clear and accurate association mapping between vulnerabilities and related assets.
- SR 8.5: The system shall give the capability to filter and sort the vulnerability data based on the level, asset, or category.
- SR 8.6: The system shall validate detected vulnerabilities to confirm their existence and reduce false positives before they are marked as confirmed and used in risk analysis or in the attack path generation.
- SR 8.7: The system shall offer summary indicators highlighting critical and high-risk vulnerabilities.

UR 9.0: The user shall be able to view attack paths that have been generated showing how vulnerabilities can be chained to compromise assets.

- SR9.1: The system shall provide generated attack paths that represent a combination of vulnerability and asset, which could be used to attack target assets.
- SR 9.2: The system shall provide the sequential list of involved assets and vulnerabilities for each attack path.
- SR 9.3: The system shall provide users with the ability to view detailed information for each step within an attack path that include the associated asset and vulnerability.
- SR 9.4: The system shall provide the user with the ability to view detailed information for each step within an attack path, including the associated asset and vulnerability.
- SR 9.5: The system shall allow the user to navigate between different attack paths generated for the same assessment scope.
- SR 9.6: The system shall provide summary information for each attack path, such as the number of steps involved and the affected target asset.

UR 10.0: The user shall be able to view attack graphs that visually represent the possible attacker paths across interconnected assets.

- SR 10.1: The system shall be able to generate and show attack graphs that indicate connections between assets, vulnerabilities, and possible paths for attackers.
- SR 10.2: The system shall show how assets, vulnerabilities, or their association can be graphically displayed through attack graph nodes and edges
- SR 10.3: The system shall enable the user to view multiple attack graphs generated for the same assessment scope.
- SR 10.4: The system shall allow the user to interact with the attack graph by selecting nodes or edges to view related asset or vulnerability details.

UR 11.0: The user shall be able to analyze risk evaluation results that reflected to the overall system risk level based on identified assets and vulnerabilities

- SR 11.1: The system shall display the overall system risk level derived from identified assets and associated vulnerabilities.
- SR 11.2: The system shall present risk evaluation results, presenting in a clear and understandable format suitable for user analysis.
- SR 11.3: The system shall display risk levels at different levels of detail that include overall system risk and asset-level risk.
- SR 11.4: The system shall provide a visual indicator, such as labels, charts and tables to represent different risk levels.

UR 12.0: The user shall be able to see the vulnerabilities and attack paths prioritized, according to their severity levels, exploitability, and potential impact assessments.

- SR 12.1: The system shall offer a list of vulnerabilities identified based upon priority.
- SR 12.2: The system shall provide a priority list of attack paths produced, according to their impact level.
- SR 12.3: The system shall enable users to filter or sort prioritized vulnerabilities or attack paths according to their attributes.
- SR 12.4: The system shall provide a way of graphically drawing attention to high priority vulnerabilities and attack paths.

UR 13.0: The authorized user shall be able to access a centralized visualization dashboard, showing a comprehensive and support understandable representation of system security results.

- SR 13.1: The system shall present a summarized view of the identified assets, vulnerabilities, valid attack paths, and risk level on the dashboard.
- SR 13.2: The system shall employ the use of visual elements such as charts, graphs, indicators, and tables within the dashboard to improve the understanding of the results.
- SR 13.3: The system shall be able to change the dashboard dynamically based on the new data being collected or changes in the results of analysis.
- SR 13.4: The system shall enable interaction with the elements on the dashboard interface for the purpose of allowing the users to view more information.
- SR 13.5: The system shall restrict access to the visualization dashboard to authorized users only.

UR 14.0: The user shall be able to create security analysis reports summarizing system findings and results.

- SR 14.1: The system shall provide a reporting feature that allows users to generate security analysis reports.

- SR 14.2: The system shall generate reports that are based on analysis results of the discovered assets, identified vulnerabilities, validated attack paths, and evaluated risk levels.
- SR 14.3: The system shall allow users to view previously generated reports.
- SR 14.4: The system shall present generated reports in a clear structured and understandable format suitable for all users including technical and non-technical.
- SR 14.5: The system shall include summary sections, and detailed analysis sections within each generated report.
- SR 14.6: The system shall allow users to export generated reports in common document formats such as PDF and CVEE.

UR 15.0: The user and the system administrator shall be able to access help information, demos, and usage guidelines with help information explaining how to use the system.

- SR 15.1: The system shall offer an administrator's manual describing how to configure, manage, maintain, and control access to the system.
- SR 15.2: The system shall offer demonstrations of how it may be used and features of the system.
- SR 15.3: The system shall make user and administrator guidelines accessible from within the system interface.
- SR 15.4: The system shall allow for understanding of the guidelines and demonstrations by users with differing levels of technical knowledge.

UR 16.0: The user should be able to give feedback to the system that would support maintenance and overall system enhancement.

- SR 16.1: The system shall provide a dedicated feedback interface accessible from within the system.
- SR 16.2: The system should support users in giving various kinds of feedback, such as system-related complaints and suggestions for improvement.
- SR 16.3: The system must provide storage for all feedback submitted into a central repository for purposes of maintenance and analysis.
- SR 16.4: The system should enable authorized system administrators to view user feedback submissions.

UR 17.0: The user shall be able to securely log out of the system to terminate the active session.

- SR 17.1: The system shall provide a mechanism for the user to completely exit the current session.
- SR 17.2: The system destroys the active user session after logout access is completed.
- SR 17.3: The system should redirect the user to the login page after a successful logout.

3.3.3 Security Requirements

Recent studies in secure software engineering highlight the importance of integrating security considerations early in the software development lifecycle through systematic approaches such as threat modeling, risk assessment, and security requirements engineering [25]. For the suggested system, a SQuaRE approach is adopted to ensure that security and quality requirements are identified. Furthermore, multiple mapping studies indicate that security-driven and risk-aware methodologies are among the most widely adopted approaches for developing secure software system [26]. As shown *Figure 2* the steps of the lightweight SQuaRE approach that are followed in the suggested system to derive the security requirements.

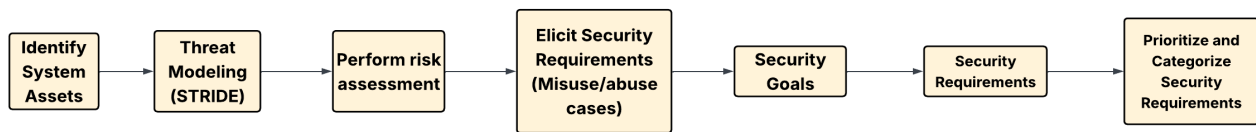


Figure 2: Steps of the Lightweight SQuaRE Approach

Identification of Critical Assets

Critical asset identification is a fundamental step in security requirements engineering and risk-based security [27]. As part of the SQuaRE-Lite security requirements engineering process, critical system assets are identified to understand what needs to be protected against potential threats. as it enables to emphasize that threats, risks, and attack scenarios are inherently associated with the assets they target [25]. The suggested system handles sensitive security-related information and performs critical analysis tasks; therefore, the following assets are considered essential and must be protected against unauthorized access, modification, or misuse.

The identification of critical assets:

- Discovered assets lists:
That include a list of discovered assets, such as hosts, services, network components, and endpoints.
- Vulnerability information
It contains the data that related to identified vulnerabilities, like vulnerability identifiers, severity scores, and affected assets.
- Attack Graph & Attack Paths
Represent the paths that show how vulnerabilities can be linked together and used to compromise system resources.
- Prioritization outcomes and risk scores.

Including all finding such as the security risks, vulnerabilities, and attack paths are evaluated and prioritized by the calculated value user according to their possible impact and likelihood.

- User Accounts & Role Information

Definitions and authentication credentials for system users, including security analysts and administrators.

- Security Dashboard & Reports

Security findings are presented through analytical and visual outputs that aid in decision-making.

- System Logs & Audit Records

Accountability, traceability, and incident analysis are supported by records of system activity and user actions.

- Assessment Scope Configuration

Defined scanning scope information, including authorized targets, IP ranges, domains, and scanning boundaries that control and limit security assessment activities.

The following *Table 3* identified assets represent the core components and sensitive data of the suggested system and protecting these assets is essential to ensure accurate risk assessment, and secure system operation.

Table 3: entities Critical Assets for Threat Modeling and Risk Analysis

Asset	Asset Value	Security property (CIA)
Discovered Assets lists	High	Confidentiality, Integrity
Vulnerability Information	High	Confidentiality, Integrity
Attack Graph & Attack Paths	High	Integrity
Prioritization outcomes and risk scores.	High	Integrity
User Accounts & Role Information	High	Confidentiality, Integrity
Security Dashboard & Reports	High	Integrity, Availability
System Logs & Audit Records	High	Integrity, Availability
Assessment Scope Configuration	High	Integrity, Confidentiality

Misuse and Abuse Cases

The identified misuse and abuse cases are mapped to the STRIDE threat categorizes and focus on protecting the conditionality, integrity, and availability of the system critical assets. Misuse cases help developers to think like an attacker used to describe potential malicious or unintended actions that may compromise system security. An abuse case identifies interactions

between actors and a system that are intended to cause harm, and aim to describe how the system must counter such abuse.

- MC1: Unauthorized access to asset inventory
An unauthorized user gains access to the discovered asset inventory to obtain sensitive information about network infrastructure.
- MC2: Manipulation of Vulnerability Information
An unauthorized user modifies vulnerability records, severity scores, or asset associations to hide critical weaknesses.
- MC3: Tampering with Attack Graphs
An attacker alters generated attack graphs or attack paths to distort the representation of potential attack scenarios and risk prioritization.
- MC4: Unauthorized Modification of System Configuration
An unauthorized user changes system configuration parameter
- MC5: Information Disclosure through Dashboards and Reports
Sensitive security findings are accessed or viewed by unauthorized users through system dashboards or generated reports.
- MC6: Sending Security Reports to Unauthorized Parties
An authorized user intentionally or unintentionally sends generated security reports to external or unauthorized parties.
- MC7: Abuse of Scanning Capabilities on Unauthorized Targets
An authorized user scans assets or domains that aren't supposed to be scanned, and this generates a false alert or misuse of resources or legal problems.
- MC8: Denial of Service (DoS or DDoS) Attack The system gets flooded with requests from the attacker.
- MC9: Web Application Attacks.
An attacker goal to exploits the common vulnerabilities as identified by the OWASP, such as SQL injection, Cross-Site Scripting (XSS), to manipulate input data and compromise backend databases.
- MC10: Manipulation of Asset Data through Web Attacks

An attacker alters asset-related data via web-based attacks, causing incorrect asset discovery results and unstable system behavior.

- MC11: Privilege Escalation Attack
System weaknesses or misconfigurations are taken advantage of by a user to gain elevated levels of system privilege unauthorized, resulting in unauthorized actions.
- MC12: Repudiation of Security-Critical Actions.
A user denying performing actions such due to lack of logging or audit mechanisms.
- MC13: Abuse of Authentication and OTP Verification Mechanism
An attacker repeatedly asks for OTP verification codes or authentication attempts to utilize system resources, disrupt legitimate user access, or cause denial of service.

Security goals

Security goals express the general security requirement that the system has to fulfill in order to protect its vital assets against threats and user misuse.

- SG1: Prevent unauthorized access to asset inventory.
- SG2: Prevent manipulation of vulnerability information.
- SG3: Prevent tampering with attack graphs.
- SG4: Prevent unauthorized modification of system configuration.
- SG5: Prevent information disclosure through dashboards and reports.
- SG6: Prevent sending security reports to unauthorized parties.
- SG7: Prevent abuse of scanning capabilities on unauthorized targets.
- SG8: Prevent denial of service attacks.
- SG9: Prevent web application attacks.
- SG10: Prevent manipulation of asset data through web attacks.
- SG11: Prevent privilege escalation attack.
- SG12: Prevent repudiation of security critical Actions.
- SG13: Prevent abuse of authentication and OTP verification mechanism.

Security Requirements

- SecR1: The system shall enforce secure authentication mechanisms, including two factor authentication to ensure that only authorized users can access the system.

- SecR2: The system shall lock a user account after a predefined number of failed login attempts, and require administrative action to restore access.
- SecR3: The system shall enforce role-based access control to limit actions, taken by users according to roles and permissions.
- SecR4: The system shall implement the least privilege principle so that a user can only access and use the required resources and functionality for his role.
- SecR5: The system shall validate role and privilege assignment to prevent illicit privilege escalation.
- SecR6: The system shall record all security relevant events, such as attempts for authentication, changes to configurations, scanning actions, and accesses to sensitive data.
- SecR7: The system shall be able to protect the information and metadata of the discovered assets against unauthorized access, change, and deletion.
- SecR8: The system shall ensure the vulnerability information cannot be altered by the unauthorized user.
- SecR9: The system shall restrict access to attack graphs and attack paths to authorized users and prevent unauthorized modification.
- SecR10: The system shall restrict security scanning activities to authorized users only.
- SecR11: The system shall prevent perform any scanning activities of unauthorized targets.
- SecR12: The system shall enforce per-IP rate limiting, on the login interface and API endpoints, to mitigate attacks such as denial of service.
- SecR13: The system shall be able to validate all user inputs, to mitigate web attacks like SQL injection, XSS attacks.
- SecR14: The system shall secure data transfer, by using encryption channels such as HTTPS.
- SecR15: The system shall restrict access to the generated reports and prevent disclosure to unauthorized user.
- SecR16: The system shall securely manage user sessions by expiring session identifiers upon logout, password change, and inactive session timeouts.
- SecR17: The system shall restrict the attempts for OTP generation and verification in order to avoid DoS attacks.

- SecR18: The system shall protect the audit logs from unauthorized viewing, editing, or deletion.
- SecR19: The system shall detect and verify for integrity breach of security-critical data such as assets, threats, and attack paths.
- SecR20: The system shall encrypt sensitive security-related data stored at rest, including asset data, vulnerability data, attack graphs, and audit logs, in order to prevent unauthorized access and exposure of data.
- SecR21: The system shall perform regular backups of critical security data like asset data, vulnerability data, attack graphs, and logs.
- SecR22: The system shall ensure secure storage and protection of the backup data against unauthorized access.

Security Requirements Categorization and Prioritization

This *Table 4* categorizes the security requirements based on security type and prioritizes them according to their importance and impact.

Table 4: Security Requirements Classification and Priority

SecR-ID	Description	Security Type	Priority
SecR1	Secure authentication with 2FA	Confidentiality	High
SecR2	Account lockout after failed logins	Confidentiality	High
SecR3	Role-Based Access Control (RBAC)	Confidentiality	High
SecR4	Least privilege enforcement	Confidentiality	High
SecR5	Prevent privilege escalation	Integrity	High
SecR6	Log all security-relevant events	Auditing	High
SecR7	Protect asset data and metadata	Integrity	High
SecR8	Protect vulnerability data	Integrity	High
SecR9	Restrict access to attack graphs	Confidentiality	High
SecR10	Restrict security scanning to authorized users	Integrity	High
SecR11	Prevent scanning of unauthorized targets	Integrity	High
SecR12	Rate limiting for DoS mitigation	Availability	High
SecR13	Input validation (SQLi, XSS prevention)	Integrity	High
SecR14	Secure communication (HTTPS/TLS)	Confidentiality	Medium
SecR15	Restrict access to security reports	Privacy	Medium
SecR16	Secure session management and timeout enforcement	Confidentiality	High
SecR17	Limiting OTP generation and verification attempts	Availability	High

SecR18	Protection of audit logs from unauthorized access or modification	Auditing	High
SecR19	Integrity verification of security critical data	Integrity	High
SecR20	Encryption of sensitive data at rest	Confidentiality	Medium
SecR21	Regular backup of critical security data	Availability	Medium
SecR22	Secure storage and protection of backup data	Confidentiality	Medium

3.4 Non-functional Requirements (NFR)

This section covers the basic quality attributes that determine how a system works. These attributes significantly affect the success of the system and the quality of the software.

3.4.1 Software Quality Attributes

This section describes the key software quality attributes that characterize the overall quality of the suggested system. These attributes define as non-functional aspects including performance, reliability, usability, maintainability, availability and portability that are essential for system effectiveness and user satisfaction.

Usability Requirements

Usability is considered a critical quality attribute, in security systems as poor usability can lead to user errors, and security misconfigurations.

- The system shall validate all input data entered by users. It must prompt them for specific error messages indicating the reasons for the error made by them.
- The system should offer a steady state of navigation that allows users to access all major functionalities of the system within a maximum of two clicks.

Reliability and Availability

Reliability and availability are necessary non-functional requirements for security systems that perform continuous monitoring and analysis.

- The system shall run all analysis activities for more than 24 hours without unexpected shutdowns.
- The system shall provide a minimum operational availability of 99% during the specified periods of operations.
- The system shall be able to store failure events, errors, and service interruptions within persistent log files that contain timestamps and error messages.

Scalability

Scalability is the ability of a system to perform under increased assets and user traffic without doing so at the cost of performance. It allows a system to perform well even when.

- The system shall support handling an increasing number of users concurrently without compromising the system's performance.

Maintainability

Maintainability is crucial for long-term system evolution as well as secure updates and upgrades.

- The system shall store in persistent database user feedback submitted through the system interfaces and to make it available to authorized administrators for review.
- The system shall be designed as composed of modular components, using clearly defined interfaces, so that independent modification, replacement, or extension of system components can occur without affecting the system as a whole.

Portability

Portability of software systems enables them to work in various environments.

- The system shall be implemented using standard web technologies that include PHP as the server-side scripting language and HTML, CSS, JavaScript, and
- The system shall be able to run on general-purpose computer hardware without needing any special hardware components.

Performance Requirements

The performance requirements of the suggested system, focusing on timely asset discovery, continuous scanning, efficient resource utilization, and scalability under multi-user. However, the actual system performance depending on factors, such as network size and scanning depth.

- The system shall be capable of discovering, and listing all reachable assets within a time frame measured in minutes for medium-sized network environments.
- The system shall support continuous asset scanning, with automated re-scanning. Ensuring that asset and exposure information remains up to date, in accordance with CTEM best practices.
- The system shall automatically generate attack graphs, shortly after asset discovery and vulnerability correlation are completed, supporting near real-time security analysis.
- The system shall be designed to operate efficiently, without excessive consumption of computational resources, during both normal operation, and analysis processes.
- The system shall support concurrent access by multiple security analysts, also enabling simultaneous use of dashboards, analysis results, and reports without any substantial impact on system performance.

3.5 Requirements Analysis and Modelling

This section presents the analysis and modeling of system requirements using actors, use cases, misuse cases, and activity diagrams. These models are used to clearly represent system behavior, interactions, and potential misuse scenarios.

3.5.1 Actors analysis

Table 5 identifies the main actors that interact with the suggested system describes their roles help understanding system boundaries and user interaction.

Table 5: Identifies the main actors with the suggested system describes their use cases

Actor	Description	Use Case
SOC Analyst	The primary users of the system and they interact with it daily. They must have a strong cybersecurity knowledge, including understanding of security risks, types of vulnerabilities, techniques of conducting an attack, and basic processes for incident response.	a) Login using secure authentication mechanisms. b) Request OTP for identity verification. c) Define assessment scope. d) Launch an authorized external scanning process. e) View discovered assets and associated metadata. f) Review validated vulnerabilities. g) Review generated attack graphs and attack paths. h) Review risk evaluation results and prioritized threats. i) Analyze high-risk attack paths. j) Generate security reports. k) Export security reports. l) Submit feedback to improving the system functionality. m) Access system help, demo, and user guidance features.
System Administrator	Responsible for take care of the system setup, configuration and its maintenance.	a) Update user accounts. b) Create user accounts. c) Deactivate user accounts. d) Assign roles and permissions, based on role-based access control (RBAC). e) Monitor system logs. f) Audit user activities. g) Configure system wide settings. h) Configure system scanning parameters.
OTP Service	The OTP Service provides an additional verification layer for user authentication by generating and validating time-based one-time passwords as part of the system's two-factor authentication (2FA) process.	a) Verify User Identity.

3.6.2 Use case & Misuse case Model/Diagram Analysis & Modelling

Figure 3 shows the interaction of authorized users with the proposed system, highlighting the functional use cases performed by the Security Operations Center Analyst, System Administrator, and One-Time Password (OTP) service during normal operation of the system. Authorized users

are represented in blue. The chart also shows the possible abuses by both external attackers, represented in red, and internal staff, represented in gray. Abuse by the attacker is displayed in black, abuse by the insider is displayed in gray, and related mitigation actions are shown in green, showing how security controls are applied to limit or prevent these attacks. To see the diagram more clearly, you will find the details in the *Appendix I*.

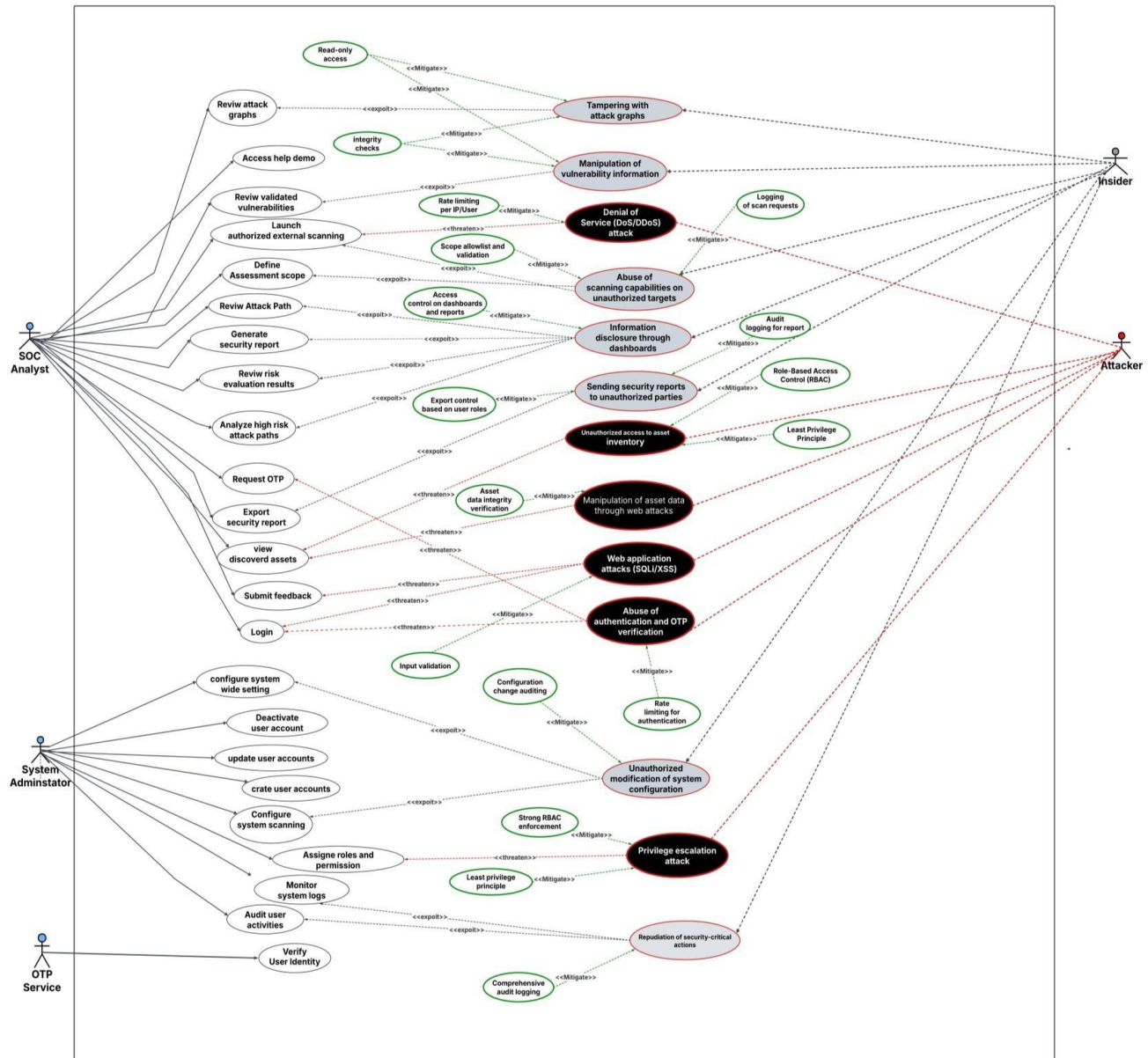


Figure 3: Integrated outline of use cases and abuses of system functions, potential security attacks and associated mitigation mechanisms

3.6.3 Use-Case Specifications

This section provides detailed specification for the most critical and important use cases of the suggested system. Each specification describes the use case goal, actors, required data, triggers, preconditions, postconditions and the normal, exceptional

execution flows. In addition, security threats and corresponding mitigation controls are included to ensure that system functionalities are analyzed from both functional and security perspectives.

Table 6 represent use case specification for UR 2.0: The user shall be able to securely log into the system using valid authentication credentials.

Table 6: Detailed Use Case Specification for UR-02 – Secure login

System Name	Secure Network Asset Discovery and Vulnerability Assessment System (ASM/CTEM).
Use case Title	Secure login to the system
Summary	The user securely logs into the system using valid authentication credentials, and two factor authentication using OTP.
Description	This user case describes the authentication process that allows authorized users to access the system securely. The system verifies user credentials, enforce OTP verification and creates a secure session. Also, all login attempts are monitored and logged for security auditing.
Actors	Authorized system users (SOC analyst, system Administrator) OTP service
Data	Input data: username, email, password, OTP code. Output data: login success, login failure, authenticated session ID
Triggers	The user selects login in login interface and submits authentication credentials.
Precondition	1. The system is available. 2. The user ha active account. 3. The login page is accessible
Postcondition	Success: 1. The user is authenticated and granted access according to assign role and permissions. 2. A secure session in created 3. The successful login recorded in audit log. Failure: 1. The user in not authenticated. 2. The system displays an error message without revealing sensitive details. 3. The failed attempt is recorded and additional protection be applied.
Basic flow	1. The user open login page. 2. The user enters email and password then submits. 3. The system request OTP verification. 4. The OTP service sends to the user email.
Alternative flow	If the OTP is not received, the user requests Resend OTP then the system sends a new OTP within defined limits.
Exception flow	1. The user enters incorrect email or password. The system rejects the login and displays a generic failure message. 2. When the account is deactivated, then the system denies access and informs the user to contact the administrator. 3. The user enters an incorrect OTP, or the OTP expires. Then the system rejects verification and allows retry within limits. 4. Multiple failed login or OTP attempts occur. The system temporarily blocks the user IP.
Exception points	After a defined number of failed attempts, the system activates protection (rate limiting). OTP codes expire after a 60 second and cannot be reused. Sessions expire automatically after inactivity, and secure tokens are enforced.
Threats	1. Brute force attack by repeated login attempts to guess credential. 2. OTP abuse by repeated OTP requests or OTP guessing. 3. Session hijacking by stealing session tokens 4. Phishing by attempting to trick users into reveling OTP credentials.
Mitigations	• Apply rate limiting and temporary lockout after repeated failed attempts.

	• Enforce strong password policy and store password using hashing with a unique salt. • Enable two factor authentication • Use secure session management • Log all authentication events
Comments	Secure login is a critical entry point to the system. Therefore, multi-layer authentication necessary to reduce unauthorized access risks.

Table 7 represent use case specification for UR 3.0: The user shall be able to define and confirm an authorized and accurate network scope before performing any security scan.

Table 7: Detailed Use Case Specification for UR-03 -Define & Confirm Assessment Scope

System Name	Secure Network Asset Discovery and Vulnerability Assessment System (ASM/CTEM)
Use case Title	Define and Confirm Assessment Scope
Summary	The user defines and confirms an authorized and accurate assessment scope before performing any security scan to ensure that only approved systems are included.
Description	The use case describes how the system allows an authorized user to configure the scan by specifying target assets and validating that the scope is authorized and correctly formatted. The define scope must be confirmed before scanning to prevent misuse and ensure accurate analysis.
Actors	SOC Analyst
Data	Input data: scope name, target scope Output data: scope validation result (accepted/rejected). Stored data: scope configuration, scope target list and audit logs.
Triggers	The user selects Assessment Scope and chooses create scope before launching a scan.
Precondition	1. The system is available. 2. The user is authenticated and authorized 3. Scope management module is accessible.
Postcondition	Success: 1. A valid scope is created and stored. 2. The scope is confirmed as authorized for scanning. 3. The action is recorded in audit logs Failure: 1. No scope is created and stored. 2. The system shows validation errors without sensitive information or authorization failure messages
Basic flow	1. The user opens the scope management interface. 2. The system displays option to create a new scope. 3. The user selects create scope. 4. The system displays a scope configuration form. 5. The user enters scope information (scope name, targets such as ranges/domains/subnet and optional notes). 6. The system validates the entered targets. 7. The system checks that the target are authorized according to system policy. 8. The system displays a scope summary for confirmation. 9. The user reviews the scope, then confirm it. 10. The system stores the scope as approved and enables scanning only for this confirmed scope if it authorized. 11. The system records the scope creation, and confirmation actions in audit logs to support accountability and non-repudiation.
Alternative flow	Reuse existing scope, the user selects a previously confirmed scope and reviews it without modifications, then proceeds to scanning.

Exception flow	- Invalid target format the user enters an invalid IP, domain, subnet format. The system rejects the input and requests correction. - Unauthorized targets the user adds targets outside approved boundaries. The system blocks confirmation and shows an authorization error without sensitive information. - Duplicate the scope contains duplicates or overlapping ranges that may cause inaccurate scans. The system rejects and requests cleanup.
Exception points	- The system shall prevent scanning unless the scope status is Confirmed or Approved. - The system shall enforce an allow list policy (authorized targets only). - The system shall validate input formats. - The system shall log all scope changes and confirmations for accountability.
Threats	- Abuse of scanning on unauthorized targets. - Insider misuse by adding external targets intentionally. - Scope tampering to hide assets or distort assessment results. - Input manipulation, via exploits the web attacks such as SQL injection, if validation is weak.
Mitigations	- Enforce scope allow list, and organization boundary validation before confirmation the target scope. - Apply RBAC, and least privilege principle, only authorized roles can create and confirm scopes. - Implement strong input validation, and secure coding for all scope fields. - Maintain audit logs for who created, updated, confirmed scopes and when to support accountability and non-repudiation. - Block scan execution unless scope is confirmed and approved.
Comments	Scope definition is critical step because it controls what the system is allowed to scan. by strict authorized checks and validation help prevent misuse, and avoid scanning unauthorized targets

Table 8 represent use case specification for UR 9.0: The user shall be able to view attack graphs that visually represent possible attacker paths across interconnected assets.

Table 8: Detailed Use Case Specification for UR-09 -View Attack Graph

System Name	Secure Network Asset Discovery and Vulnerability Assessment System (ASM/CTEM)
Use case Title	View Attack Graphs and Associated Attack Paths.
Summary	The user views attack paths that visually represent possible attacker paths across interconnected assets, based on discovered assets, identified vulnerabilities, and the system relationships.
Description	This use case describes how the system generates and presents an interactive attack paths to help the user understand potential attack progression across the environment. The user can view nodes (assets), edges (relationships), and associated vulnerability information, and inspect related attack paths to support risk-based decision-making. Also, access to attack paths is restricted due to the sensitivity of the information.
Actors	SOC Analyst.
Data	Input: - CTEM-processed and normalized data including validated prioritized vulnerabilities, - Associated assets and their metadata (e.g., asset type, services, and exposure details). Output data - Attack graph visualization representing using nodes and edges, attack path list, graph summary metrics (e.g., number of nodes, edges, highest-risk paths). Stored data: - Generated attack graph structure, attack paths, asset relationships, vulnerability metadata, user access logs.

Triggers	The user selects Attack Path generation from the dashboard.
Precondition	1. The system is available. 2. The user is authenticated and authorized and have permissions allow attack path access. 3. At least one successful scan result exists for the selected scope (assets and associated vulnerabilities available). 4. Attack graph data for the selected scan is available.
Postcondition	Success: 1. The attack graph and related attack paths are displayed to the user. 2. The user can inspect graph elements (assets, vulnerabilities, paths) and apply filters. 3. The system records the access and viewing event in audit logs. Failure: 1. The graph is not displayed no data or unauthorized. 2. The system shows an appropriate error message without leaking sensitive details.
Basic flow	1. The system receives the list of discovered assets and validated vulnerabilities. 2. The system creates initial node with an attacker node and external assets nodes 3. The system models vulnerabilities as exploitation transitions between system states 4. The system executes weighted random walk sampling starting from attacker node 5. The system incrementally expands the attack graph as new exploitation paths are discovered. 6. The system identifies attack path before full graph construction completed 7. The system monitors the construction process for convergence to reduce the compactional overhead and avoid it. 8. The system stores the constructed attack graph and all discovered attack paths.
Alternative flow	If the graph is already generated, the system loads it directly.
Exception flow	• No scan data because there no completed scan results exist for the selected scope. The system notifies the user to run a scan first. • Unauthorized access the user lacks permission to view attack graphs so the system denies access. • The system fails to generate the graph due to incomplete data or internal error.
Exception points	The system shall restrict access to attack graphs using RBAC and least privilege.
Threats	• Information disclosure, unauthorized users view attack graphs and learn network structure and weaknesses. • Tampering with attack graphs an insider attempts to modify graph data to hide risky paths or distort prioritization. • Injection attacks via filters or inputs malicious input through graph filtering • Privilege escalation. A user attempts to bypass access controls to view restricted graphs. • Repudiation actions. A user denies accessing sensitive attack graph data if auditing is insufficient
Mitigations	• Enforce RBAC and least privilege for attack graph viewing and exporting. • Apply read-only access for graph data for non-admin users; restrict regeneration/modification operations. • Implement audit logging for attack graph access (who, when, which scope, which scan). • Apply input validation for any filters, search fields used in graph interactions.
Comments	Attack graphs contain highly sensitive security information therefore access control, auditing are essential to prevent misuse and ensure trustworthy risk analysis.

3.6.4 Activity Diagram

The activity diagram describes the overall workflow process of the proposed system, from secure user login to the end of the session as shown in *Figure 4*.

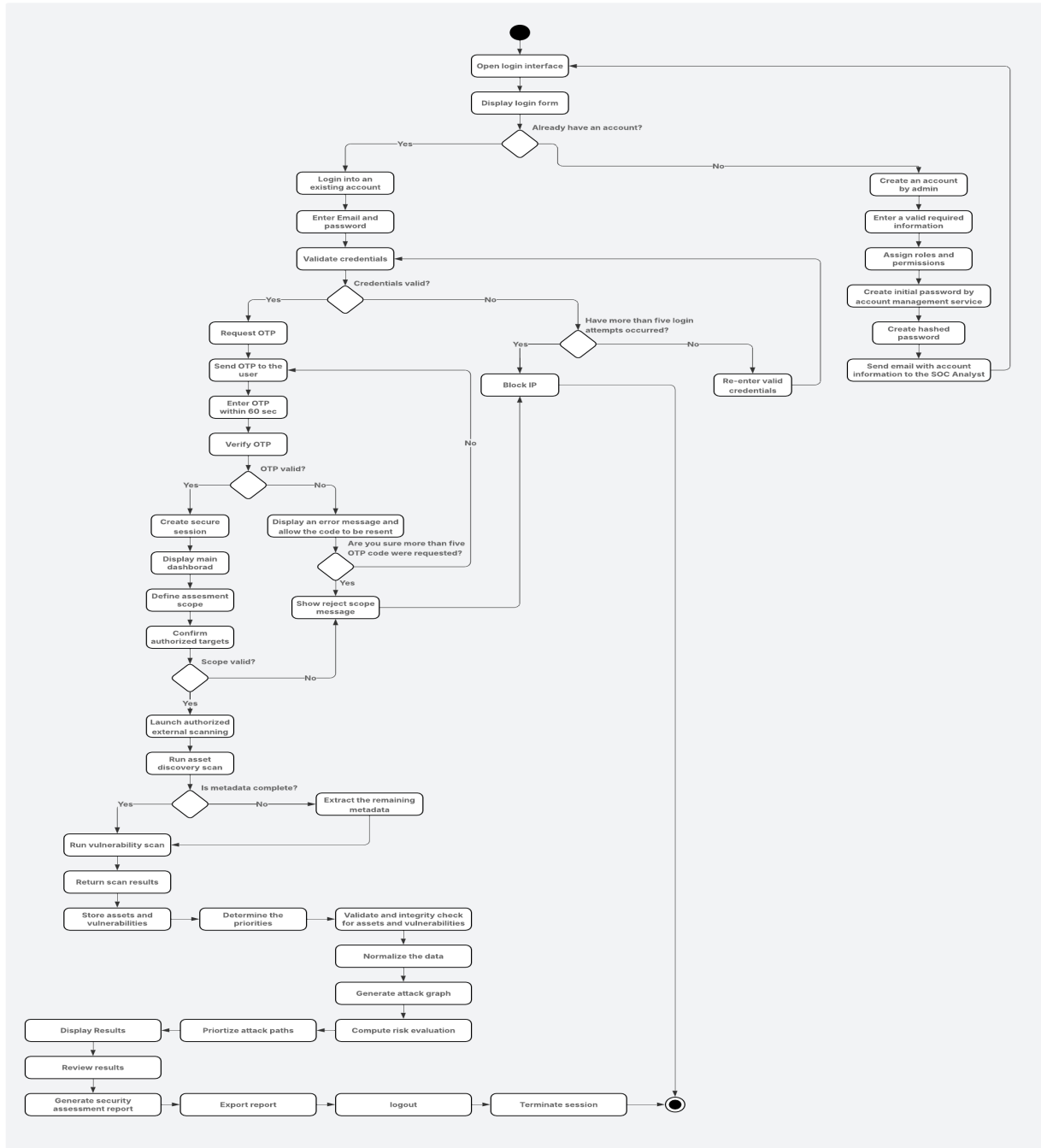


Figure 4: Activity diagram showing the core system workflow

CHAPTER 4

SOFTWARE DESIGN

4.1 Design Overview

This chapter presents the software design of the suggested system. The design follows a modular and layered architectural approach to ensure scalability, maintainability and security.

4.1.1 Introduction

The system is based on a modular design methodology, where the system is divided into independent functional components so that each has an independent function, which makes it easy to use and maintain, and supports the safe separation of responsibilities between system models.

In addition, a web-based client-server architecture is also adopted as in *Figure 5* where users interact with the system through browser-based interfaces. Also, the core processing tasks including asset discovery, vulnerability analysis and attack graph generation are handled by back-end server. This separation enhances security analysis process and allows centralized management of data analysis.

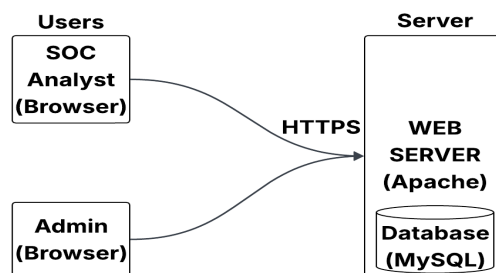


Figure 5: High-level client-server architecture of the proposed system

4.1.2 Environment Overview

The suggested system is deployed within a web-based client server environment, where end user including SOC Analysts and Administrators, access the system through web browser running on client devices. The client side responsible only for presenting the user interface, while all core system functions, such as asset discovery, vulnerability analysis, and attack scheme creation, are executed on a back-end server that hosts web server and database services. The communication between the client device and the server established over HTTPS to ensure secure data transmission as illustrated in *Figure 5*.

In addition, the system interacts with the external environment to retrieve vulnerability information from external CVE and vulnerability databases, this deployment approach ensures centralized processing, improved security, and efficient management of system resources.

4.1.3 Constraints, Assumptions and Dependencies

This project was developed based on several assumptions and constraints that may affect the system and its behavior, performance and deployment environment.

Assumptions:

- The system is deployed as a web-based application, and accessed via web browsers.
- Organizations using the system possess externally visible assets on the internet.
- Security analysts and administrators have knowledge of cybersecurity principles, such as vulnerability assessment, and attack paths.
- The operating environment permits outbound network scanning of externally exposed assets.
- Open-source scanning tools used by the system are available and remain effective while system is running.

Constraints:

- The system depends on a set of open-source Attack Surface Management (ASM) and vulnerability scanning tools, such as Nmap, Masscan, Amass, and OpenVAS, to support asset discovery and vulnerability detection processes which may impose limitation related to tool accuracy, coverage and performance.
- The system performance may be affected by the number of assets under analysis and complexity if generated attack graphs.
- The scanning and analysis process are subject to legal and ethical constraints and must be limited to authorized and predefined targets only.

Dependencies:

- The attack graph generation process depends on the availability CVE and CVSS datasets to enhance vulnerabilities and calculate risk scores.
- The system relies on open-source software libraries, supporting the attack graph generation, random walk sampling, and attack path analysis functionalities.
- A backend processing environment with a persistent database are required to store and manage information that related to discovered assets, identified vulnerabilities, generated attack paths, and analysis results.

Any changes to these assumptions, constraints or dependencies could affect the system's accuracy, performance, or availability.

4.2 System Design Models

This section presents the system design models used to describe the structure and behavior of the suggested system. Different UML diagrams are employed to provide

4.2.2 Class Diagram using Secure UML

The class diagram based on the suggested system using the Secure UML class diagram method is introduced, in which security-related concepts such as roles, permissions, and actions are extended into the class diagram. *Figure 7* represents a secure UML class model for the system. To see the graphic more clearly, you will find the details in the *Appendix 3*.

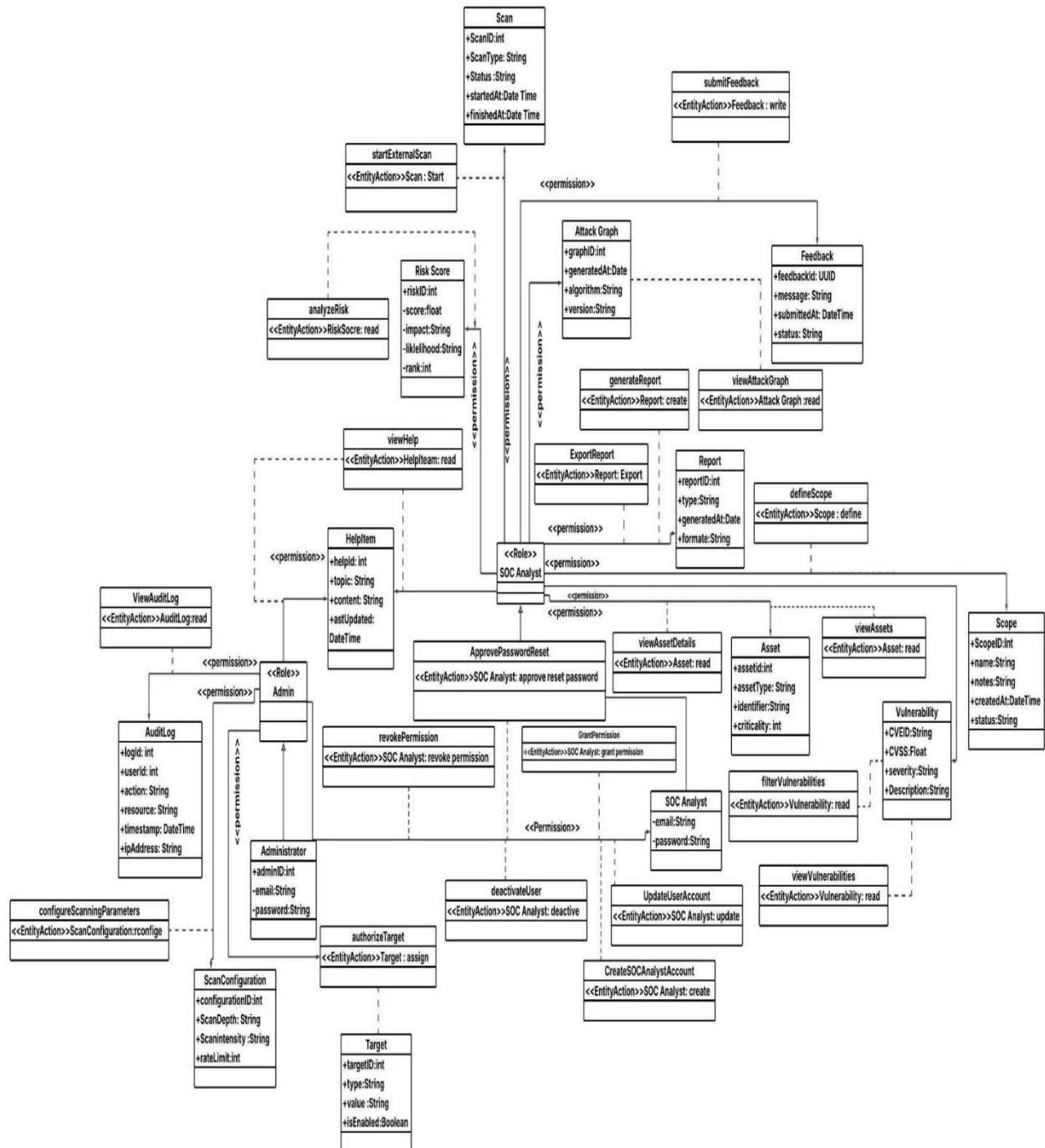


Figure 7: Secure UML class diagram illustrating roles, permissions, and protected system actions

4.2.3 Sequence Diagrams

Sequence diagrams are used to represent key use cases and show the flow of requests and responses between the user interface, back-end services, and data management components.

Sequence Diagrams for secure login use case

Figure 8 displays the sequence diagram for the use case secure login, representing the interaction flow among the users (SOC analyst and administrator) and various components during the entire course of authentication and access to the system resources.

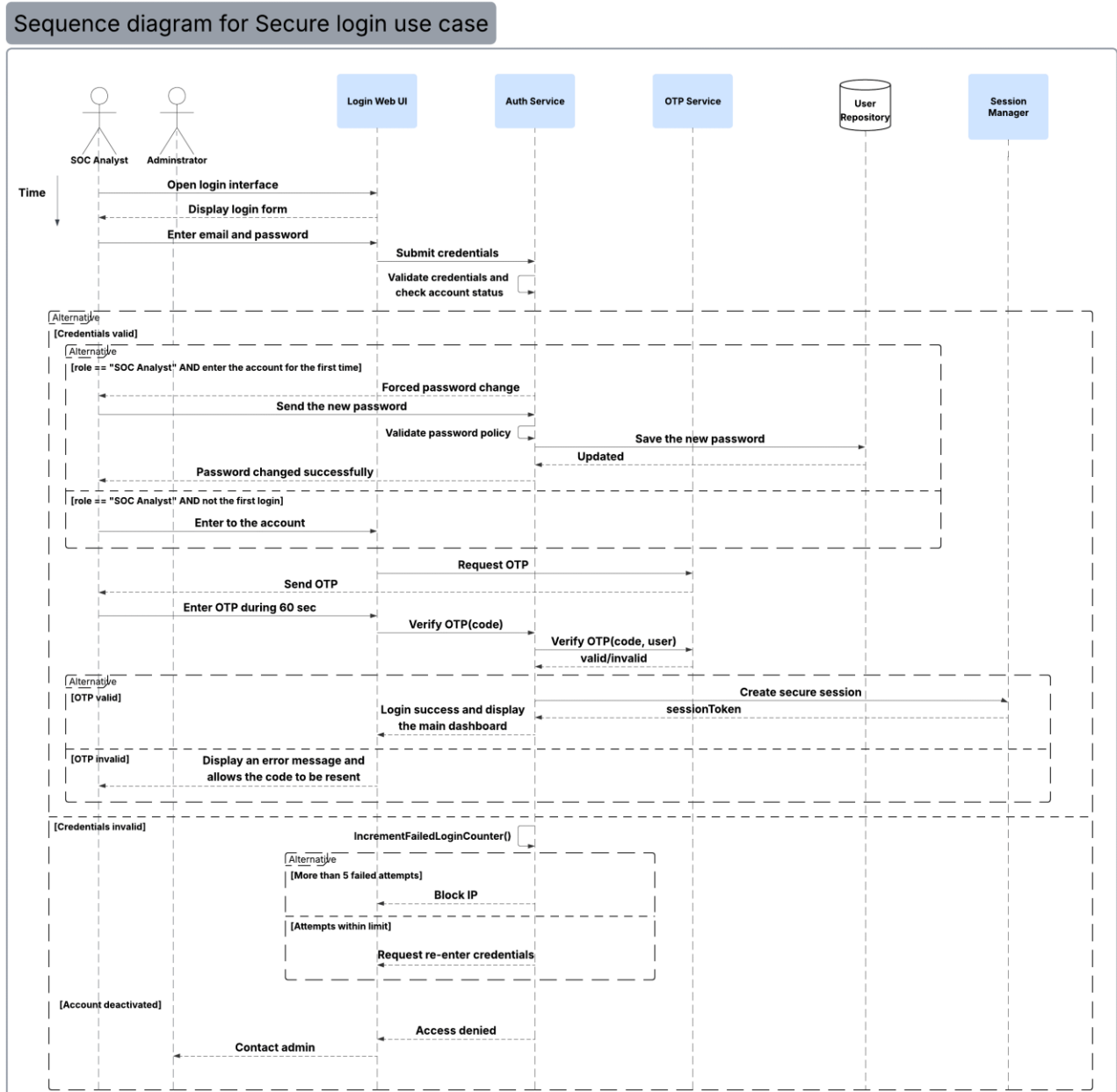


Figure 8: Sequence diagram for the secure login use case

Sequence Diagrams for Managing user Accounts securely

Figure 9 displays the sequence diagram for the management of user accounts securely by outlining the interaction between the administrator, SOC analyst, and the system components during the management of user accounts.

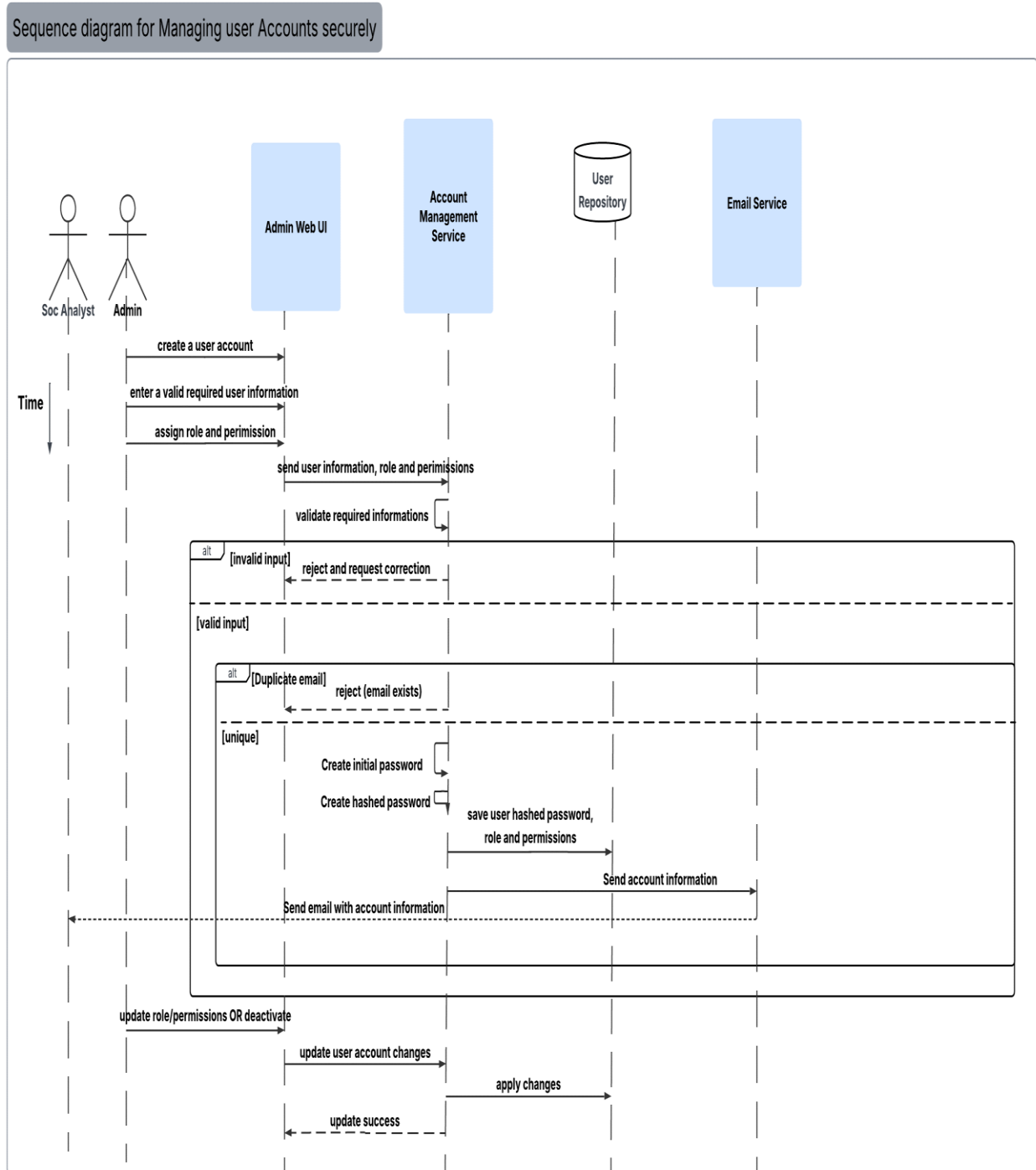


Figure 9: Sequence diagram for secure user account management

Sequence Diagrams for Define and Confirm Assessment Scope

Figure 10 displays the sequence diagram for defining and confirming the assessment scope.

Sequence diagram Define and Confirm Assessment Scope

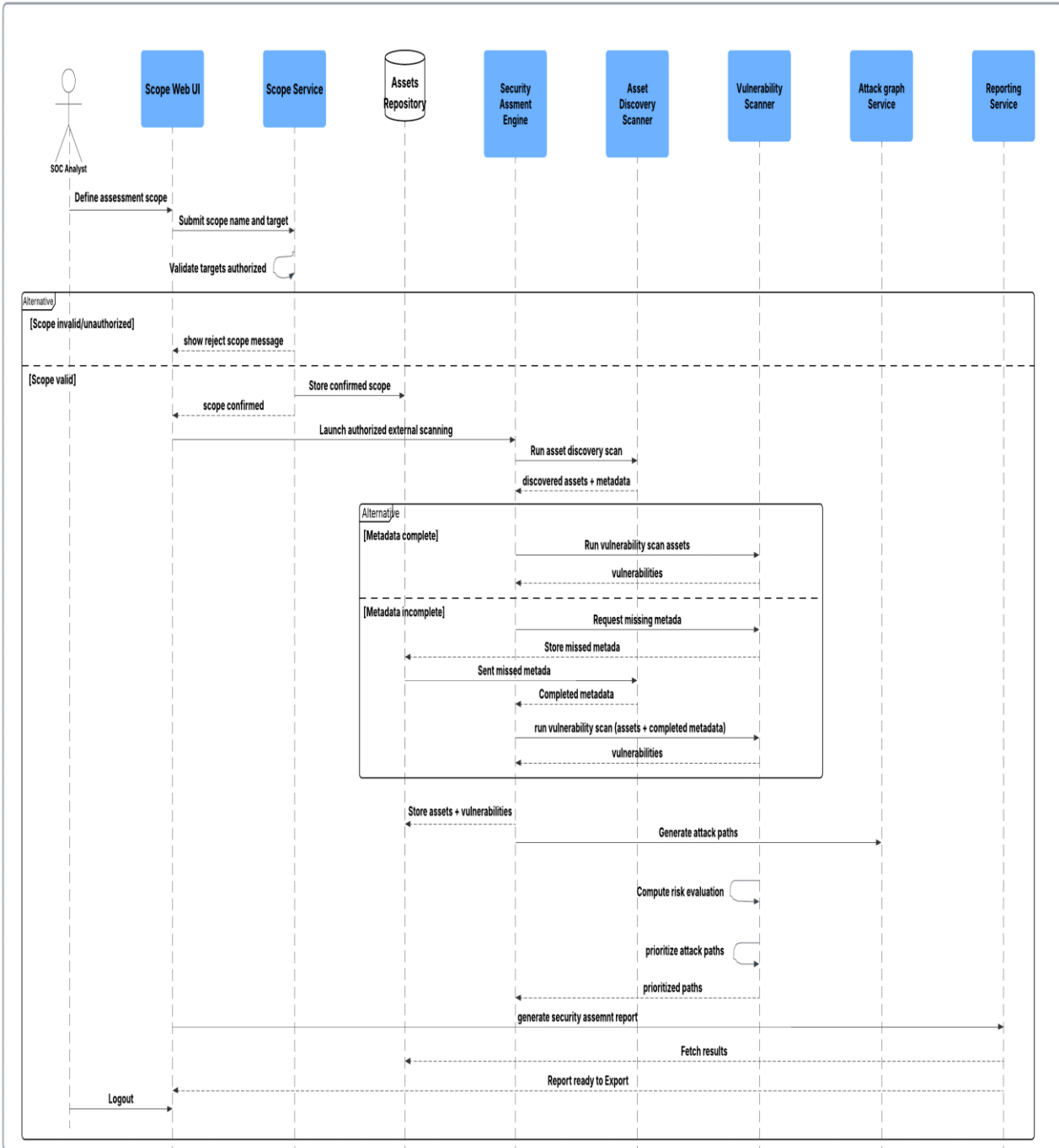


Figure 10: Sequence diagram showing the workflow of defining and confirming the assessment scope and executing the security assessment process.

4.2.4 State Diagram

The state diagram is used to model the dynamic behavior of the system, by the different states. The system can be in and the transitions between these asset states based on specific events. It provides a clear view of how the system responds to user actions, security conditions, and processing steps throughout its lifecycle, as shown in *Figure 11*.

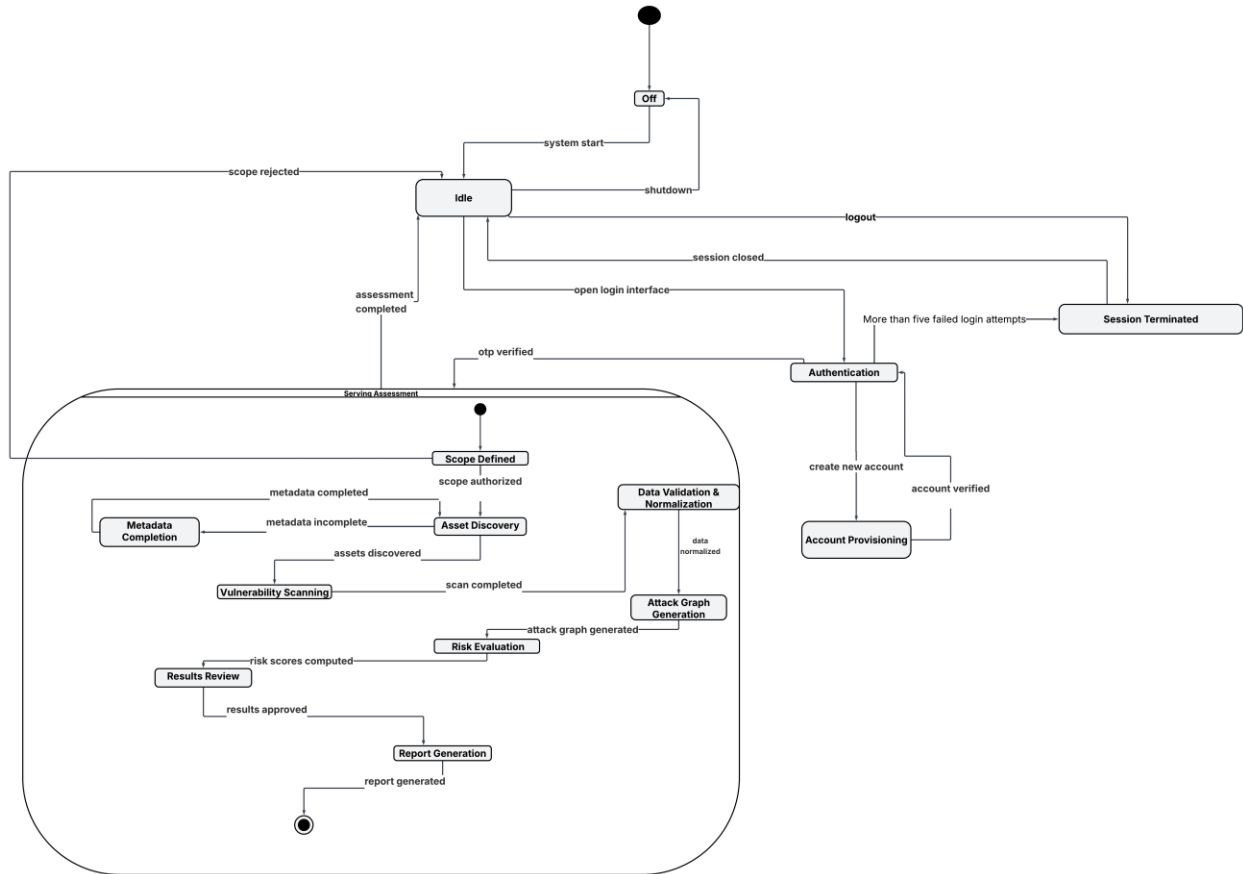


Figure 11: State diagram illustrating the system behavior

4.3 System Architectural Models

This section presents the system architecture, describing the main components and their interactions. It illustrates how core functionality is organized and include architecture risk analysis, threat modeling and deployment consideration.

4.3.1 Design Goal

The design of the system aims to achieve the many non-functional system requirements and quality characteristics mentioned in the previous chapter, to ensure a safe, efficient and user-friendly system.

- Usability: Design the system interfaces to be clear and easy to use.

- **Reliability:** The system ensures the accuracy of results and the consistency of work without any failures that have negative effects on the business or users.
- **Availability:** The system is designed to be available at any time when needed.
- **Scalability:** The design allows for the addition of new users or features without the need to shut down the entire system or the need for a large number of changes.
- **Maintainability:** The system is based on a modular design which makes it easy to modify or repair the system or add new features.
- **Performance:** The system is designed to be able to perform all functions in a timely manner and as quickly as possible.
- **Security:** Many security functions such as authentication and role assignment have been used to ensure data integrity and prevent unauthorized access.

4.3.2 Component diagram

As shown in *Figure 12*, the diagram shows an overview of the system by identifying its main components and their interactions. It shows the user interface and its interaction with the backend services. It ensures security control and data continuity across the MySQL database system. This design enables system scalability, maintenance, and security through a complete separation of user interfaces, core services, and infrastructure components.

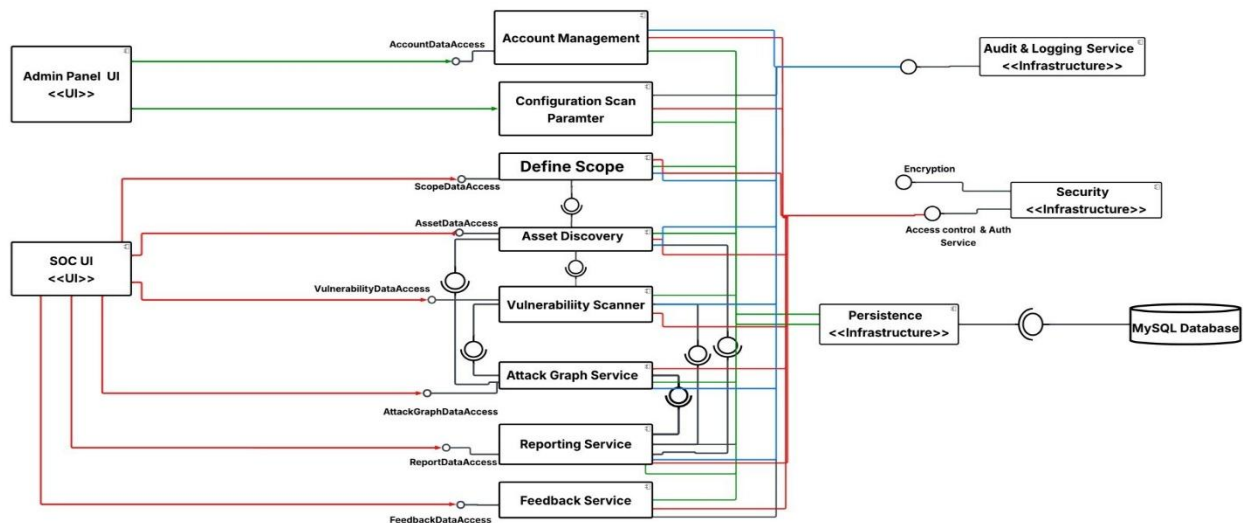


Figure 12: The main components of the system with their own responsibilities and interactions.

4.3.3 Software Architecture

The suggested system software architecture will be designed in conformity with the layered architectural style, as shown in *Figure 13* below, in which the system will be separated into three different layers: the UI Layer which handles user input, data display, and system interaction, the Business Layer which contains the core system services, and the Infrastructure Layer. The separation in this case will lead to better

modularity, security, and maintainability due to the separation of responsibilities in these layers.

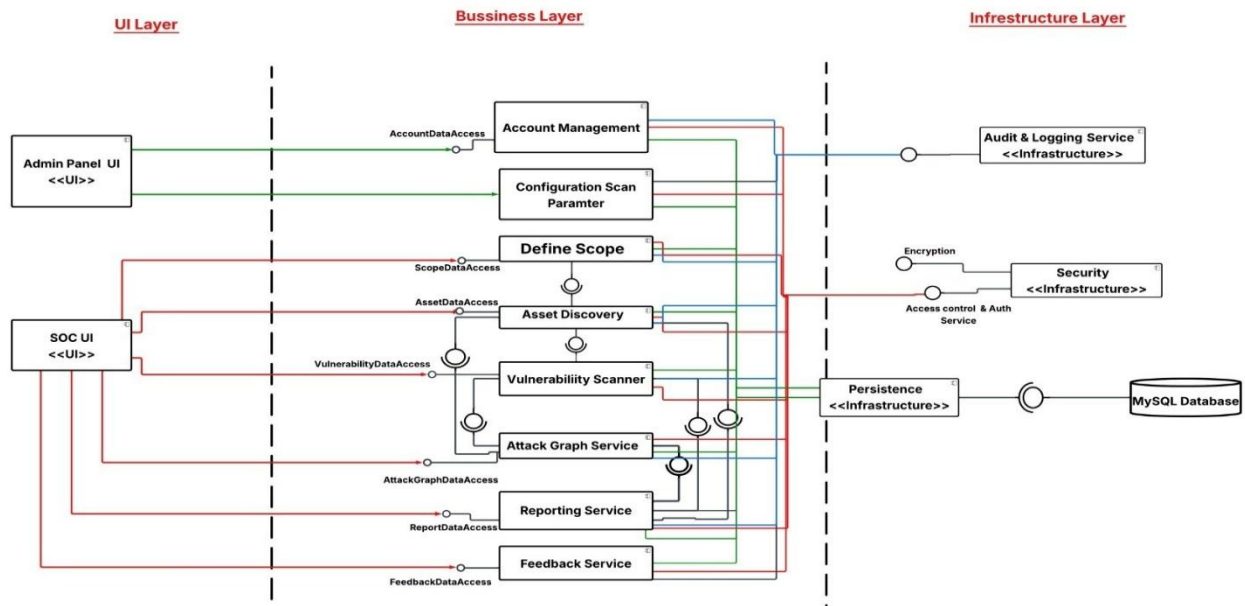


Figure 13: High-level layered software architecture illustrating the interaction between UI, business, and infrastructure layers.

4.3.4 Architectural Risk Analysis and Threat Modelling

This section presents the architectural risk analysis and threat modeling for the suggested system. It describes the system architecture, identifies potential security threats based on STRIDE, presents the threat trees, and analyzes risks using DREAD to help define mitigation strategies to protect system components' data flow and trusted boundaries

Architecture Diagram

Proposes the design scheme as shown in *Figure 14* of the suggested system at a high level, including key components, data flow, key resources, etc. It generally divides a system into client end, application server, database server, and external environment for secure system design.

The Apache server is used to forward requests to the backend application that runs the business logic, all data is stored in a centralized MySQL database. The system also communicates with external vulnerability databases, such as CVE, to improve the accuracy of detected vulnerabilities and update them with CVSS information. This architecture supports threat modeling by clearly defining trust boundaries, data flows, and assets.

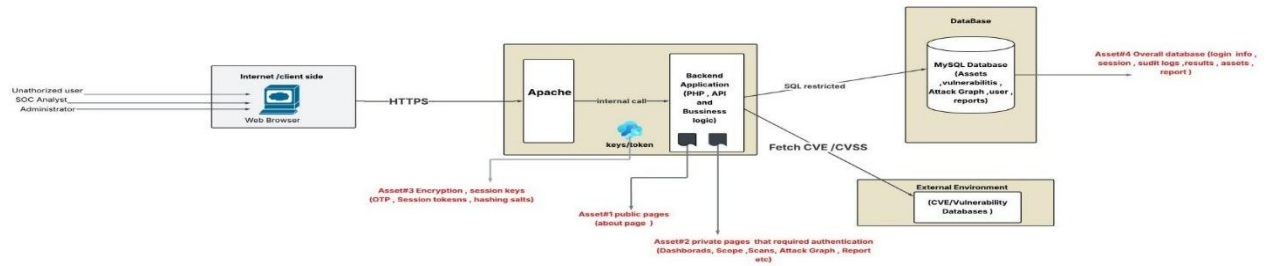


Figure 14: High-level system architecture showing major components, data flows, and critical assets

Decompose Application

The system is broken down into its core parts to fully understand the system security, so that the basic components of the system, data flow, entry points, trusted boundaries, and critical assets are known. Through all of this, it is possible to identify and control where attacks occurred. As shown in Figure 15.

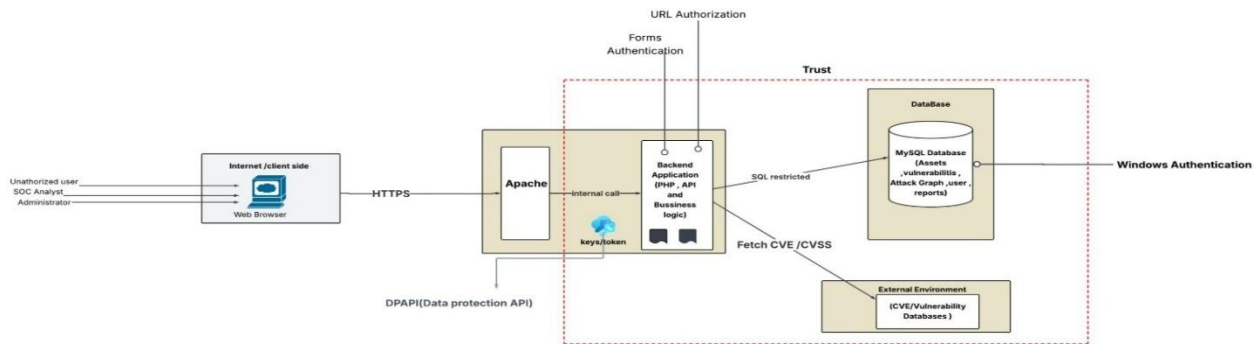


Figure 15: Application decomposition and trust boundary identification for threat modelling

DFD

The DFD diagram describes the data flows in the system, how they travel through the system, what they pass through, and where they are within the system's trust boundaries. External entities (SOC Analyst, Admin, and unauthorized user) begin interacting with the system via the web browser as shown in Figure 16, where all data is sent using HTTPS, forming the trust boundary between users and the application. Within the application, the Controller processes requests and coordinates between the View and the rest of the system components, while clarifying the flow of data such as authentication data, assets, vulnerabilities, and analysis results. The MySQL database exists under a separate trust boundary to protect sensitive data and ensure access only to authorized components.

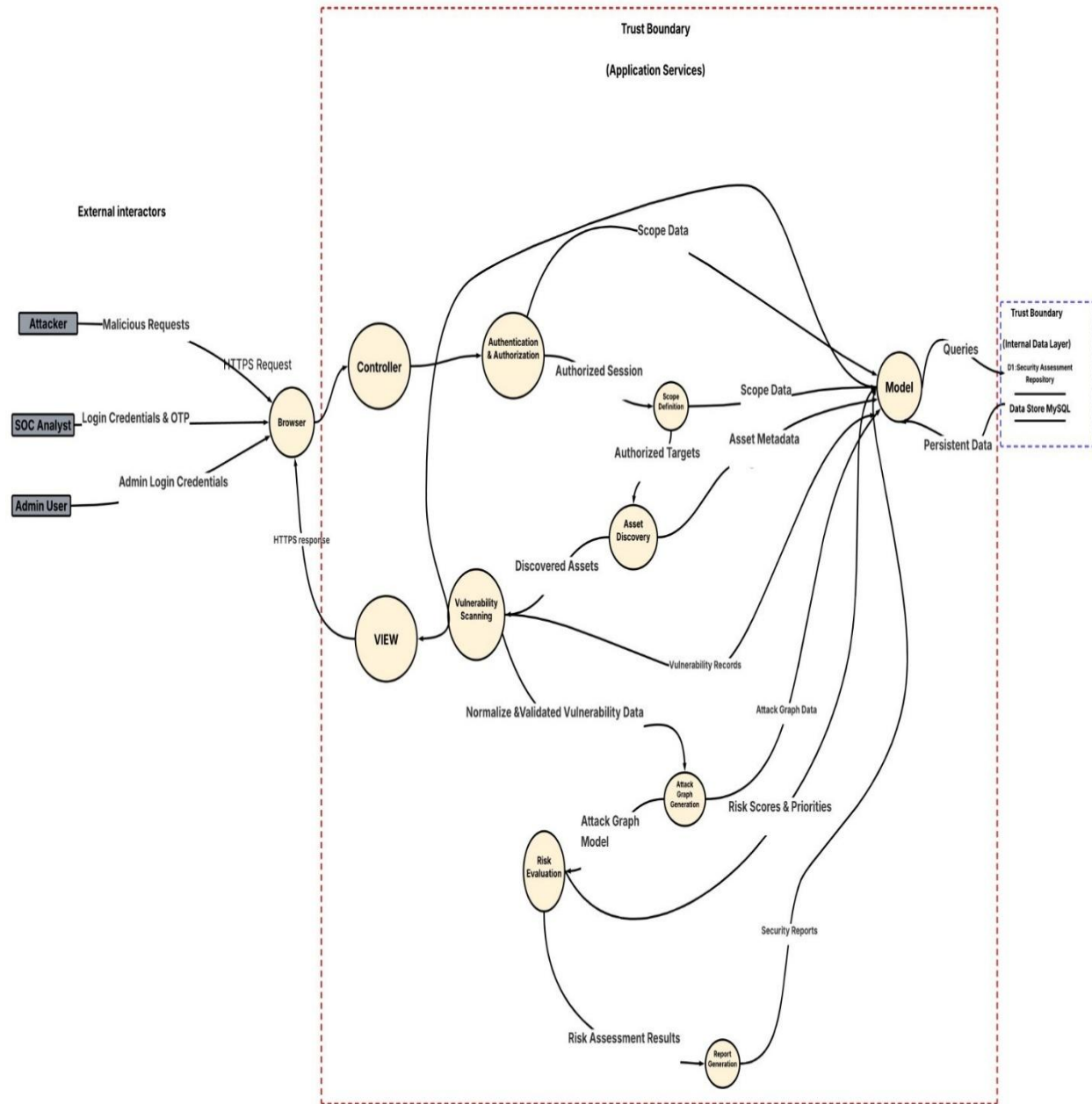


Figure 16: Data Flow Diagram (DFD) of the suggested system showing main processes, data flows, data stores, external entities, and trust boundaries

Identify Threats

In this step potential security threats are identified based on the system architecture, trust boundaries and data flows. The STRIDE threat modeling methodology is used to analyze possible threats affecting system components. Figure 17 illustrates STRIDE threat categories used to identify potential security threats affecting to the system.

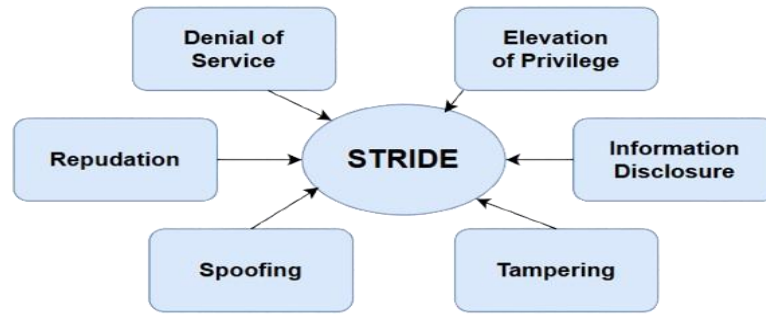


Figure 17: STRIDE threat modeling stages

STRIDE is used to classify threats based on their nature and impact, based on different system components, including spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege. The methodology is applied to system components, data flows, and trust boundaries shown in the *Figure 15* to effectively identify possible threats before moving on to risk analysis.

The following *Table 9* summarized the identified security threats based on STRIDE threat modeling methodology.

Table 9: STRIDE-based classification of identified security threats in the proposed system

Threat	Threats in the system include:
Spoofing	• Impersonating legitimate users or administrators' entity using stolen credentials. • Phishing attacks to capture user login information. • Unauthorized API calls that pretending to be the official web interface. • Backend service impersonation by inject false data or alter analysis results.
Tampering	• Modification of discovered asset, service data or results that stored in the database. • Alteration of vulnerability details to hide real security risks • Manipulation of attack graphs or attack paths. • Tampering with risk assessment results. • Unauthorized modification of assessment scope configurations.
Repudiation	• Denial of performing unauthorized activities within the system. • Denial of modifying system configurations or user permissions.
Information Disclosure	• Unauthorized access to asset details lists. • Exposure of vulnerability information that related to assets. • Disclosure of attack graphs and attack paths information. • Unauthorized access to security reports results. • Leakage of sensitive dashboard information, due to weak access control.
Denial of service	• Flooding web interfaces, or backend APIs with excessive requests. • Resource exhaustion CPU, memory that leading to service unavailability. • OTP flooding leading to exhaust authentication resources.

Elevation of privilege	- Exploiting access control weaknesses to gain higher privileges than authorized. - Bypassing authorization checks to access restricted functionalities and services. - Exploiting poor separation of duties. - Hijacking sessions to gain access privileged resources.
-------------------------------	--

Document the Threats

Table 10 documents the identified security threats. In the suggested system based on the STRIDE threat modeling, each threat is targeted to a component, possible attack techniques and the countermeasure.

Table 10: Documented STRIDE-based security threats for the proposed system

Threat Name	STRIDE Category	Description	Target component	Attack techniques	Countermeasures
Credential impersonation	Spoofing	An attacker impersonates a legitimate user using stolen credentials	Authentication dashboard	Phishing, credential theft	OTP authentication, secure session management
API identity spoofing		Unauthorized API calls pretending to be the authorized web interface	Backend APIs	API impersonation	API authentication, token validation
Asset Data Manipulation	Tampering	Modification of discovered asset inventory data	Asset discovery	Unauthorized data modification	RBAC, audit logging, integrity checks and restricted write access
Vulnerability Data Tampering		altering vulnerability details	Vulnerability data	Data manipulation	
Attack Graph Manipulation		Manipulation of attack graphs or attack path results	Attack graph service	Data manipulation	
Action Denial	Repudiation	User denies perform system action	System functionality	Action denial	Comprehensive audit logs
Configuration Change Denial		Denial of modifying scope, roles, or configurations	Admin panel	Lack of traceability	
Unauthorized Dashboard Access	Information Disclosure	Unauthorized access to dashboards or security reports	Dashboard	Broken access control	Authorization checks, RBAC
Asset Metadata Exposure		Exposure of IPs, services, and open ports	Asset repository	Data leakage	Least privilege access
Security Report Leakage		Disclosure of vulnerability and risk evaluation results	Reporting service	Unauthorized read access	Secure report access
Scanner Resource Exhaustion	Denial of service	Overloading asset discovery or scanning modules	Discovery scanning service	Resources exhaustion	Scan scheduling
OTP Flooding		Excessive OTP request	Authentication Service	OTP abuse	OTP rate limiting

API Request Flooding		Flooding the system with requests	Web UI, Backend API	Request flooding	Rate limiting
Privilege Escalation via Authorization Bypass		Gaining authorized privilege by bypassing authorization	Authorized layer	Authorized bypass	RBAC
Insecure API Privilege Escalation	Elevation of Privilege	Exploiting insecure APIs to perform privileged actions	Backend APIs	API abuse	Secure API design, authorization validation
Session Hijacking		Exploiting session weaknesses to access system functions	Session management	Session hijacking	Secure cookies and secure session management

Attack tree

The concept of attack trees is employed for modeling and displaying the different ways in which the attacker may reach a specific security objective by dividing the attack into smaller logical attacks. Attack trees are developed based on the different categories of the STRIDE attack threats for effective analysis of different attack scenarios against the system. The attack trees are constructed with the goal of representing a given high-level class of threats and making clear attacks for effective mitigation of the risks. The root represents the primary goal, while the children are the goals that, if achieved, fulfill the root, and the specific goals of each child must be fully achieved as shown in *Figure 18* and *Figure 19* illustrate the attack tree for STRIDE threats. To see the diagrams more clearly, you will find the details in the *Appendix 4*.

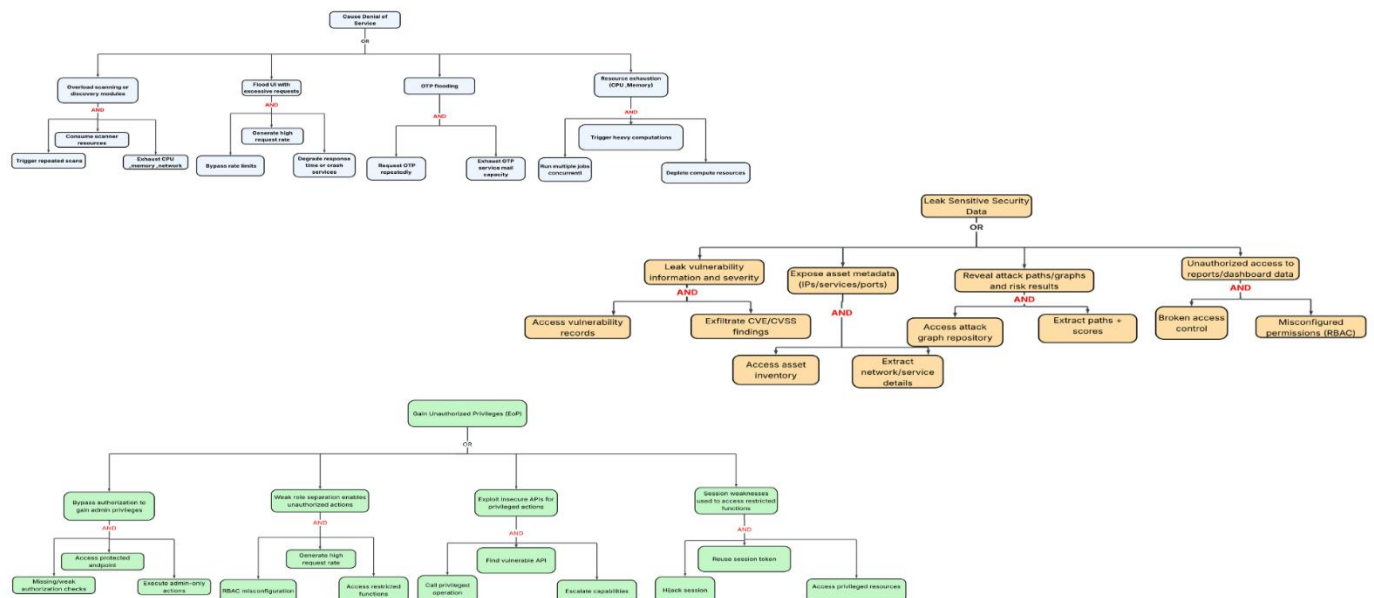


Figure 18: Attack Trees represent leak sensitive security data threats, denial of service threats, and gain of privilege action threats

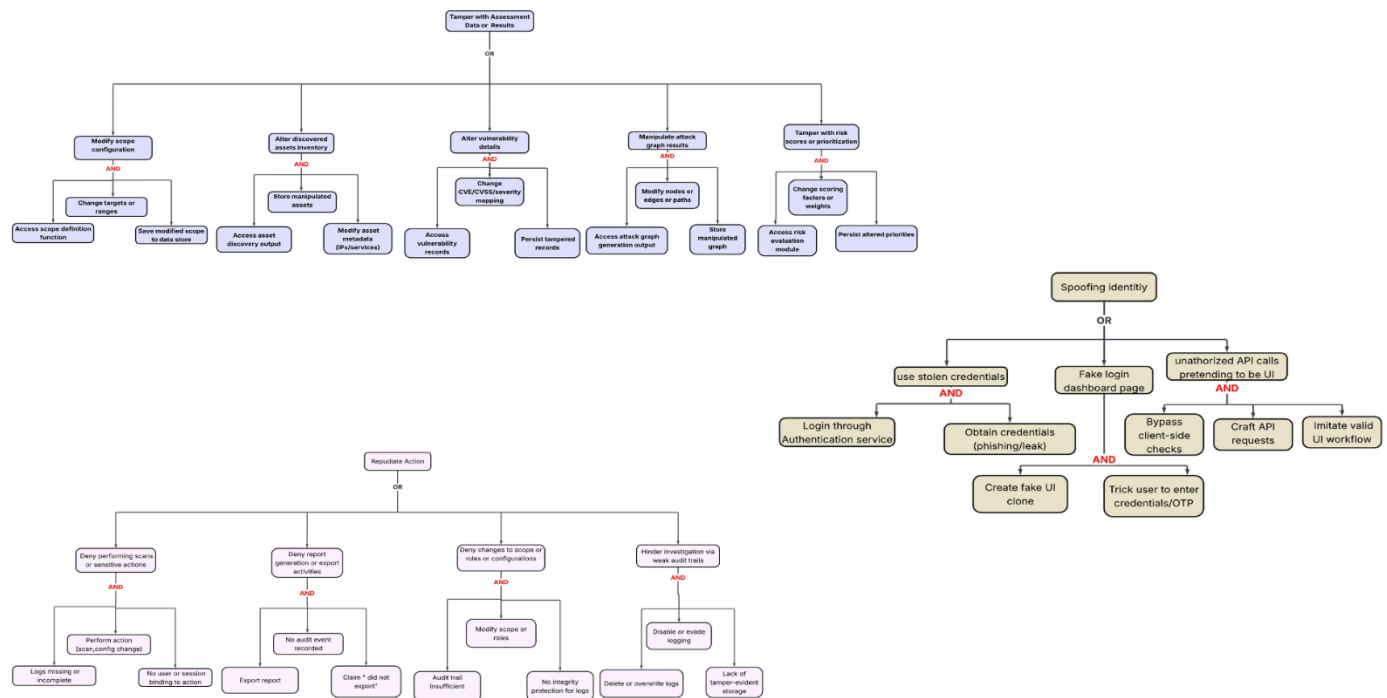


Figure 19 : Attack Trees represent repudiation action threats, tampering with assessment data or results threats, and spoofing identity threats

Rate Threats

Table 11 represent the DREAD risk analysis technique is used to calculate the severity of the threats.

DRFAD helps in determining the severity based on five main parameters, which are Damage Potential, Reproducibility, Exploitability, Affected Users, and Discoverability. These parameters have to be given the scores: High (3), Medium (2), and Low (1). The DRFAD scores combine to form the DREAD scores, based on which the severity of the threats can be determined.

Table 11: DREAD-based Risk Assessment of Identified Threats

Threat ID	STRID Category	Threat Description	D	R	E	A	D	Total Score	Risk Level	AVG
T1	Spoofing	Impersonation of legitimate user or administrator	3	3	2	3	2	13	High	2.6
T2	Tampering	Modification of discovered assets or vulnerability data	3	2	2	3	2	12	High	2.4
T3	Repudiation	Denial of performing sensitive actions due to lack of logs	2	2	2	2	2	10	Medium	2.0
T4	Information Disclosure	Unauthorized access to asset, vulnerability, or attack path data	3	3	2	3	3	14	High	2.8

T5	Denial of Service	Flooding system services or OTP requests to disrupt availability	3	2	2	3	2	12	High	2.4
T6	Elevation of Privilege	Gaining higher privileges through authorization weaknesses	3	2	2	3	2	12	High	2.4

Plan and Review

The results of the DREAD-based risk assessment as shown in *Table 11* identify the threats in order of their associated risk to guide mitigation planning. High-level risks are mitigated first, while medium-level risks are managed through monitoring and control. Security risks are re-evaluated whenever the design of the system changes, new threats are identified, or on a regularly basis as part of regular security reviews in order to ensure that the security position of the system remains acceptable.

4.3.4 Deployment Diagram

The deployment diagram of the suggested system displaying how the system components are distributed across different nodes, as shown in *Figure 20*. The client device hosts a web browser that interacts with the server over a secure HTTPS connection. The server node runs off the Apache web server, the backend application login, and the MySQL database to handle core system functionality and data storage. In addition, the system communicates with external CVE and vulnerability databases to retrieve vulnerabilities.

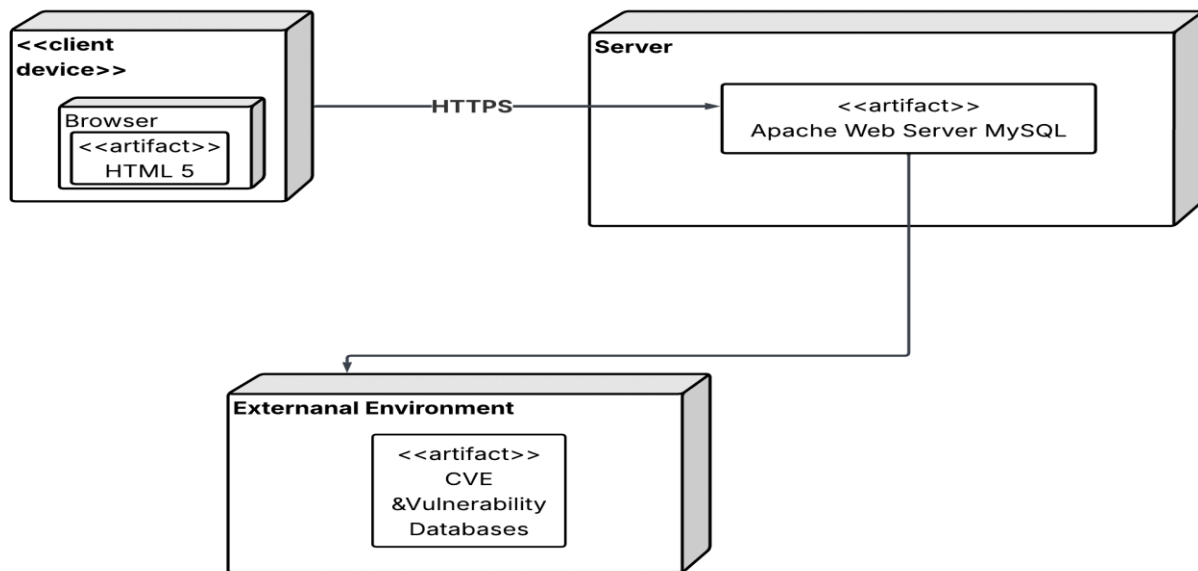


Figure 20: Deployment diagram of the proposed system

4.4 Data Management and Data Stores

The suggested system uses a single centralized database to store and manage all system data. Within this data is logically organized into multiple internal data stores, where each data store

represents a specific category of system information. By this improves data organization, support efficient correlation between different types of data and facilitates security analysis, visualization, and reporting.

1-User and Access control Data store

This data store contains details regarding to authorize users, like user accounts, role assignments, and permission sets. It helps implement authorization and authentication processes that enable only authorized users to access operations of the system based on their role and permission.

2-Asset data Store

This data stores information about discovered assets, within the defined system scope such as IP addresses, asset types, operating systems and related metadata. It represents the foundation for security analysis processes that including vulnerability assessment and attack path generation.

3-Vulnerability data store

This data store maintains information about security vulnerabilities identified on discovered assets. Within this data store, information such as vulnerability identifiers such as CVEs, severity level, and CVSS scores. This data store enables the association between results that include the vulnerabilities, and affected assets to supports risk evaluation.

4- Attack graph and Attack path data store

This data store is used to store attack graph components, and attack paths, including nodes, and edges that present the relationships how vulnerabilities can be exploited to move between assets.

5-Analysis and risk results data store

This data store stores the results of security analysis processes, like calculated risk levels, and overall exposure assessment results, these results are used for visualization within the dashboard, and for generating security reports.

6-Reports and logs data store

The data stores generate security reports and maintain a log of the activities carried out on the system, which helps to document, audit, and monitor the entire process to ensure security and traceability.

- Data relationships and access control

These data stores are logically connected in terms of relational keys, thus facilitating correlation between assets, vulnerability data, attack graphs, and risk evaluation data. All data stores are strictly controlled through the persistence layer in order to allow authorized backend services to access data stores while restricting any unauthorized components from directly accessing any data stores

- The system follows a data retention policy, which outlines the period for which different types of data have to be retained. Major security data, like vulnerabilities, attack paths, and the results of analysis, have to be retained for a specified period for analysis purposes. The old and unimportant data can be archived or deleted in accordance with system policies that optimize storage with the preservation of major security data.

The Entity Relational Diagram (ERD) represents the logical structure of the system database by displaying the main entities, their attributes, and the relationships between them, as shown in *Figure 21*. To see the graphic more clearly, you will find the details in the *Appendix 5*.



The system is designed to follow several security principles to ensure data protection and prevent unauthorized access. The system is based on the mechanism of authentication, role and control of permissions, implementation of the principle of least privileges, validation of entered data, and securing the mechanism of uploading files.

4.5.1 Authentication

The system relies on the two-factor authentication mechanism based first on the user's personal email and the stored password, which is encrypted, and then entering the OTP number that is sent to the user on his personal email after verifying the email and password. Rate limiting is applied to prevent Brute Force Attack converters and limit the number of failed login attempts within a specified period of time. Safe sessions are used.

4.5.2 Authorization

The system uses a role-based access control model, where different permissions are assigned to each role so that each user can access only the functions and data they need to perform their tasks. User privileges are ascertained when trying to access sensitive resources and the principle of least privilege is applied.

4.5.3 Data Access

The system verifies and encodes all the input data, whether from the user or the system itself, and pre-prepared queries (Prepared Statement) are used to avoid any injection attacks.

4.5.4 File Upload

When uploading files, the system checks the file type and size before accepting it to allow only authorized files, and they are stored in a separate location to ensure the protection of the system and the rest of the files.

4.6 Interfaces

4.6.1 User Interfaces

These interfaces including the login screen, administrative user management, dashboard overview, asset management, vulnerability analysis, attack graph visualization, feedback, and report generation screens, all interfaces follow a consist layout that include a top navigation header, and a sidebar menu for navigation between system features. The interfaces present security data using visual elements such as table, charts, graphs and status to enhance readability. The system also provides a clear error messages without sensitive information for invalid inputs, and authentication failures.

1. **Dashboard Overview:** This interface provides a high-level summary of the system results, including total assets, vulnerabilities and overall risk score.

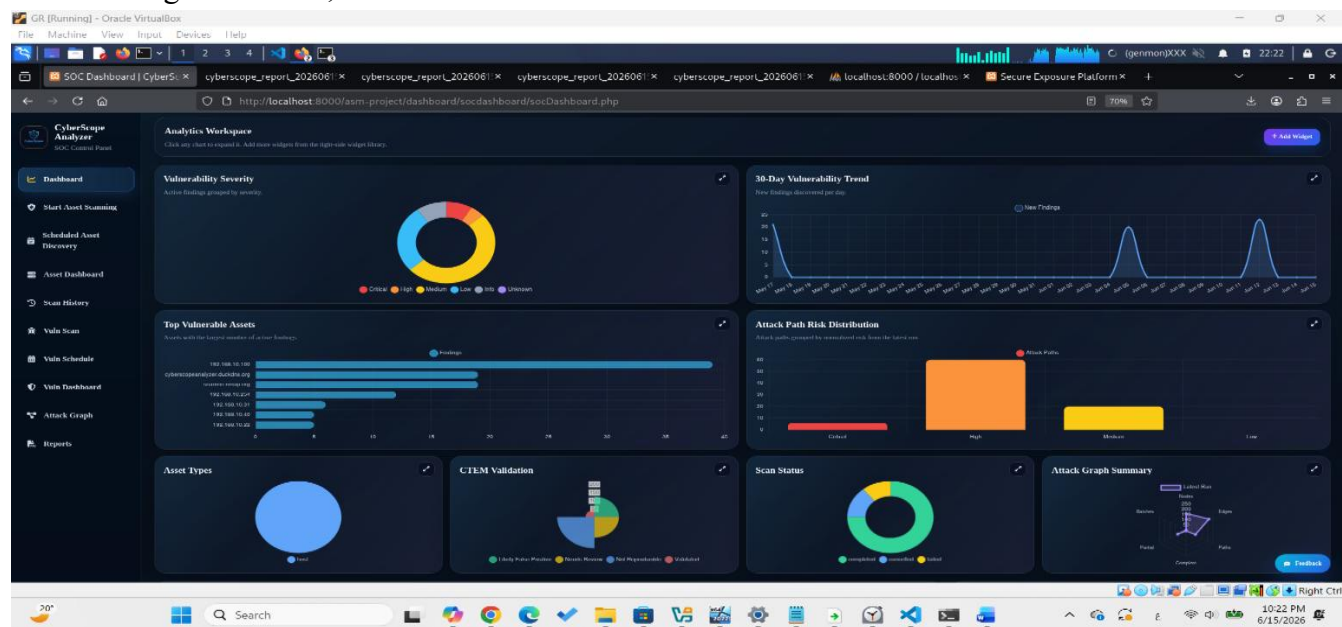


Figure 22: Basic dashboard

2. **Assets Dashboard:** This interface displays all discovered assets, within the defined scope along their security status.

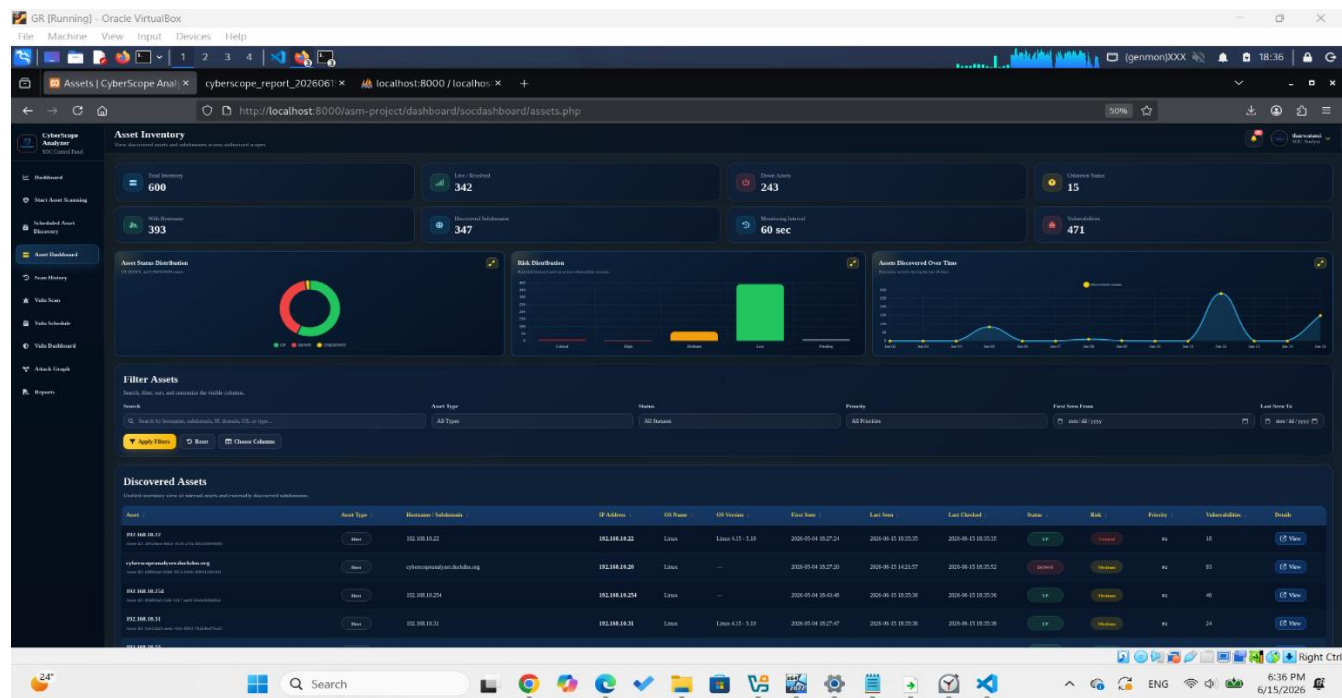


Figure 23: Assets dashboard

3. Vulnerabilities dashboard: This interface presents the identified vulnerabilities, and categorized by severity including CVE details.

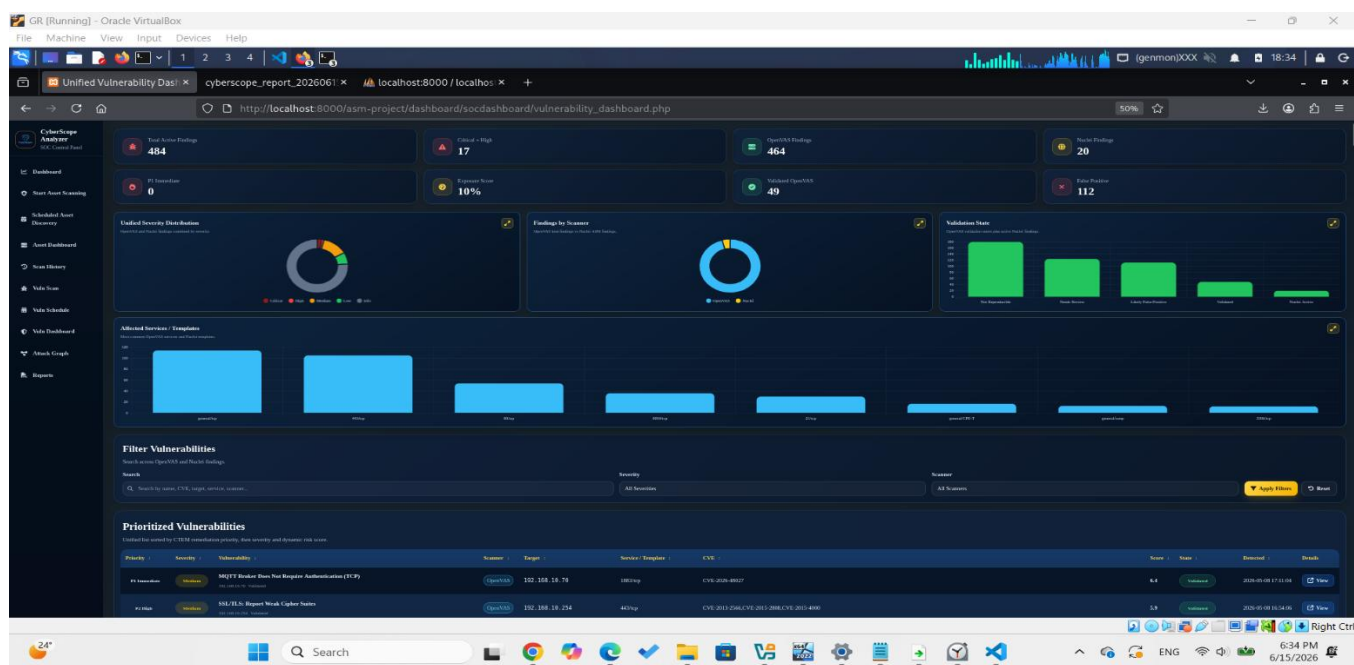


Figure 24: Vulnerabilities Analysis Dashboard

4. Attack graph dashboard: This interface visualizes potential attack paths that generated from discovered assets and vulnerabilities.

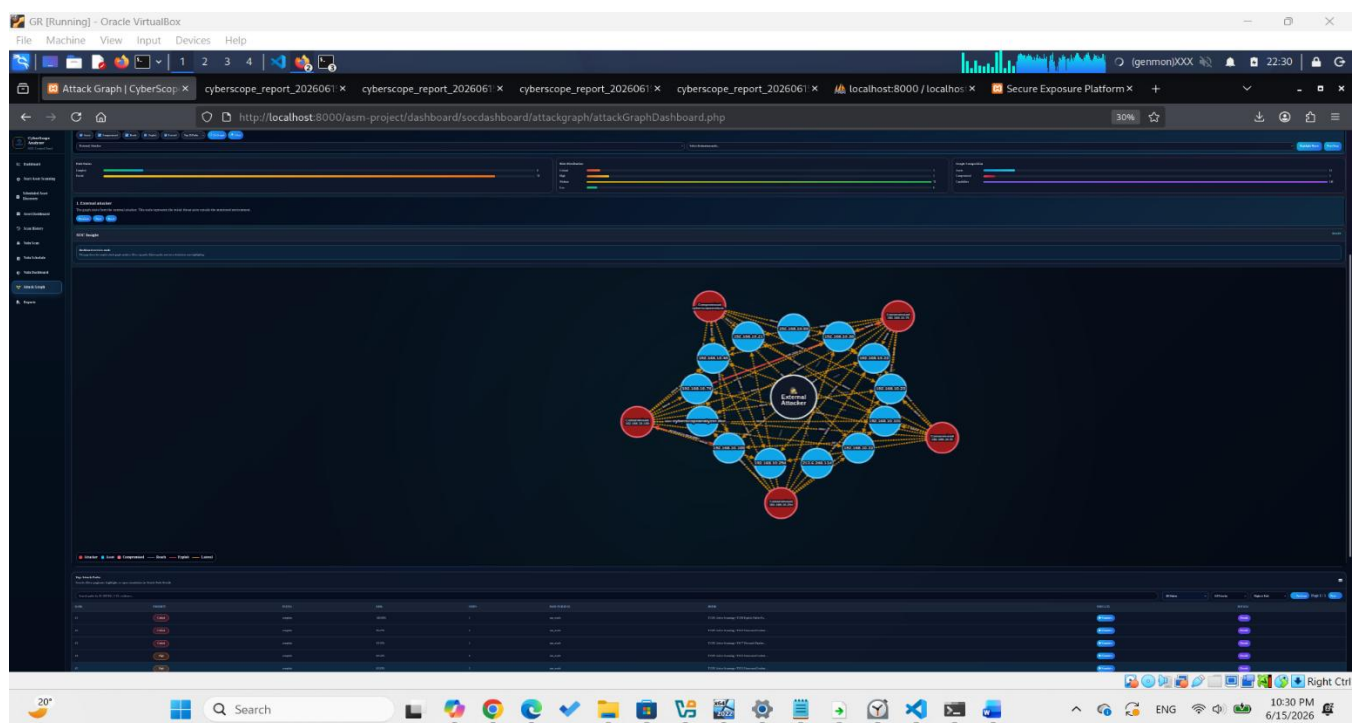


Figure 25: Attack Graph Visualization Interface

4.6.2 System Interfaces

The suggested system interacts with several internal and external systems across specific system interfaces to support security analysis functions. Externally, the system integrates open-source scanning tools to detect assets and detect vulnerabilities, where the system processes and analyzes the results in depth, and integrates publicly available vulnerability databases to provide additional information about the discovered vulnerabilities. Internally, the system relies on APIs to enable communication between backend components, including asset management, vulnerability analysis, and attack blueprinting. The system also interacts with a centralized database management system for storing and retrieving data.

4.6.3 Software Interfaces

The suggested system will interact, via precisely defined software interfaces, with various internal and external software components to achieve its defined security analysis functions. These interfaces provide structured communication, data exchange, and integration between system modules and external tools. Externally, the system interacts with many open-source scanning applications by executing commands on the terminal, and interacting with vulnerability databases via application programming interfaces (APIs), to retrieve information about vulnerabilities. Internally, the system interacts with a centralized MySQL database management system, using SQL-based data access interfaces.

Software libraries and frameworks also provide the system with support for core functionalities like attack graph generation, random walk sampling, risk scoring, and result visualization. Application programming interfaces and shared data models are used to exchange data internally between components, thus ensuring modularity and maintainability.

CHAPTER 5

SYSTEM IMPLEMENTATION & VALIDATION

5.1 Introduction

This chapter presents the implementation, verification, testing, and evaluation of the suggested system. This chapter describes the implementation of the system's main modules, what tech they used during development and both functional and security tests they ran. Finally, the evaluation results that show the effectiveness of the system in achieving its goals.

5.2 Implementation approach

5.2.1 ASM Module Implementation

The ASM module was developed for automated detection and analysis of the external attack surface of an enterprise. Users can define scanning scopes via IP addresses, domains, or hostnames, after which all the required work will be done automatically, including asset discovery and vulnerability assessment.

Several security tools were used in the discovery process to identify active assets and exposed services. For example, if the scope is of the network type, it first uses MASSCAN, which detects all open services, and then NMAP verifies all services, correlates them to the discovered assets, and extracts all the remaining information about the asset. Openvas then detects the vulnerabilities for each asset. If the scope is of the subdomain type, the Subfinder finds all the domains, then DNSX finds the IP addresses of the domains, then HTTPX increases the effectiveness of each domain, and after collecting the information, nuclei finds the vulnerabilities associated with the discovered assets.

All gathered information is stored in the database of the system, making it possible to manage and analyze discovered assets, services, and vulnerabilities. In addition, the module also supports scheduled scanning, enabling periodic assessments to detect newly exposed assets or changes in the attack surface without requiring manual intervention.

5.2.2 CTEM Validation Module

The CTEM validation module was implemented to continuously validate discovered vulnerabilities and reduce any false positives. While the scanning process alone cannot be fully relied upon in validating vulnerabilities, each finding is done based on different indicators, including severity, CVSS score, Quality of Detection (QoD), CVE availability, network service evidence, presence within CISA files, and their use in a ransomware attack.

Based on the results of the vulnerability verification generated by indicators extracted from Openvas in addition to the use of CISA that includes the shoulder of trusted and proven vulnerabilities, the vulnerabilities are categorized into statuses: verified, needs to be reviewed, non-repeatable, or potentially false positive, with a validation score generated for the reliability of

the results. The unit conducts periodic re-verifications, frequently checking for high-risk vulnerabilities to keep exposure information up to date.

For security operations purposes, validated high and critical vulnerabilities will generate notifications and email alerts for SOC users. Furthermore, mitigation recommendations generated by AI for each validated vulnerability would be saved alongside with other relevant information, providing users with concise remediation guidance to help address the identified security issues.

5.2.3 Attack Graph Module

The Attack Graph module aims to create potential attack paths by correlating discovered assets, vulnerabilities, and network connections. Unlike the individual presentation of vulnerabilities, the module analyzes how an attacker could exploit multiple vulnerabilities to attack the network and access valuable assets.

The generated graph is based on various nodes, including attackers, assets, network services, and compromised hosts, and edges which represent the relations like network connectivity, vulnerability exploitation, and lateral movement within the network. To create an attack graph, the module utilizes validated vulnerability data gathered during the operation of the ASM and CTEM modules, ensuring that only reliable findings are included in the analysis.

To prioritize attack scenarios, this module uses the controlled random walks algorithm and the Dijkstra's shortest-path algorithm for calculating the shortest path in terms of the lowest-resistance. Each generated path is assigned a normalized risk score based on the severity and exploitability of the involved vulnerabilities. Additionally, the attack techniques is translated into MITRE ATT&CK framework, providing analysts with standardized threat intelligence and improving the interpretation of attack paths.

The generated attack graph, risk scores, and attack paths are stored in the database and visualized in the form of an interactive dashboard, allowing SOC analysts to explore possible attack scenarios and focus on the most critical risks.

5.3 Software Testing and Verification

5.3.1 Test Case generation and Testing

The results of the detailed test cases are in *Appendix 6*.

Table 12: Test Cases and Implementation Results

Test case ID	Test case description	Pre-conditions	Test data	Test steps	Expected result	Actual result	Status
TC-01	Verify login functionality with valid credentials.	User must have a registered account.	Username: 1220826@student.birzeit.edu	1. Open the CyberScope Analyzer interface.	If the information is correct, the two-factor	The user's identity was initially verified and redirected to	Pass

			Password: #####, And an answer to the arithmetic equation.	2. Navigate to the login screen. 3. Enter the username, password, and solve the equation. 4. Click on the "Login: button.	authentication page is redirected.	the two-factor authentication page.	
TC-02	Verify login functionality with invalid credentials.	User must have a registered account.	Username: Ayham21@gmail.com Password: pass123	1. Open the CyberScope Analyzer interface. 2. Navigate to the login screen. 3. Enter the username, Password, and solve the equation. 4. Click on the "Login: button.	Access denied and error message displayed.	Access denied and error message displayed.	Pass
TC-03	Verify asset discovery functionality.	1. User must have a registered account. 2. User logged in with SOC privileges.	Target Scoop (Network: 192.168.10.0/24).	1. Open Start Asset Scanning page. 2. Select target. 3. Start scan 4. Wait for completion.	Successfully discover and store assets.	The assets were successfully discovered and stored.	Pass
TC-04	Verify CTEM Validation.	1. User must have a registered account. 2. User logged in with SOC privileges.	Target Asset (IP:192.186.10.23).	1. Open Vuln Scan page. 2. Select target. 3. Start scan 4. Wait for completion.	Successfully detect, verify, filter, sort, and store vulnerabilities.	Vulnerabilities have been successfully detected, verified, filtered, sorted, and stored.	Pass
TC-05	Verify Attack Graph Generation.	1. User must have a registered account. 2. User logged in with SOC privileges.	Assets and vulnerabilities discovered in the scan.	1. Open Attack Graph page. 2. Click on the 'Refresh Graph' button.	Build the complete graph and discover all the possible paths.	The full diagram is built and all possible paths are discovered.	Pass
TC-06	Verify Attack Path Simulation.	1. User must have a registered account. 2. User logged in	Assets and vulnerabilities of the selected Path.	1. Open Attack Graph page. 2. Select path. 3. Click on the 'Simulate' button.	View the chosen path, all the assets and vulnerabilities that pass through it,	The chosen path, all the assets, vulnerabilities that go through it, and all the	Pass

		with SOC privileges.			and all the information about it.	information about it are displayed.	
TC-07	Verify generate security report.	1. User must have a registered account. 2. User logged in.	All sections selected by the report creator (All sections).	1. Open Report page. 2. Select the desired sections. 3. Click on the 'Generate PDF Report' button.	A report is generated that contains all the selected sections.	A report has been created with all the selected sections.	Pass
TC-08	Verify role-based access control.	1. The user must have a specific role. 2. User logged in with SOC privileges.	Admin Dashboard URL.	1. Change the URL to go to the admin dashboard.	Block the transition, log out, and redirect to the login page.	The transition was blocked, logged out, and the login page was redirected.	Pass
TC-09	Verify two-factor authentication.	1. User must have a registered account. 2. The user has entered valid information.	OTP number that was sent to the user on email(850877).	1. The user enters the OTP number sent to the email. 2. Click on the Verify button.	Access the dashboard if the number is correct.	The user is logged in and accessed the dashboard.	Pass
TC-10	Verify Scheduled Scan execution.	1. User must have a registered account. 2. User logged in with SOC privileges.	1. Schedule Name: test. 2. Scope: ScanMe Nmap Test scanme.nmap.org subdomain. 3. Interval: weekly. 4. Scheduled Time: 10:27 pm.	1. Open Scheduled Asset Discovery page. 2. Fill in the required information and schedule the scan. 3. Click on the '+Create Scheduled Scan' button.	Set a new date for the scan and conduct the scan when the required time comes.	A new scan was scheduled and the scan was conducted on time.	Pass

5.3.2 Static Application Security Testing (SAST)

Static test was performed on the method of reviewing the source code of the system without running it, to detect and fix software errors at an early stage. Since the system contains several programming languages, a special tool was used to examine each language. Test result in *Appendix 6*.

- PHP: To examine all of the authentications, sessions, permissions, and database queries, which were written in it, the PHPStan tool was used.

- Javascript: To examine the frontend codes, the ESLint tool was used.
- Python: To examine the logic of the implementation of the Attack Graph methodology, the Pylint tool was used.

5.3.3 Dynamic Application Security Testing (SAST)

Dynamic testing was performed on the method of testing the functionality of the system during its operation, to detect errors and security vulnerabilities and to know the behaviour of the system during operation. To do this, the OWASP ZAP tool was used, which analyses the movement of requests and responses between the server and the client to detect security vulnerabilities.

The tool checked and attempted to bypass the login process, managed sessions, and attempted to exploit privileges. It also tried to inject SQL queries or javascript code to make sure that the system validated and encoded the inputs. Check for error messages to see if they reveal sensitive information. Dynamic testing confirmed that the system is operating safely while running and effectively applies protection principles. Test result in *Appendix 6*.

5.4 Software Evaluation

The results of the detailed test cases are in *Appendix 6*.

Table 13: Software evaluation tests and results

Test ID	Test Type	Test Scenario / Objective	Test Description	Expected Result	Security Focus	Status (Pass/Fail)
ST-01	System Testing	Verify the behavior of the entire system.	Full system workflow testing from login to attack path and report generation.	System works as intended end to-end.	Ensure that the system works properly and securely at the same time.	Pass
ST-02	Integration Testing	Verify interaction between modules.	Examine the integration between backend and frontend and the integration of the technologies and tools used.	Data flows between system units efficiently and securely.	Prevent data flow insecurely.	Pass
ST-03	Authentication Testing	Verify the security of the login mechanism.	Valid/invalid check login attempt, and attempts to attack it.	Allow trusted users to log in, and prevent any attack attempt.	Prevent any unauthorized access.	Pass

ST-04	Authorization Testing	Validate role based access control (RBAC).	Testing the user's authority to access the system's functions, and that he can access what he is allowed to do.	The user has access to what is authorized.	Prevents privilege escalation.	Pass
ST-05	Input Validation Testing	Test injection attacks.	Attempt SQL injection, XSS, and malformed inputs.	The user cannot inject any malicious code through the input fields.	Prevent all injection attacks.	Pass
ST-06	API Security Testing	Validate secure API usage.	Verify that the API is being used securely.	Only authorized users have access to API endpoints.	Protect system functions from tampering.	Pass
ST-07	Penetration Testing	Simulate real-world attacks.	Attempting to access the system in unauthorized ways or accessing unauthorized ones, exploiting vulnerabilities, breaking authentication and bypassing restrictions.	The system prevents all types of attacks and takes protective measures.	Fully protect the system from tampering and vandalism.	Pass

CHAPTER 6

CONCLUSION AND FUTURE WORK

Conclusion

This project presented a comprehensive security that integrated the best practices of Attack Surface Management (ASM), Continuous Threat Exposure Management (CTEM), and attack graph analysis to provide a visible view of an organization's external attack surface and security posture, by correlation the discovered assets, identified vulnerabilities, and potential attack paths, then enables risk evaluation based on real exploitation scenarios rather than isolated vulnerability findings.

This proposed system is particularly suitable for small and medium enterprises, as it improves security visibility, and supports risk-based prioritization, and assists security terms in making informed decisions.

Future Work

Several enhancements can be considered to extend the capabilities of the suggested system. One important direction for future work is the integration of artificial intelligence (AI) techniques to enables continuous and autonomous security assessment. This includes implementing automated scheduling mechanisms for periodic asset discovery and vulnerability scanning without manual intervention.

Moreover, an AI, driven analytics process can be conducted on previous scan records in order to recognize patterns, forecast the most dangerous routes of an attack, and continuously revise the risk scores accordingly. The system updates might even include features for automatic reactive measures through detected risk levels, thereby enabling a seamless change from a security risk assessment only to a proactive and intelligent security management solution.

Glossary

Definition	Term
Attack Surface Management	ASM
Continuous Threat Exposure Management	CTEM
Common Vulnerability and Exposure	CVE
Common Vulnerability Scoring System	CVSS
Cybersecurity and Infrastructure Security Agency	CISA
Hyper Text Markup Language	HTML
Cascading Style Sheet	CSS

CHAPTER 7

APPENDIX

Appendix 1

To improve clarity and readability, the system use case were divided into two separate diagrams based on user roles and responsibilities to helps in clearly identifying role specification, association threats and security mitigation.

As shown in *Figure 26*. The use case associated with the SOC Analysts roles that related to security monitoring and analysis, for each identified threats, appropriate mitigation are defining.

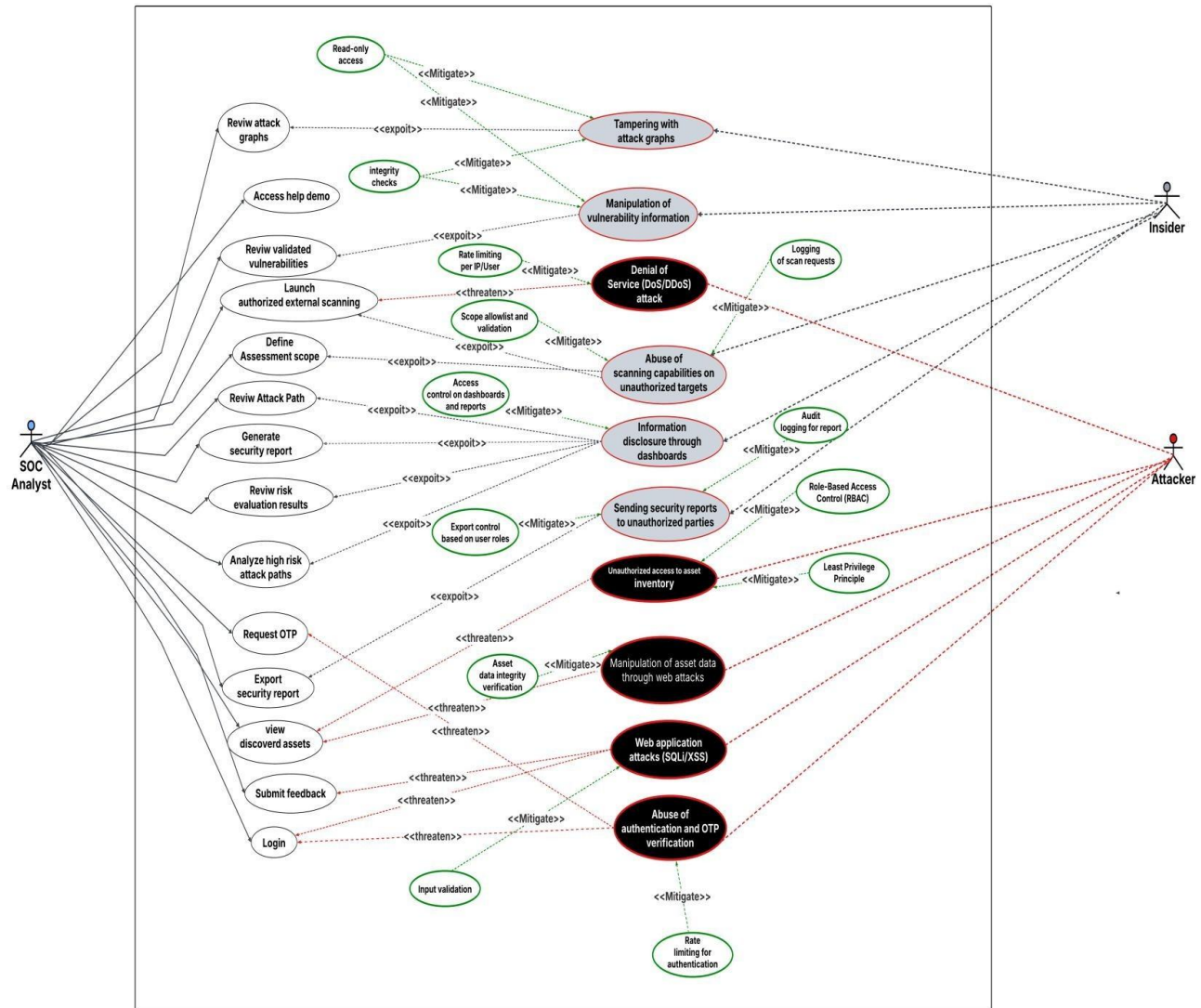


Figure 26: Secure Use Case Diagram for SOC Analyst.

As shown *Figure 27*. The use case associated with Administrator and OTP service roles focusing on system management and authentication, and for each threat, appropriate mitigation is defining.

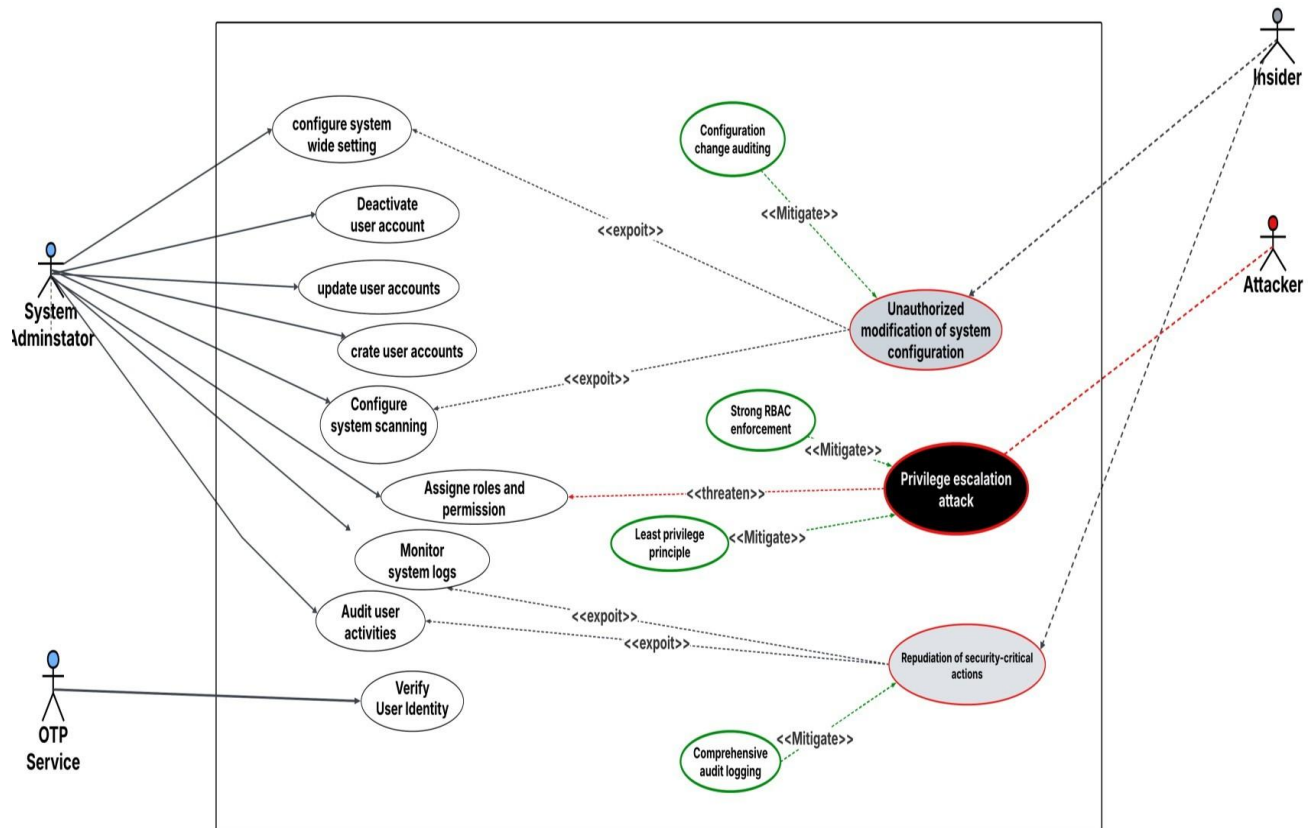


Figure 27: Secure Use Case Diagram for System Administrator and OTP Service

Appendix 2

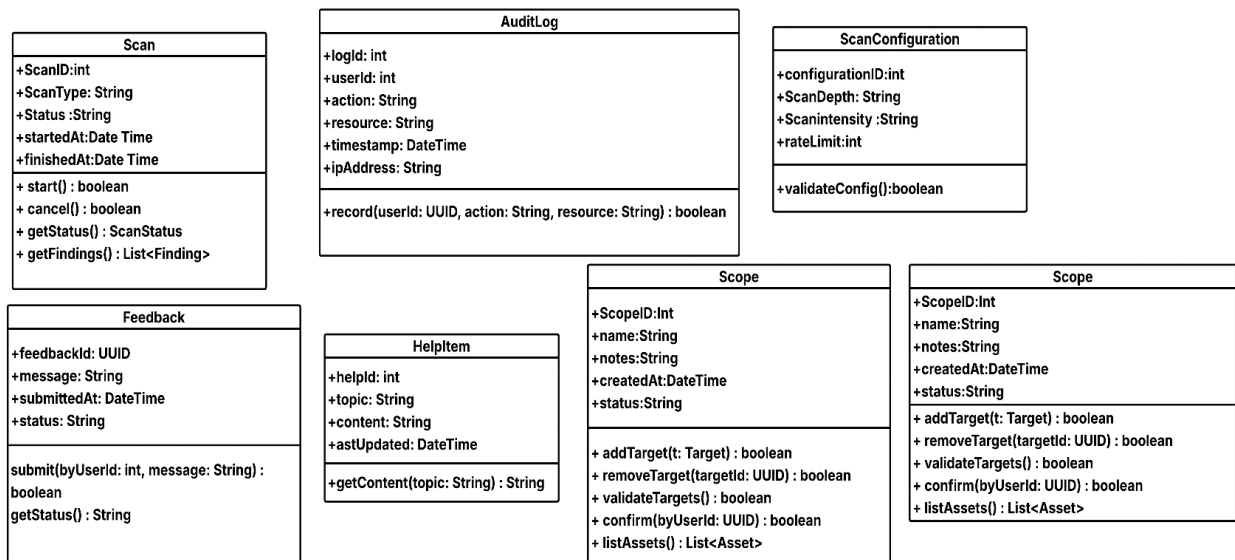


Figure 28: first: detailed view of system classes and their attributes and operations

Administrator
+adminID:int -email:String -password:String
+ createUser(fullName: String, email: String, roleName: String) : User + updateUser(userID: UUID, email: String, isActive: boolean) : User + deactivateUser(userID: UUID) : boolean + assignRole(userID: UUID, roleName: String) : boolean + grantPermission(roleName: String, permissionName: String) : boolean + revokePermission(roleName: String, permissionName: String) : boolean + approvePasswordReset(userID: UUID) : boolean + viewAuditLogs(from: DateTime, to: DateTime) : List<AuditLog> + configureScanningParameters(params: ScanConfig) : boolean + configureSystemSettings(settings: SystemSettings) : boolean + authorizeTarget(targetid:int): boolean + viewHelp(topic: String)

SOC Analyst
-email:String -password:String
-login(email,password):boolean -viewProfile() + requestOtp(email: String) : boolean + defineScope(name: String, targets: List<TargetInput>) : Scope + viewAssets(scopelid: UUID) : List<Asset> + viewAssetDetails(assetid: UUID) : Asset + viewVulnerabilities(assetid: UUID) : List<Vulnerability> + filterVulnerabilities(scopelid: UUID, severity: String, assetid: UUID) : List<Vulnerability> + startExternalScan(scopelid: UUID) + getScanStatus(scanid: UUID) : String + viewAttackPaths(scopelid: UUID) : List<AttackPath> + viewAttackPathSteps(pathid: UUID) : List<AttackStep> + viewAttackGraph(scopelid: UUID) + analyzeRisk(scopelid: UUID) + getPrioritizedThreats(scopelid: UUID) : List<RiskItem> + generateReport(scopelid: UUID, type: ReportType) : boolean + exportReport(reportid: UUID, format: ExportFormat) : File + submitFeedback(message: String) + viewHelp(topic: String)

1

Figure 29: Second detailed view of system classes and their attributes and operations

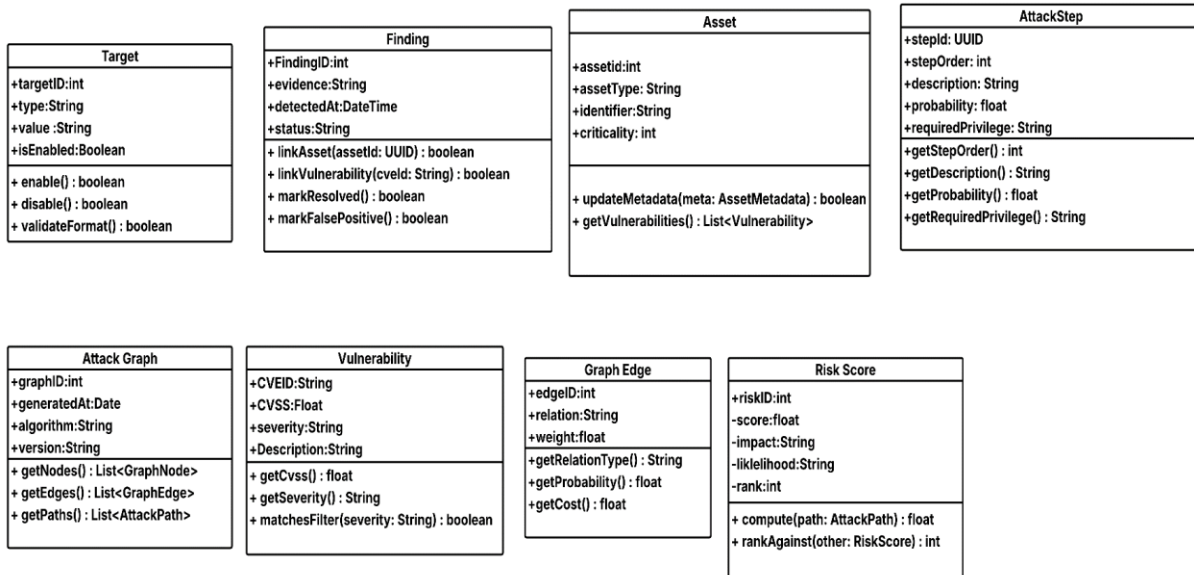


Figure 30: Third detailed view of system classes and their attributes and operations

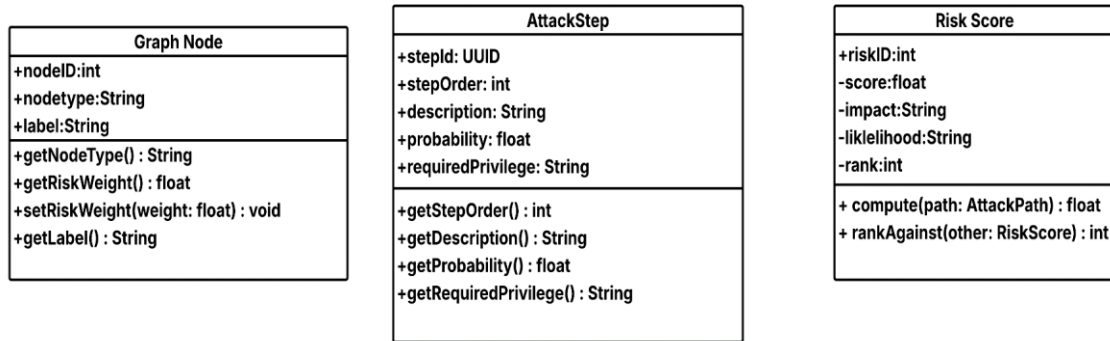


Figure 31: Fourth detailed view of system classes and their attributes and operations

Appendix 3

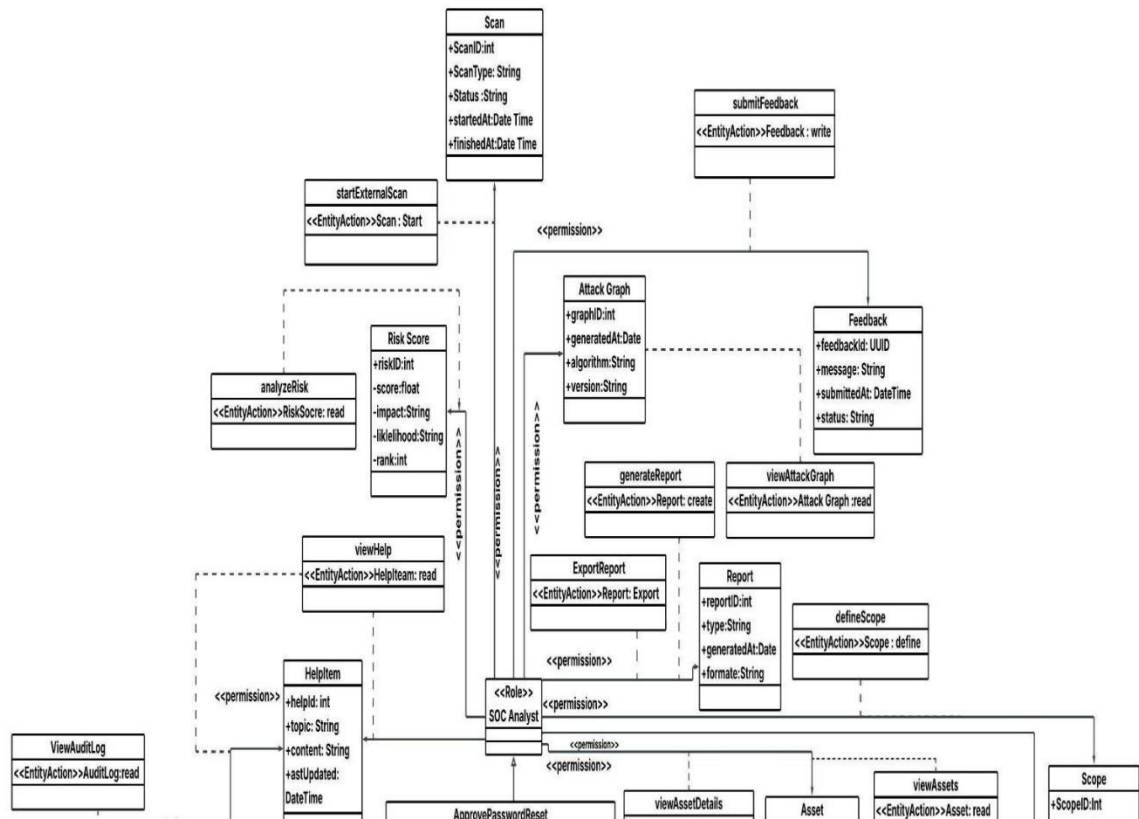


Figure 32: First detailed Secure UML class diagram illustrating roles, permissions, and protected system actions

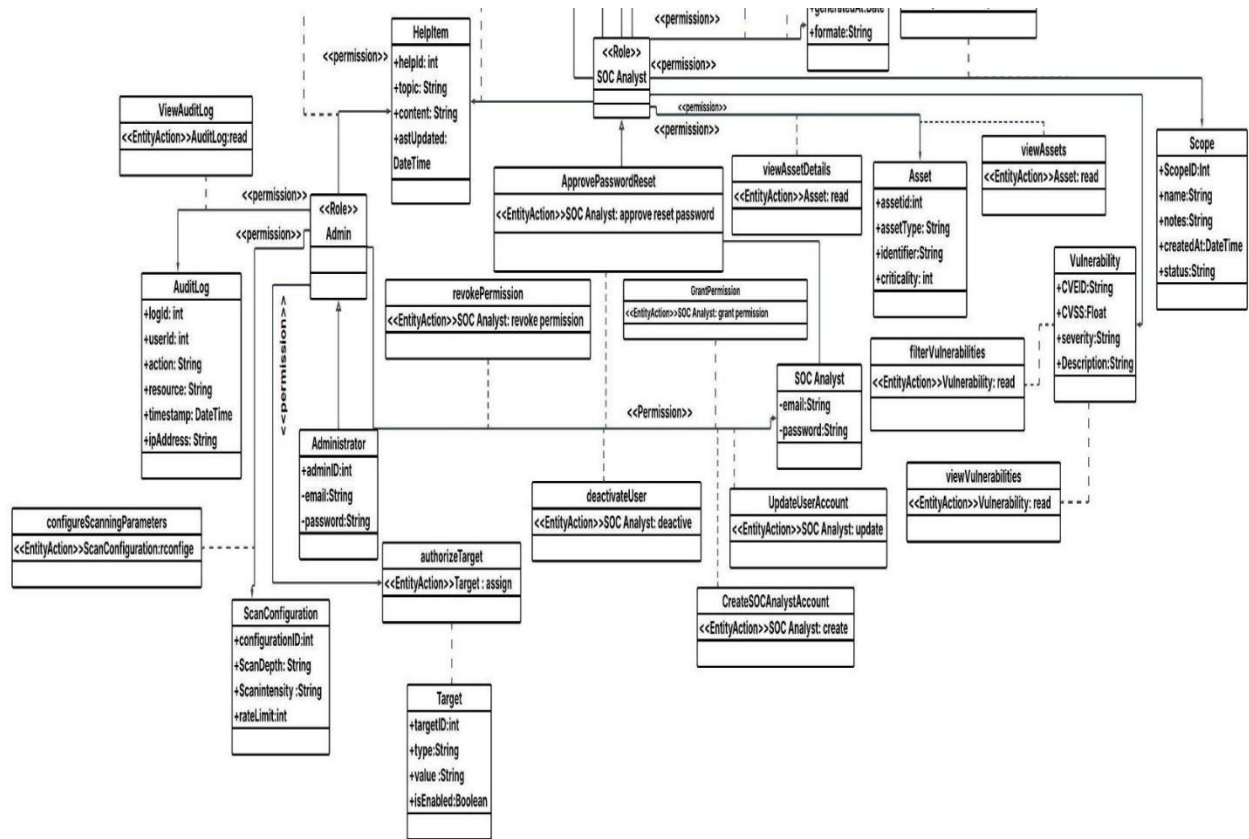


Figure 33: Second detailed Secure UML class diagram illustrating roles, permissions, and protected system actions

Appendix 4

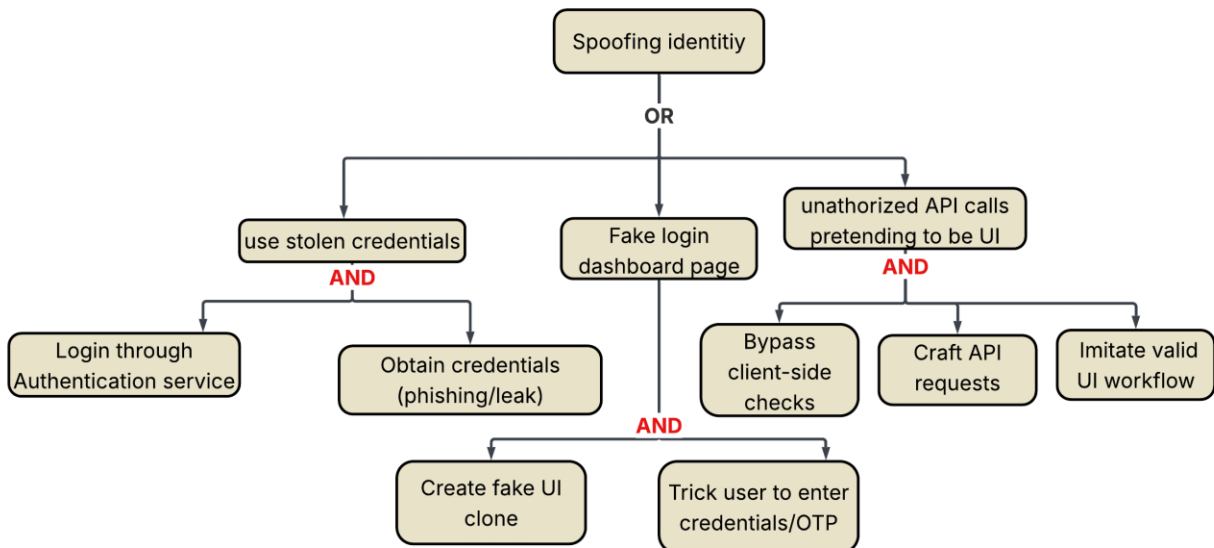


Figure 34: Attack tree representing spoofing identity threats and possible attack paths

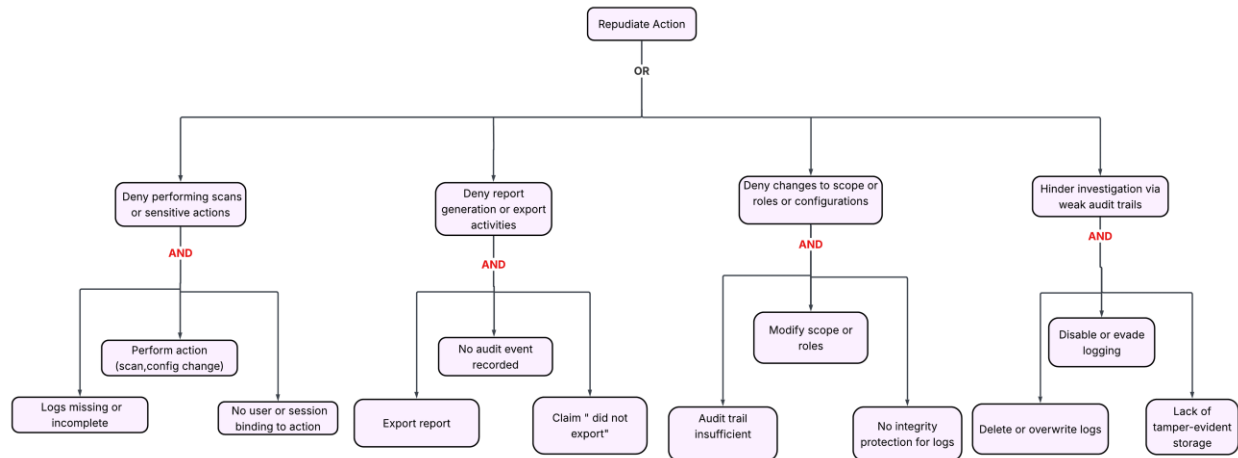


Figure 35: Attack tree representing repudiation action threats and possible attack paths

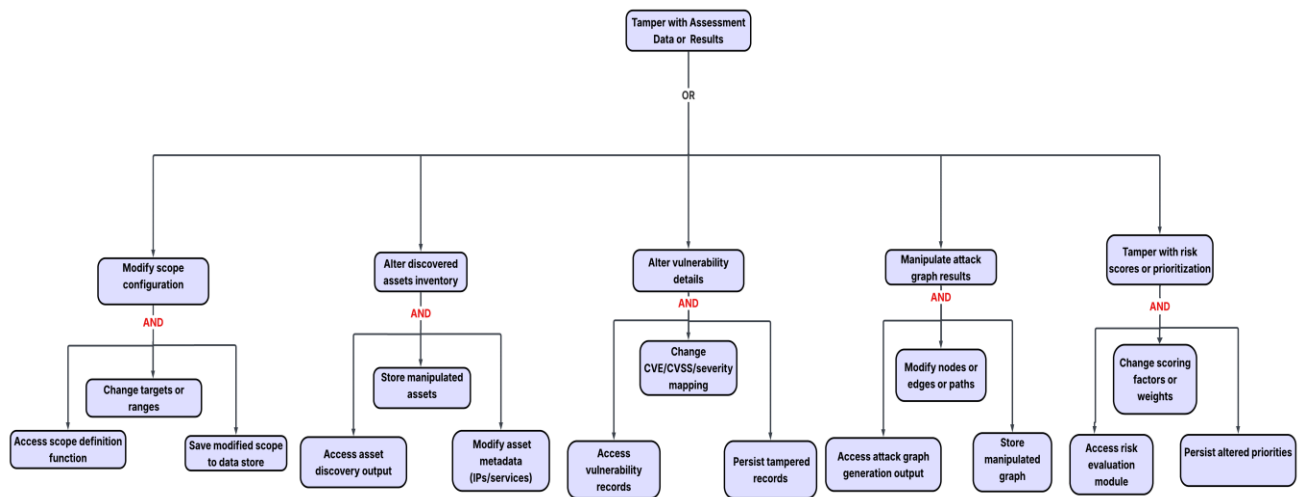


Figure 36: Attack tree representing tampering with assessment data or results threats and possible attack paths

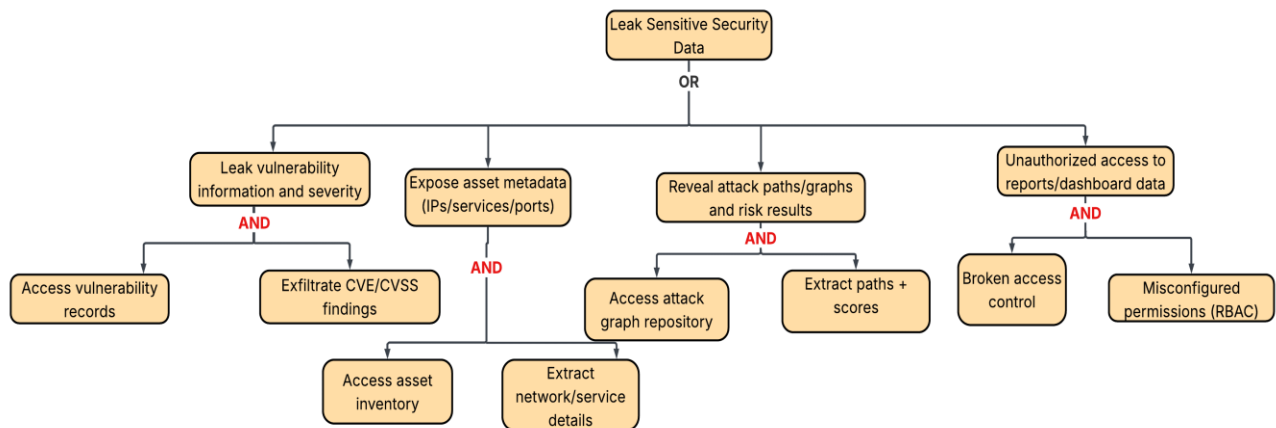


Figure 37: Attack tree representing leak sensitive security data threats and possible attack paths.

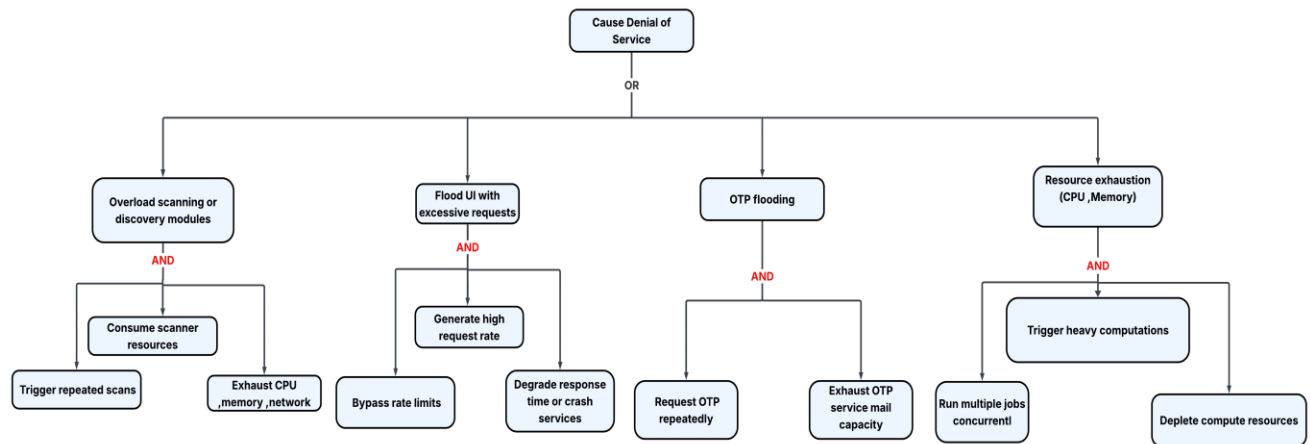


Figure 38: Attack tree representing case denial of service threats and possible attack paths.

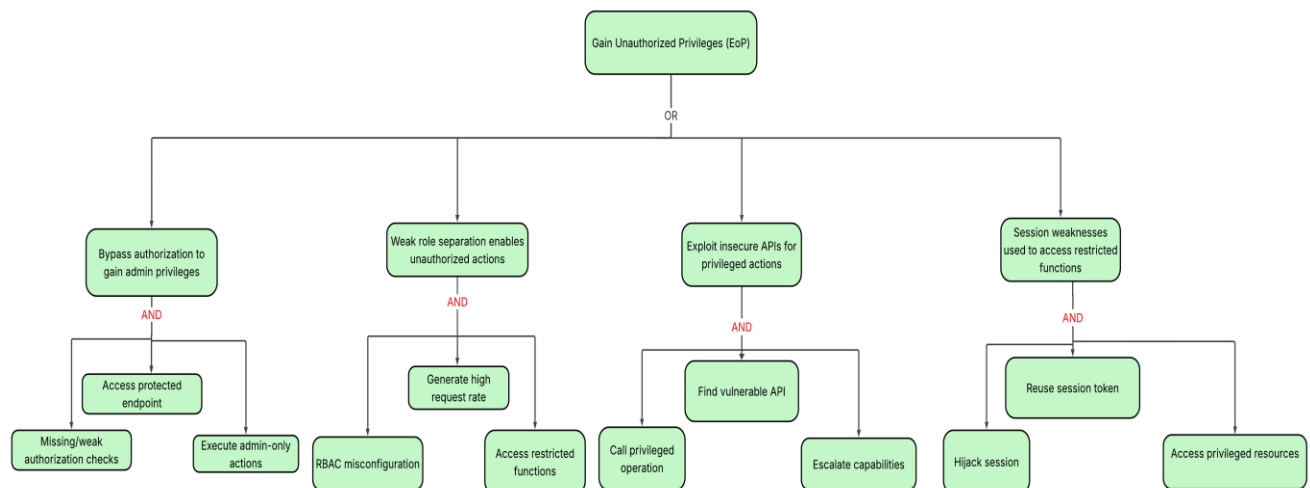


Figure 39: Attack tree representing gain of privilege action threats and possible attack paths

Appendix 5

tools		authorized_employees		forgot_password_email_attempts		notifications		SOCNotifications		subdomains	
id_p	int(11) NN	id_p	int(11) NN	id_p	int(11) NN	notification_id_p	int(11) NN	notification_id_p	int(11) NN	id_p	int(11) NN
name	varchar(50) NN	full_name	varchar(100) NN	email	varchar(255) NN	actor_user_id	char(30) NN	title	varchar(255) NN	scan_id	char(30) NN
description	text	email	varchar(150) NN	attempt_count	int(11) NN	target_role	varchar(50) NN	message	text NN	domain	varchar(255) NN
enabled	tinyint(1)	organization	varchar(150)	locked_until	datetime	title	varchar(255) NN	type	enum('info, warning, critical, success')	subdomain	varchar(255) NN
		is_active	tinyint(1) NN	last_attempt_at	datetime	message	text NN	is_read	tinyint(1)	ip_address	varchar(45) NN
		created_at	timestamp NN	created_at	timestamp NN	type	varchar(50) NN	created_at	timestamp NN	status	enum('up, down, unknown')
				updated_at	timestamp NN	is_read	tinyint(1) NN	user_id	char(30) NN	has_web_service	tinyint(1) NN
						created_at	datetime NN			web_status_code	int(11) NN
										web_title	varchar(255) NN
										web_technologies	text NN
										web_url	varchar(1000) NN
										risk_level	enum('low, medium, high, critical, unknown')
										vulnerability_count	int(11) NN
										vulnerabilities_summary	text NN
										last_checked_at	timestamp NN
										discovered_at	timestamp NN

Figure 40: First detailed ERD

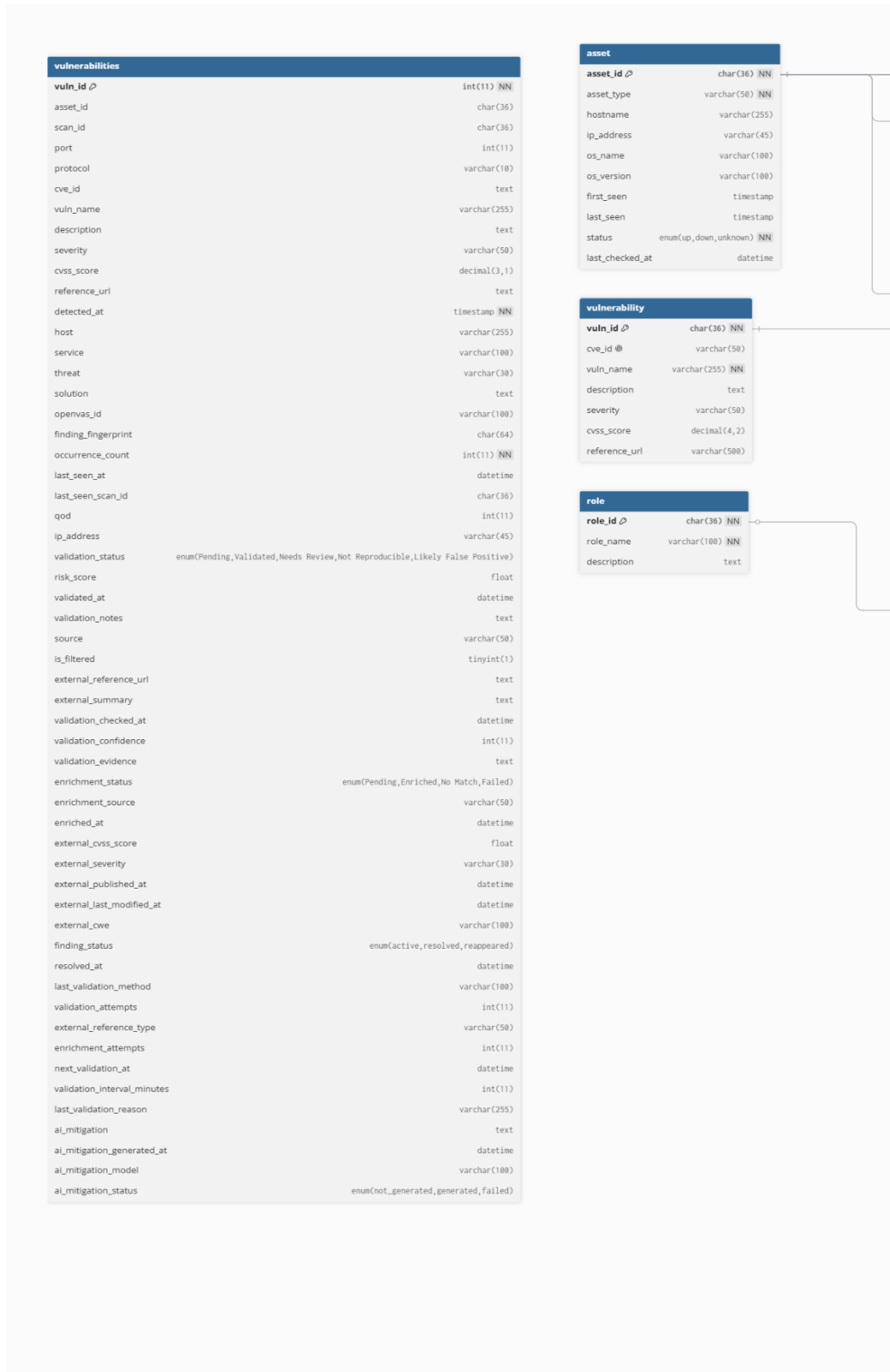


Figure 41: Second detailed ERD

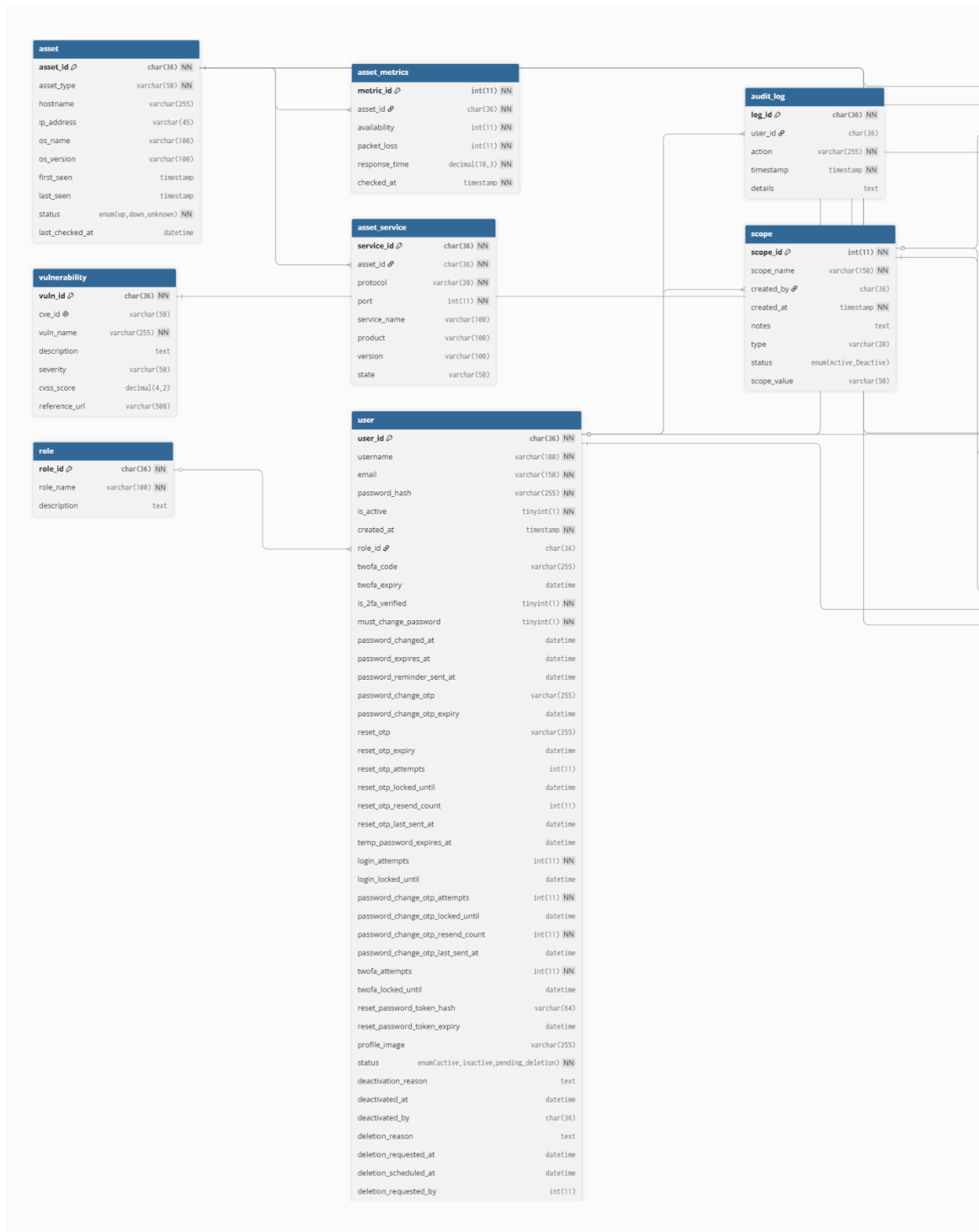


Figure 42: Third detailed ERD

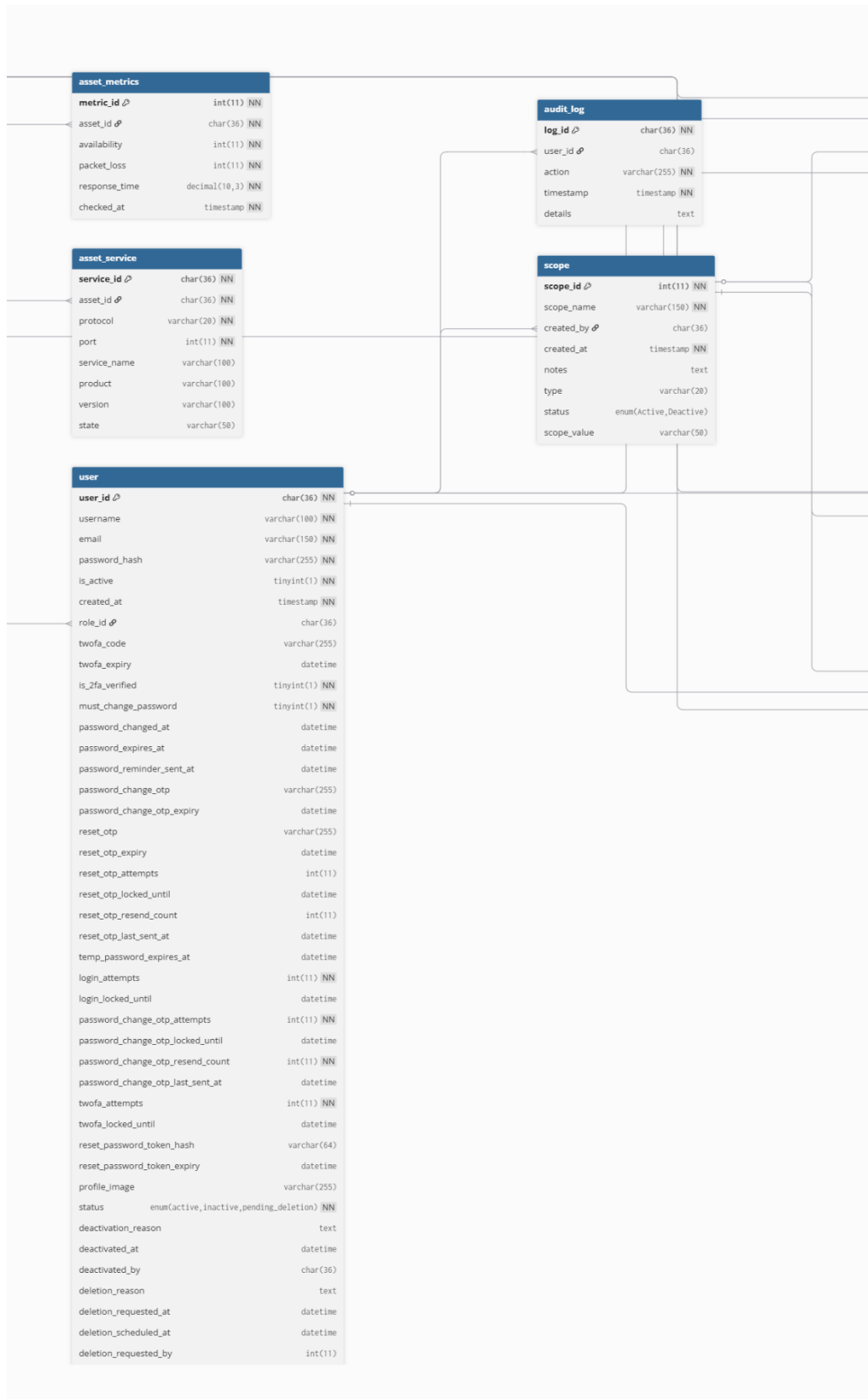


Figure 43: Fourth detailed ERD

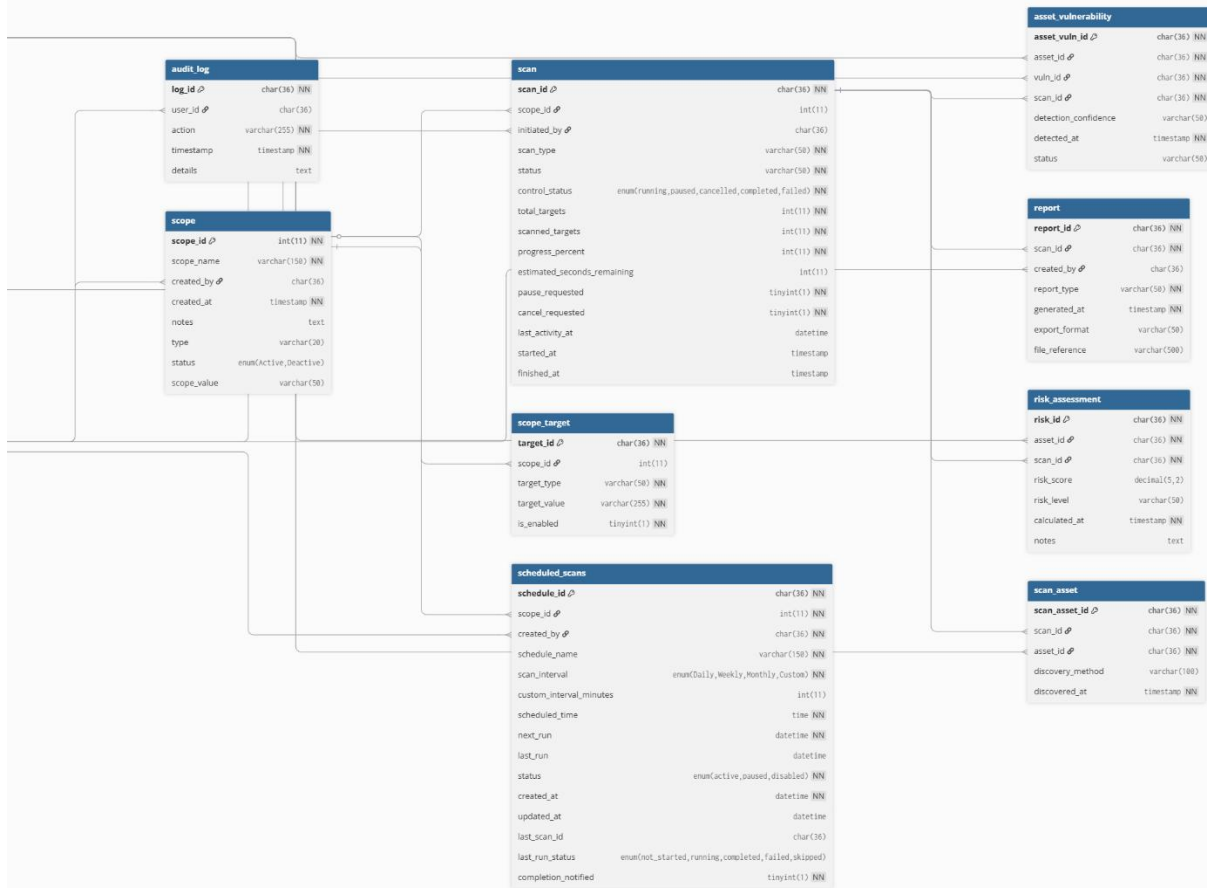


Figure 44: Fifth detailed ERD

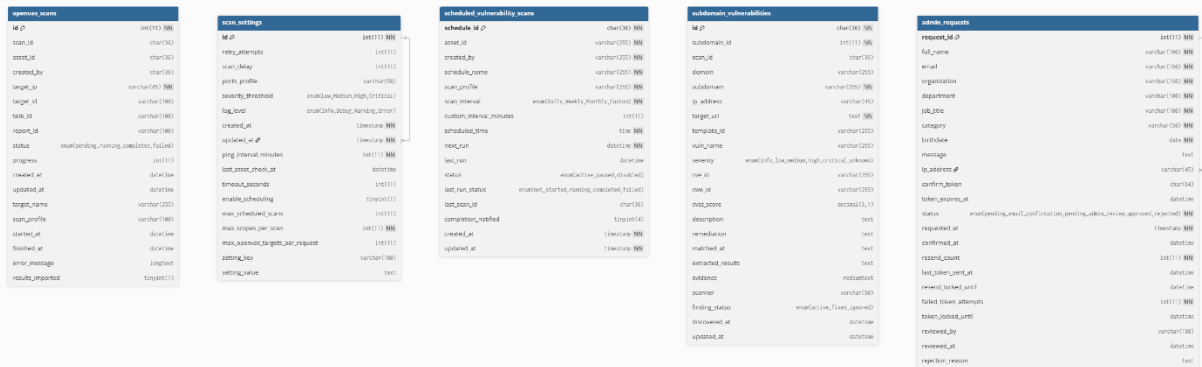


Figure 45: Sixth detailed ERD

Appendix 6

In this section, we will present the details of the test cases that have been performed for all the functions of the system.

In this section, we will present the details of the test cases that have been performed for all the functions of the system. To make sure that all of the following tests are performed (System Testing, Integration Testing, Authentication Testing), the figures from *Figure 46* to *Figure 69* will display all the functions.

First, login and enter the required information and test case 1 will be taken in *Table 12*.

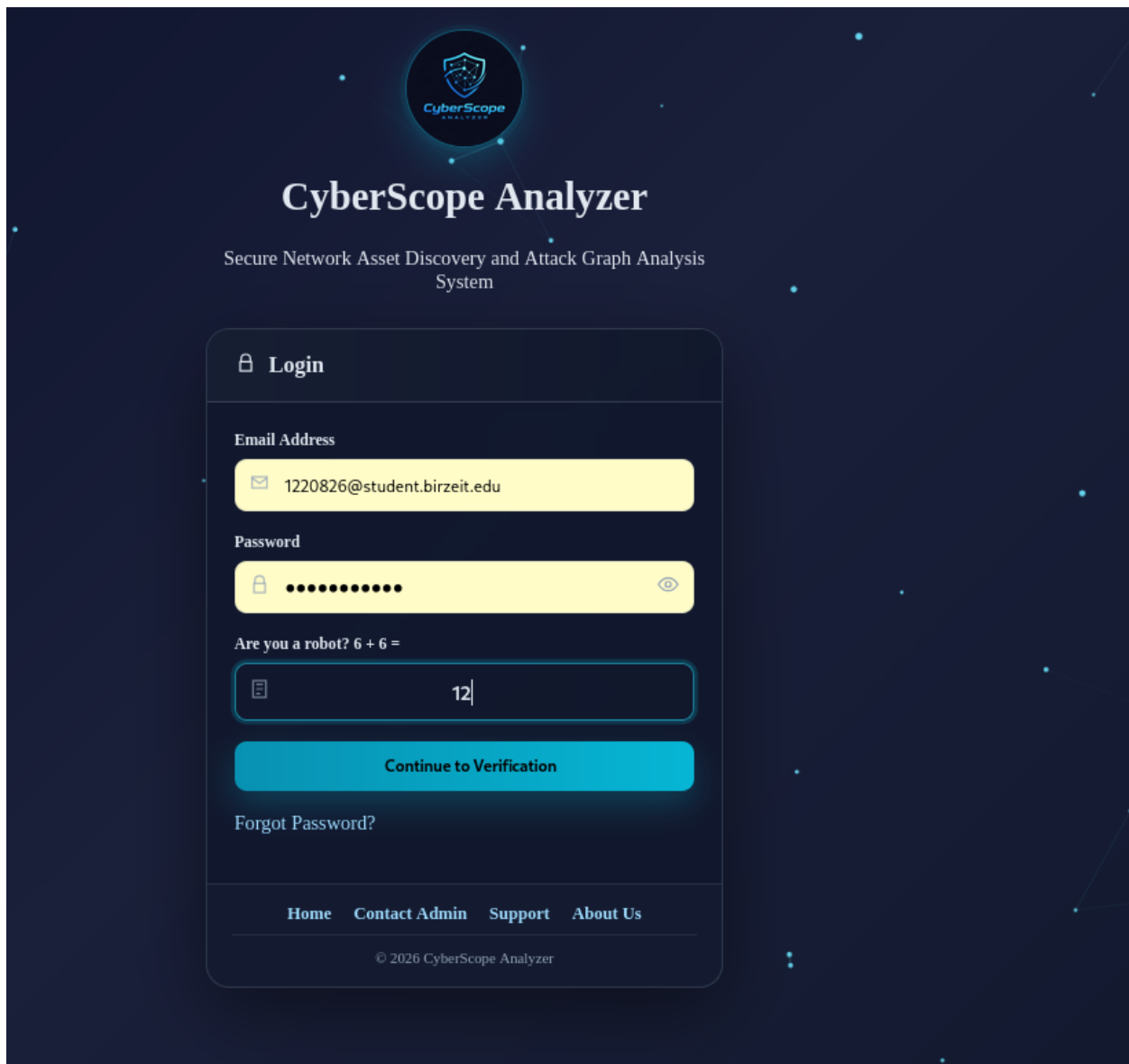


Figure 46: Login to the site

Second, Two-factor authentication and entering the OTP number sent to the email, and test case 2 will be taken in *Table 12*.

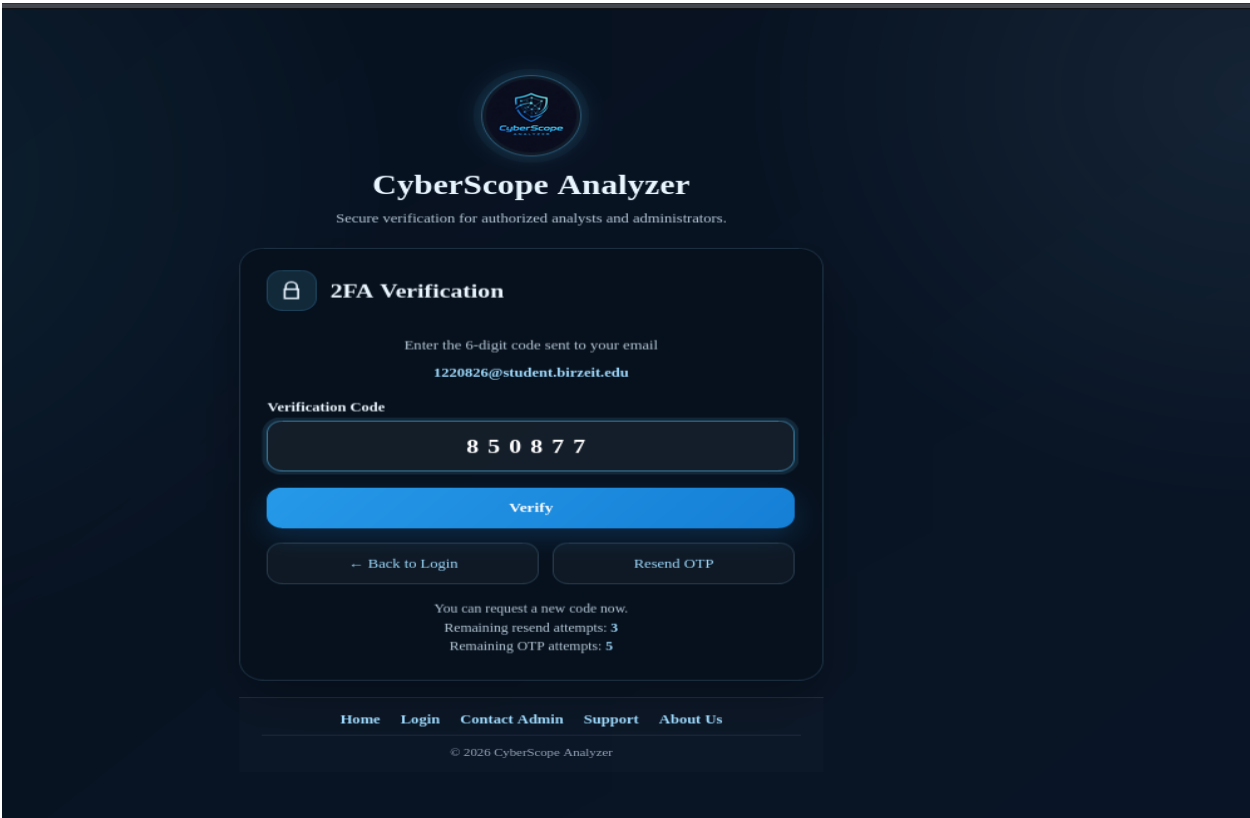


Figure 47: Two factor authentication

If the authentication process is successful, the system will be accessed, specifically the dashboard interface.

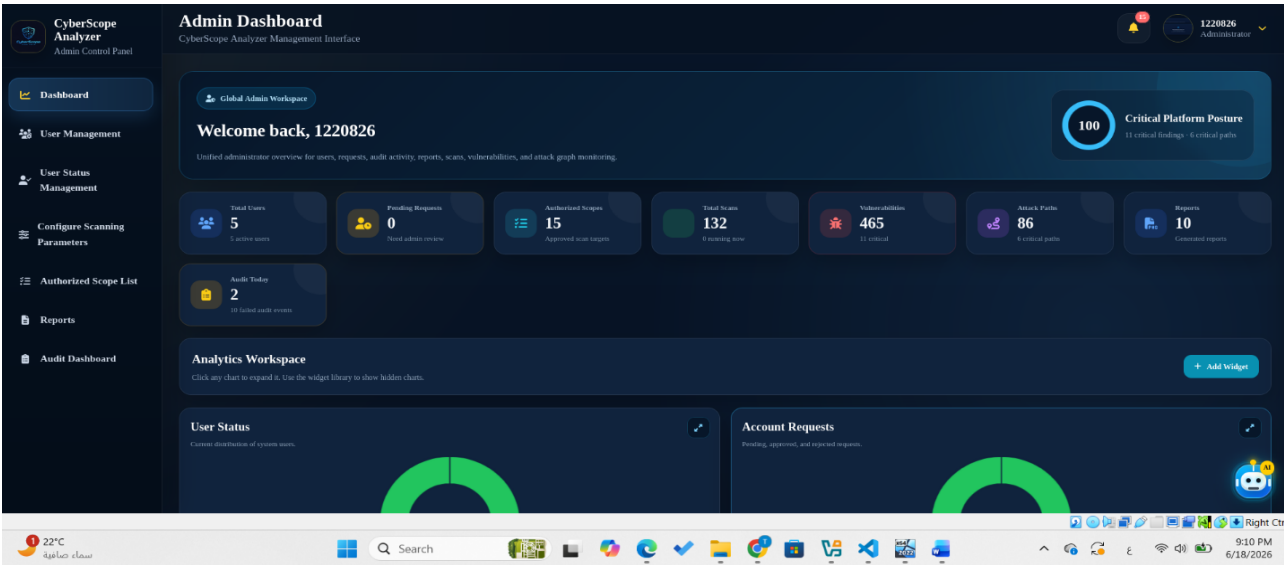


Figure 48: Redirect to dashboard

To create the scanning process the scope must be selected and test case 3 will be taken in *Table 12*.



Figure 49: The scan process is underway

After the completion of the scan process, all the discovered assets are displayed with their information as shown in *Figure 50*.

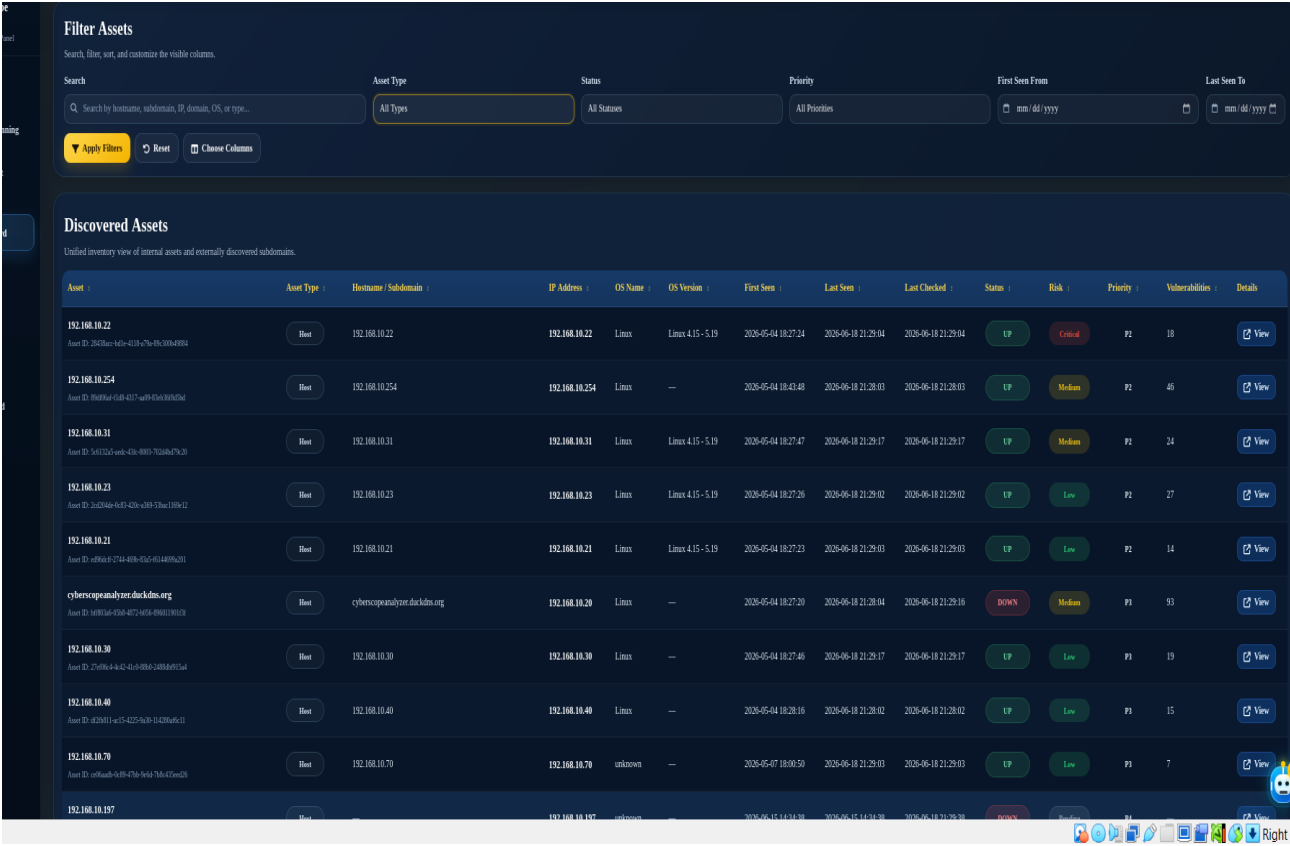


Figure 50: View discovered assets

To create the vulnerability scanning process, the asset must be selected and test case 4 will be taken in *Table 12*.

New Vulnerability Task

Select your target to perform a vulnerability scan.

Name

CyberScope Vulnerability Scan

Scan Profile

Full and Fast

Only profiles allowed by Admin settings are shown.
Allowed severities: Low, Medium, High, Critical

Comment

Optional note for this vulnerability assessment

Min QoD

70

Execution

Run Now

Scan Targets

Assets and discovered subdomains. Admin limit: maximum 3 target(s) per request.

☐

192.168.10.22
192.168.10.22
80/tcp http

ASSET | UP

☐

192.168.10.31
192.168.10.31
21/tcp ftp

ASSET | UP

☐

192.168.10.30
192.168.10.30
3306/tcp mysql

ASSET | UP

☒

192.168.10.23
192.168.10.23
80/tcp http

ASSET | UP

☐

192.168.10.60
192.168.10.60
554/tcp rtsp

ASSET | UP

☐

192.168.10.20
cyberscopeanalyzer.duckdns.org
21/tcp ftp, 443/tcp http, 3306/tcp mysql, 8000/tcp http-alt

ASSET | DOWN

☒ Add results to Assets
☒ Apply Overrides

Cancel

Save & Start

Figure 51: Select Asset for vulnerability scanning

OpenVAS Scan Tasks

Search IP profile name...

Running

All Severity

All Priority

Filter CVE...

Disable click any row to open the detailed task view.

TARGET	STATUS	PROFILE	PROGRESS	FINDINGS	SEVERITY	PRIORITY	LAST UPDATE	ACTIONS
192.168.10.23 192.168.10.23	Running	Full and Fast	75%	1	NA	High	2025-06-18 21:52:20	0 ⓘ

Figure 52: Starting vulnerability scanning process

TARGET	STATUS	PROFILE	PROGRESS	FINDINGS	SEVERITY	PRIORITY	LAST UPDATE	ACTIONS
192.168.10.23 192.168.10.23	Running	Full and Fast	90%	1	NA	High	2025-06-18 22:11:36	0 ⓘ

Figure 53: The vulnerability scanning process is ongoing

192.168.10.23 192.168.10.23	Completed	Full and Fast	100%	1	2 (Low)	High	2025-06-18 22:14:09	0 ⓘ
--------------------------------	-----------	---------------	------------------	---	---------	------	---------------------	-----

Figure 54: Vulnerability scanning process completed

98

After the vulnerability scanning process is complete, all discovered vulnerabilities are displayed with their information as shown in *Figure 55*.

Prioritized Technical Findings

Click any table header to sort ascending/descending.

Vulnerability	CVE	Port	Service	CVSS	Severity	Validation	Priority	Detected	Action
ICMP Timestamp Reply Information Disclosure 192.168.10.21	CVE-1999-0524	0	general/tcp	2.1	Low	Needs Review	P3 Medium - P3	2026-06-18 23:14:09	View
Allowed HTTP Methods Enumeration 192.168.10.21	N/A	80	80/tcp	0.0	Info	Needs Review	P3 Medium - P3	2026-05-09 21:08:02	View
HTTP Security Headers Detection 192.168.10.21	N/A	80	80/tcp	0.0	Info	Needs Review	P3 Medium - P3	2026-05-09 21:08:02	View
HTTP Server Banner Enumeration 192.168.10.21	N/A	80	80/tcp	0.0	Info	Needs Review	P3 Medium - P3	2026-05-09 21:08:02	View
HTTP Server type and version 192.168.10.21	N/A	80	80/tcp	0.0	Info	Needs Review	P3 Medium - P3	2026-05-09 21:08:02	View
IP Forwarding Enabled - Active Check 192.168.10.21	CVE-1999-0511	0	general/tcp	0.0	Info	Needs Review	P3 Medium - P3	2026-05-09 21:08:02	View
Services 192.168.10.21	N/A	80	80/tcp	0.0	Info	Needs Review	P3 Medium - P3	2026-05-09 21:08:02	View
Web Application Scanning Consolidation / Info Reporting 192.168.10.21	N/A	80	80/tcp	0.0	Info	Needs Review	P3 Medium - P3	2026-05-09 21:08:02	View
TCP Timestamps Information Disclosure 192.168.10.21	N/A	0	general/tcp	2.6	Low	Pending	P4 Low - P4	2026-05-09 21:08:02	View

Figure 55: View discovered vulnerabilities

If a high-stakes vulnerability is found, an urgent email is sent to the SOC Analyst as shown in *Figure 56*.

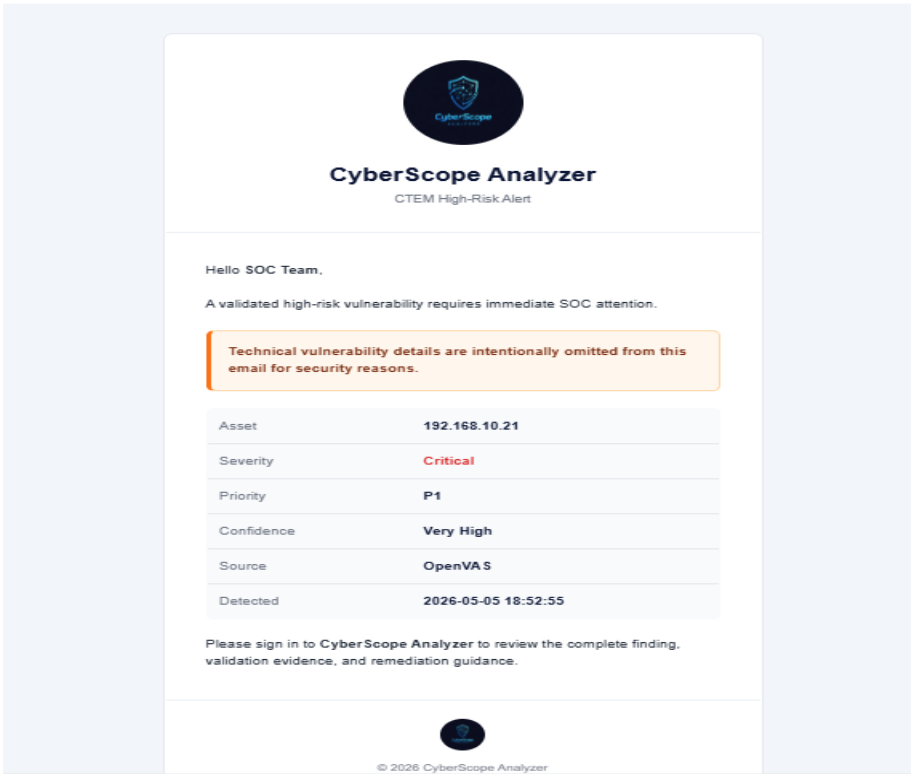


Figure 56: Send an urgent email to SOC Analyst

After the entire scanning of the selected scope is complete, the Attack Graph is created as shown in *Figure 57*, and all possible paths are output as shown in *Figure 58*.

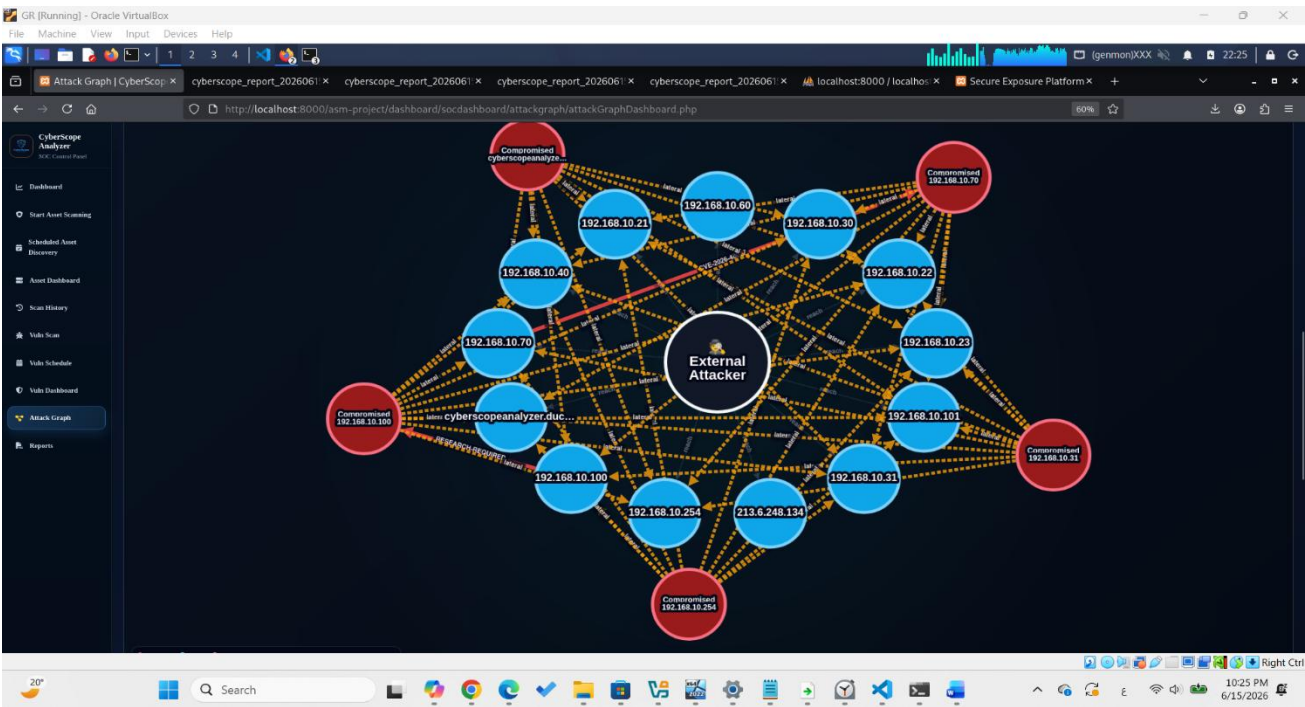


Figure 57: Graph Attack Offer after its completion

Search, filter, paginate, highlight, or open simulation in Attack Path Details								
Search paths by IP, MITRE, CVE, evidence...					All Status	All Priority	Highest Risk	Previous Page 1 / 1 Next
RANK	PRIORITY	STATUS	RISK	STEPS	MAIN EVIDENCE	MITRE	SIMULATE	DETAILS
#1	Critical	complete	100.00%	1	can_reach	T1595 Active Scanning • T1190 Exploit Public-Fa...	Simulate	Details
#2	Critical	complete	95.27%	2	can_reach	T1595 Active Scanning • T1552 Unsecured Credent...	Simulate	Details
#3	Critical	complete	93.35%	1	can_reach	T1595 Active Scanning • T1677 Poisoned Pipeline...	Simulate	Details
#4	High	complete	82.22%	4	can_reach	T1595 Active Scanning • T1552 Unsecured Credent...	Simulate	Details
#5	High	complete	82.03%	2	can_reach	T1595 Active Scanning • T1552 Unsecured Credent...	Simulate	Details
#6	High	complete	81.56%	2	can_reach	T1595 Active Scanning • T1552 Unsecured Credent...	Simulate	Details
#7	High	complete	79.87%	2	can_reach	T1595 Active Scanning • T1552 Unsecured Credent...	Simulate	Details
#8	High	partial	71.25%	1	can_reach	T1595 Active Scanning • T1082 System Informatio...	Simulate	Details
#9	Medium	partial	69.58%	1	can_reach	T1595 Active Scanning • T1082 System Informatio...	Simulate	Details
#10	Medium	partial	68.36%	1	can_reach	T1595 Active Scanning • T1040 Network Sniffing	Simulate	Details

Figure 58: View all detected attack paths

The user can also see all the steps of the chosen attack path and its information as shown in *Figure 59*, *Figure 60*, and *Figure 61*.

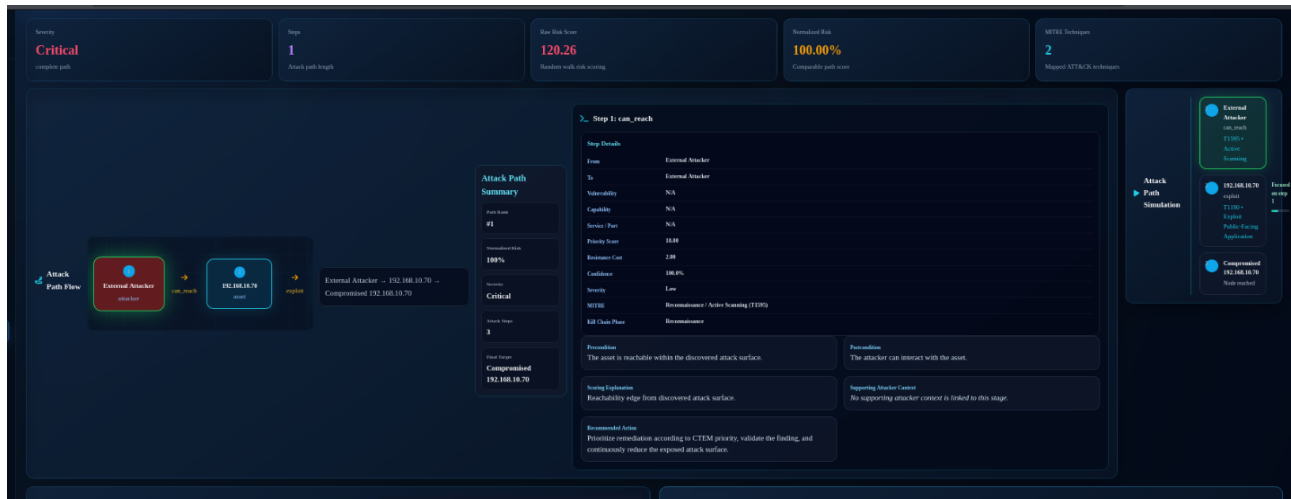


Figure 59: View the first step and its information from the chosen attack path.

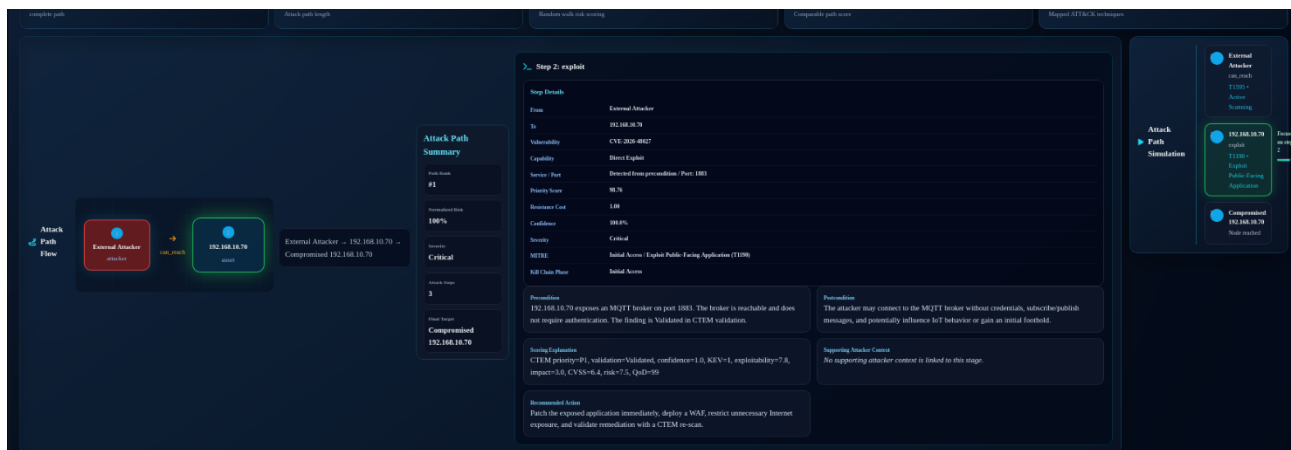


Figure 60: View the second step and its information from the chosen attack path.

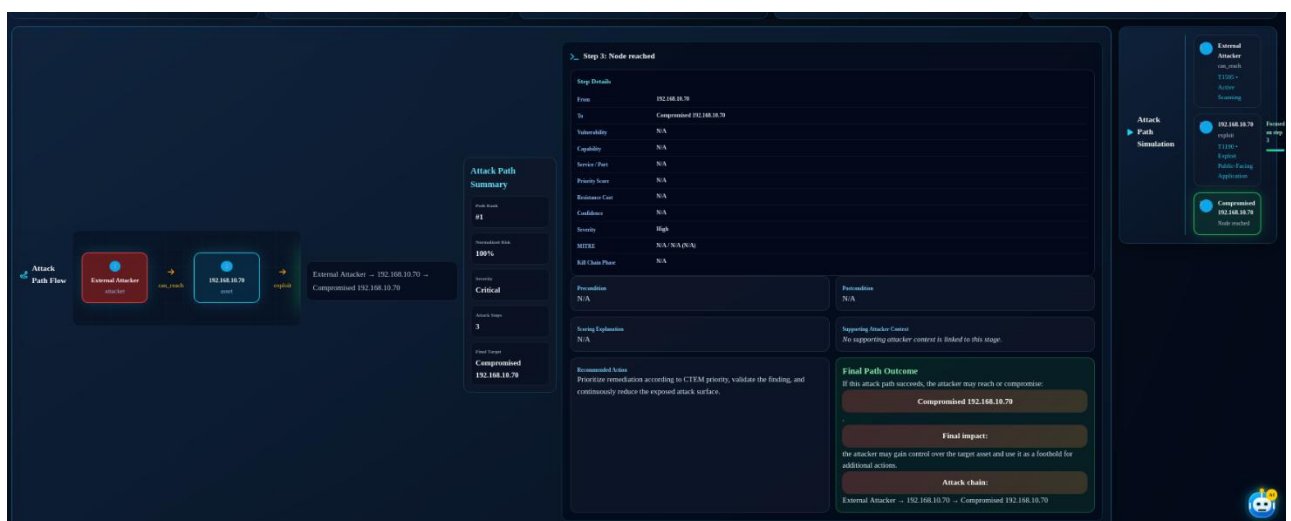


Figure 61: View the third step and its information from the chosen attack path.

To create the report, you must select the sections to be displayed in the report, and test case 7 will be taken *Table 12*.

Report Sections

Select the sections that should be included in the PDF report.

☒

Executive Summary
High-level summary for decision makers.

☒

Service / Open Ports Summary
Open ports and detected services.

☒

Top Critical Vulnerabilities
Most dangerous findings based on severity and risk.

☒

ASM Findings
External attack surface discovery results.

☒

Security Recommendations
Prioritized remediation actions.

☒

Asset Summary
Discovered hosts, services, and exposure overview.

☒

Vulnerability Summary
Severity distribution and vulnerability overview.

☒

Attack Graph Analysis
Attack paths, lateral movement, and MITRE mapping.

☒

CTEM Validation Results
Validated findings and false-positive reduction.

Generate PDF Report

Figure 62: Choose which sections to display in the report

CyberScope

CyberScope Analyzer

Security Assessment Report

Report

Overall Risk: CRITICAL

Generated By	tharwatassi (SOC)
Generated At	2026-06-15 19:09:16
Report Type	EXECUTIVE
Classification	Internal Use Only
Related Scan ID	4ccf5f22-baa2-40a6-a00a-547c3f447e0c
Scan Status	Completed
Scan Started	2026-06-15 14:19:33
Scan Finished	2026-06-15 15:50:09

Figure 63: Report Generated

To test on a scheduling process for the scan, test status 10 will be taken in *Table 12*.

Create Schedule

Choose a scope, interval, and run time.

Schedule Name

test

Scope

ScanMe Nmap Test | scanme.nmap.org | subdomain

Interval

Weekly

Scheduled Time

10 : 27 PM

+ Create Scheduled Scan

Figure 64: Enter the required information for the scan

Existing Scheduled Scans

Please, resume, disable, and monitor automated scan plans.

Schedule	Scope	Interval	Next Run	Schedule Status	Last Run	Created By	Actions
test Created: 2025-06-18 22:26:33	ScanMe Nmap Test scanme.nmap.org / subdomain	Weekly At 22:27:00	2025-06-18 22:27:00 Last: Not yet	Active	Not Started No scan yet	tharwatassi	Pause Disable Delete

Figure 65: Scan Scheduled

Existing Scheduled Scans

Please, resume, disable, and monitor automated scan plans.

Schedule	Scope	Interval	Next Run	Schedule Status	Last Run	Created By	Actions
internal network scanning Created: 2025-05-07 18:15:49	Internal Lab Network 192.168.10.0/24 / network	Weekly At 18:16:00	2025-06-23 18:16:00 Last: 2025-06-19 19:41:53	Active	Running Scan ID: 023Buc2...	tharwatassi	Pause Disable Delete

Figure 66: The scanning process started at the time specified

To test the vulnerability scheduling process, we will first enter the required information as shown in *Figure 67*, then it will be scheduled as shown in *Figure 68*, and it will run on time as shown in *Figure 69*.

Create Vulnerability Schedule

Select target, OpenVAS profile, interval, and run time.

Schedule Name

Testing Schedule vulnerability scanning

Target Asset

192.168.10.40 | 192.168.10.40

Scan Profile

Full and Fast

Interval

Weekly

Scheduled Time

10 : 10 PM

+ Create Vulnerability Schedule

Figure 67: Enter the required information for the scan table

TARGET	STATUS	PROFILE	PROGRESS	FINDINGS	SEVERITY	PRIORITY	LAST UPDATE	ACTIONS
192.168.10.40 192.168.10.40	Pending	Full and Fast	0%	0	N/A	P1 Low	2026-06-19 22:04:02	  

Figure 68: Scan Scheduled

192.168.10.40 192.168.10.40	Completed	Full and Fast	100%	2	2.0 (Low)	P1 Medium	2026-06-19 22:29:02	 
--------------------------------	-----------	---------------	------	---	-----------	-----------	---------------------	---

Figure 69: The scanning process started at the time specified

To make sure that the Authorization Test is performed, the figures from *Figure 70* to *Figure 71* will show the validation of the test.

In *Figure 70*, the SOC interface was accessed and then the URL was changed to try to access the admin interface, and as we will see in *Figure 71* we will see that the system was blocked from trying to access and the user was logged out of his account, as is also in the test case 8 in *Table 12*.

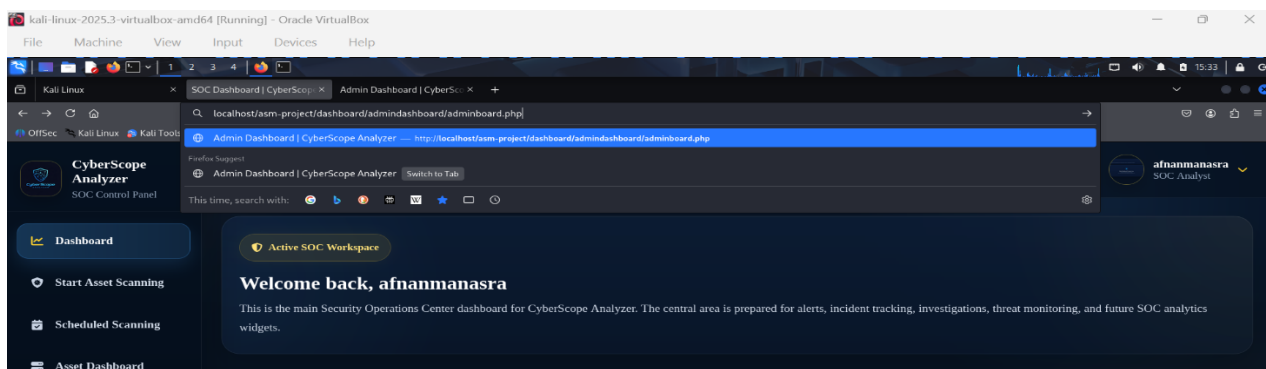


Figure 70: Trying to change the URL to access the admin interface from the SOC interface

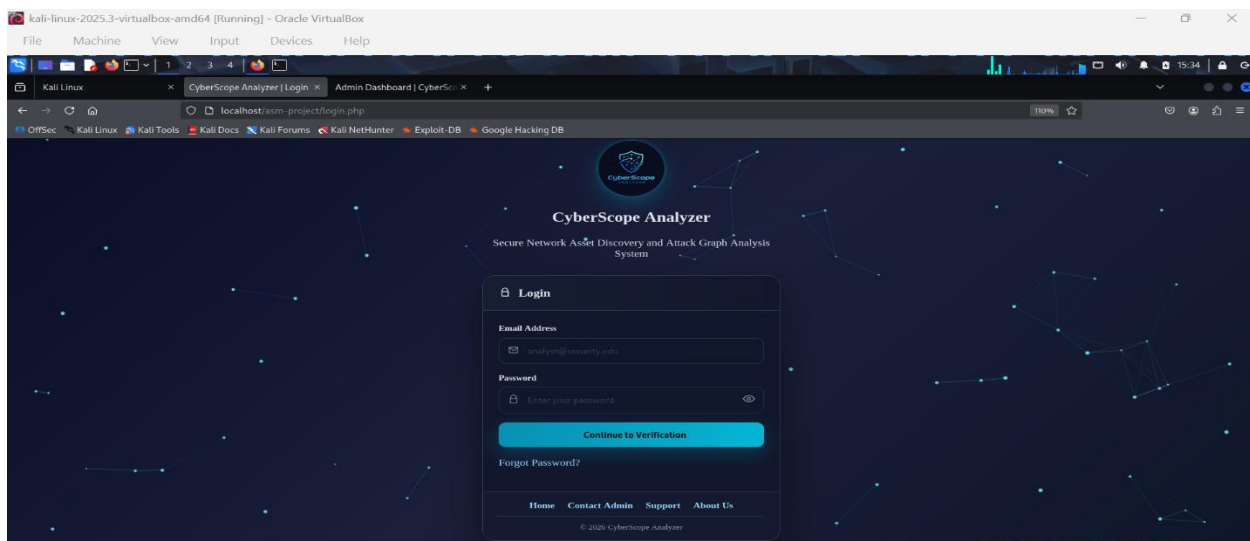
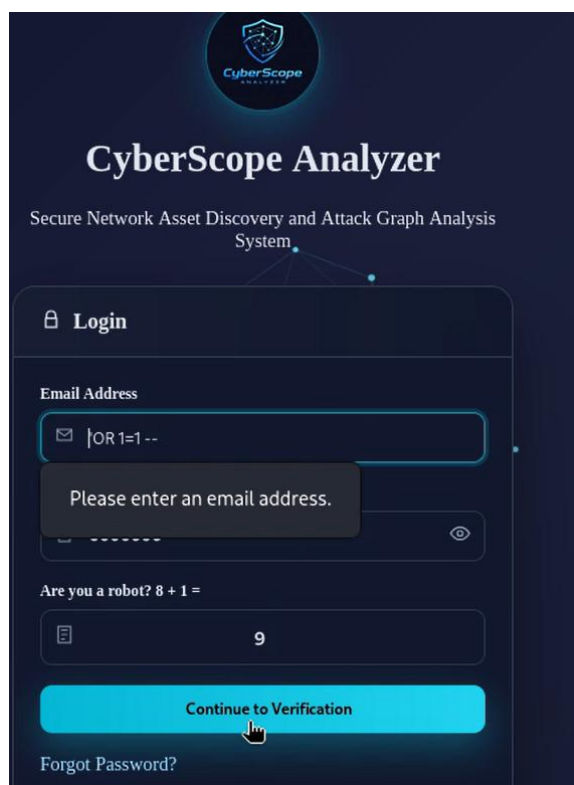


Figure 71: Block access and user log-out

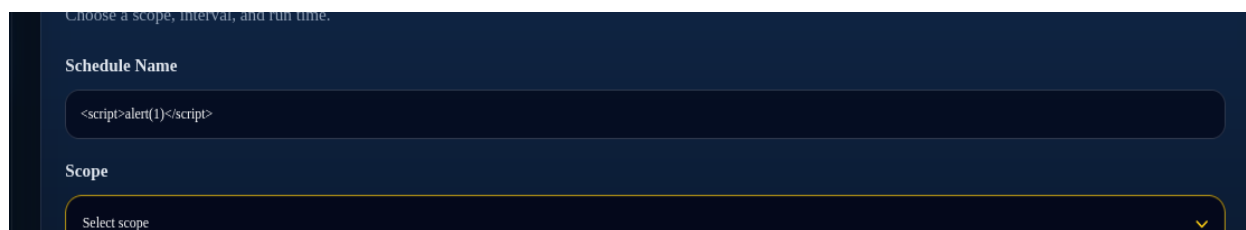
To make sure that the Input Validation Test is performed, the figures from *Figure 72* to *Figure 75* will show the test validation process.

In *Figure 72*, an attempt was made to inject a SQL query but was not executed, in *Figure 73* an attempt was made to inject JavaScript code but was not executed, in *Figure 74* an attempt was made to enter incorrect information, and in *Figure 75* an attempt was made to leave the input fields blank.



The screenshot shows the CyberScope Analyzer login interface. At the top, the logo and title "CyberScope Analyzer" are displayed, followed by the subtitle "Secure Network Asset Discovery and Attack Graph Analysis System". Below this is a "Login" section with an "Email Address" field. The field contains the text "OR 1=1 --" and a message "Please enter an email address." is shown below it. There is also a CAPTCHA question "Are you a robot? 8 + 1 =" with the answer "9" entered. A "Continue to Verification" button is at the bottom of the login section, and a "Forgot Password?" link is below it.

Figure 72: Try SQL injection



The screenshot shows the configuration page for the CyberScope Analyzer. At the top, it says "Choose a scope, interval, and run time." Below this is a "Schedule Name" field containing the text "<script>alert(1)</script>". There is also a "Scope" field with a dropdown menu showing "Select scope".

Figure 73: Try XSS attack

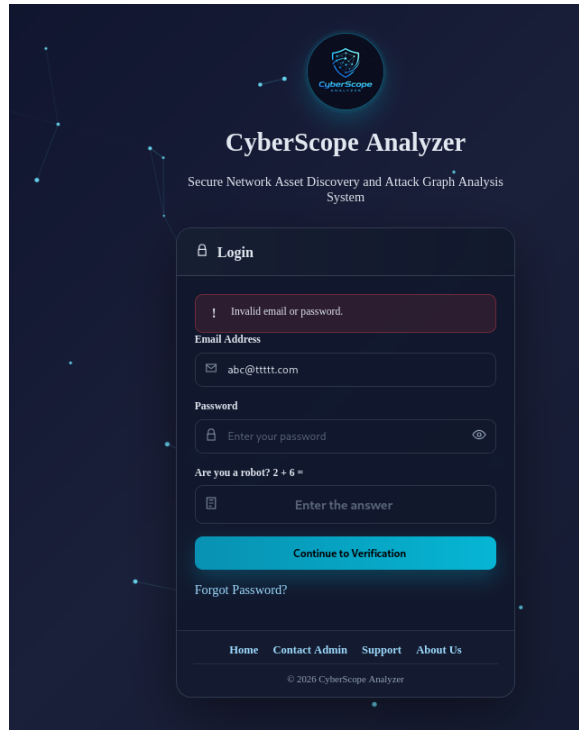


Figure 74: Try invalid credential

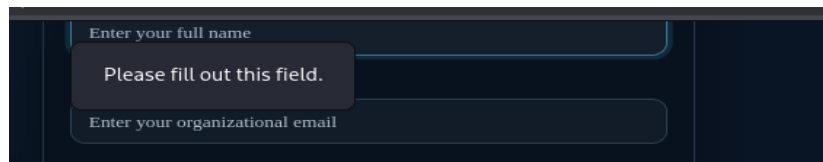


Figure 75: Try make the input field empty

To ensure that the Penetration Testing is performed and that the Dynamic Testing is performed, the OWASP ZAP tool has been used, *Figure 76* will show the test.

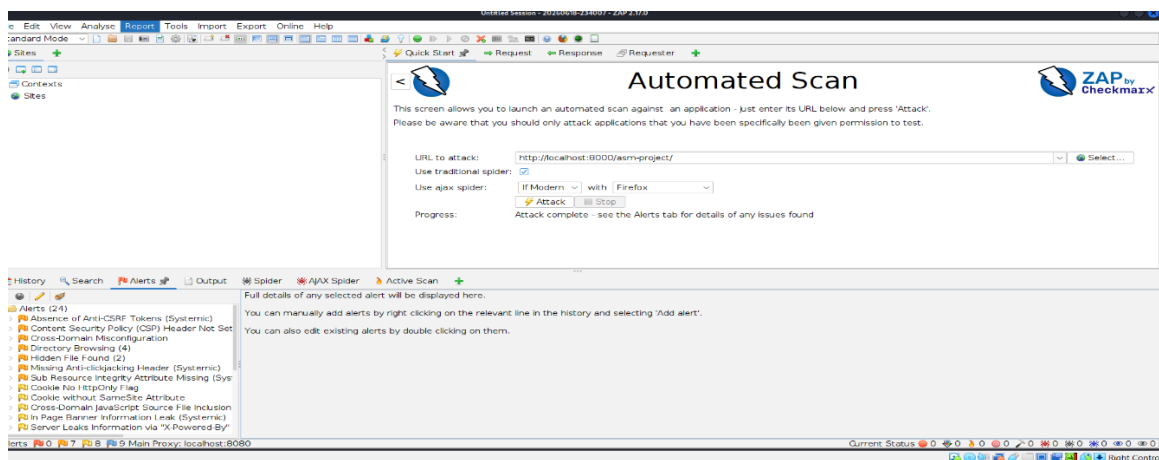


Figure 76: The test was completed and the vulnerabilities were exposed

In order to solve the problems that emerged, some of them were false positive by virtue of their pre-existing existence as shown in *Figure 77*, and some of them were modified and because they did not exist, and the modifications were as in *Figure 78*, *Figure 79*, and *Figure 80*.

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = '$_SESSION['csrf_token']'");
```

Figure 77: SQL Injection false positive

```
// CSRF token generation
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

Figure 78: Fixed CSRF on login page

```
// ...
if (!headers_sent()) {
    header("X-Frame-Options: DENY");
    header("X-Content-Type-Options: nosniff");
    header("Referrer-Policy: strict-origin-when-cross-origin");
    header("Permissions-Policy: geolocation=(), microphone=(), camera=()");
    header("Content-Security-Policy: default-src 'self'; img-src 'self' data:; font-src 'self' https://cdnjs.cloudflare.com data:; style-src 'self' https://cdnjs.cloudflare.com data:;");
}
```

Figure 79: Apply security headers to protect system sessions.

```
session_set_cookie_params([
    'lifetime' => 0,
    'path'     => '/',
    'domain'   => '',
    'secure'   => $isHttps,
    'httponly' => true,
    'samesite' => 'Strict'
]);
```

Figure 80: Apply security cookies parameters

Static testing was performed for Python codes as shown in *Figure 81*, JavaScript codes as shown in *Figure 82*, and PHP codes as shown in *Figure 83*.

```

        cve_id=vuln.get("cve_id"),
        vuln_name=vuln.get("vuln_name"),
        vulnerabilities=[vuln_info],
        vulnerability_count=1, (duplicate-code)
dashboard/socdashboard/attackgraph/engine/dijkstra_paths.py:1:0: R0801: Similar lines in 2 files
==generate_attack_graph:[964:970]
==generate_attack_graph_backup_components:[921:927]
        capability_type=capability,
        vuln_id=vuln.get("vuln_id"),
        cve_id=vuln.get("cve_id"),
        vuln_name=vuln.get("vuln_name"),
        vulnerabilities=[vuln_info],
        vulnerability_count=1, (duplicate-code)
dashboard/socdashboard/attackgraph/engine/dijkstra_paths.py:1:0: R0801: Similar lines in 2 files
==generate_attack_graph:[965:970]
==generate_attack_graph_backup_components:[1019:1024]
        vuln_id=vuln.get("vuln_id"),
        cve_id=vuln.get("cve_id"),
        vuln_name=vuln.get("vuln_name"),
        vulnerabilities=[vuln_info],
        vulnerability_count=1, (duplicate-code)
dashboard/socdashboard/attackgraph/engine/dijkstra_paths.py:1:0: R0801: Similar lines in 2 files
==generate_attack_graph:[1019:1024]
==generate_attack_graph_backup_components:[965:970]
        vuln_id=vuln.get("vuln_id"),
        cve_id=vuln.get("cve_id"),
        vuln_name=vuln.get("vuln_name"),
        vulnerabilities=[vuln_info],
        vulnerability_count=1, (duplicate-code)
dashboard/socdashboard/attackgraph/engine/dijkstra_paths.py:1:0: R0801: Similar lines in 2 files
==generate_attack_graph:[1031:1036]
==generate_attack_graph_backup_components:[1246:1251]
        kill_chain_phase=attack_context["kill_chain_phase"],
        mitre_tactic=attack_context["mitre_tactic"],
        mitre_technique=attack_context["mitre_technique"],
        mitre_technique_id=attack_context["mitre_technique_id"],
        mitre_confidence=attack_context["mitre_confidence"], (duplicate-code)
dashboard/socdashboard/attackgraph/engine/dijkstra_paths.py:1:0: R0801: Similar lines in 2 files
==generate_attack_graph:[1246:1251]
==generate_attack_graph_backup_components:[1031:1036]
        kill_chain_phase=attack_context["kill_chain_phase"],
        mitre_tactic=attack_context["mitre_tactic"],
        mitre_technique=attack_context["mitre_technique"],
        mitre_technique_id=attack_context["mitre_technique_id"],
        mitre_confidence=attack_context["mitre_confidence"], (duplicate-code)

Your code has been rated at 7.89/10

(tharwat@GR)~/opt/lampp/htdocs/asm-project
$

```

Figure 81: Python codes static testing

```

(tharwat@GR)~/opt/lampp/htdocs/asm-project
$ cat static_js_result.txt

/opt/lampp/htdocs/asm-project/dashboard/admindashboard/adminDashboard.js
311:27 warning 'performance' is not defined no-undef
320:17 warning 'requestAnimationFrame' is not defined no-undef
327:9 warning 'requestAnimationFrame' is not defined no-undef

/opt/lampp/htdocs/asm-project/dashboard/chatbot/chatbot.js
19:13 warning 'setTimeout' is not defined no-undef
72:30 warning 'AbortController' is not defined no-undef
110:22 warning 'e' is defined but never used no-unused-vars
138:9 warning 'requestAnimationFrame' is not defined no-undef

/opt/lampp/htdocs/asm-project/dashboard/socdashboard/SOCdashboard.js
133:27 warning 'performance' is not defined no-undef
137:35 warning 'requestAnimationFrame' is not defined no-undef
139:13 warning 'requestAnimationFrame' is not defined no-undef
160:9 warning 'setTimeout' is not defined no-undef
197:21 warning 'setTimeout' is not defined no-undef

/opt/lampp/htdocs/asm-project/dashboard/socdashboard/attackgraph/attackGraphDashboard.js
5:5 warning 'selectedPath' is assigned a value but never used no-unused-vars
1183:5 warning 'setTimeout' is not defined no-undef
1972:5 warning 'setTimeout' is not defined no-undef
1993:5 warning 'setTimeout' is not defined no-undef
2300:9 warning 'clearTimeout' is not defined no-undef
2301:19 warning 'setTimeout' is not defined no-undef

/opt/lampp/htdocs/asm-project/dashboard/socdashboard/attackgraph/attackPathDetails.js
208:23 warning 'setInterval' is not defined no-undef
236:23 warning 'setInterval' is not defined no-undef
262:9 warning 'clearInterval' is not defined no-undef
443:10 warning 'detailBox' is defined but never used no-unused-vars
509:14 warning 'e' is defined but never used no-unused-vars

/opt/lampp/htdocs/asm-project/eslint.config.js
1:12 warning 'require' is not defined no-undef
3:1 warning 'module' is not defined no-undef

* 25 problems (0 errors, 25 warnings)

(tharwat@GR)~/opt/lampp/htdocs/asm-project
$

```

Figure 82: Javascript codes static testing

```
163 Property Dompdf\Frame::$prev_sibling (Dompdf\Frame) does not accept null.
    assign.propertyType
165 Property Dompdf\Frame::$style (Dompdf\Css\Style) does not accept null.
    assign.propertyType
167 Property Dompdf\Frame::$containing_block (array<float>) does not accept array<string, null>.
    assign.propertyType
179 Property Dompdf\Frame::$position (array<float>) does not accept array<string, null>.
    assign.propertyType
188 Property Dompdf\Frame::$decorator (Dompdf\FrameDecorator\AbstractFrameDecorator) does not accept null.
    assign.propertyType
278 Property Dompdf\Frame::$style (Dompdf\Css\Style) does not accept null.
    assign.propertyType
279 Cannot unset property Dompdf\Frame::$style because it might have hooks in a subclass.
    unset.possiblyHookedProperty
934 Property Dompdf\Frame::$prev_sibling (Dompdf\Frame) does not accept null.
    assign.propertyType
940 Property Dompdf\Frame::$next_sibling (Dompdf\Frame) does not accept null.
    assign.propertyType
971 Property Dompdf\Frame::$next_sibling (Dompdf\Frame) does not accept null.
    assign.propertyType
977 Property Dompdf\Frame::$prev_sibling (Dompdf\Frame) does not accept null.
    assign.propertyType
1121 Property Dompdf\Frame::$next_sibling (Dompdf\Frame) does not accept null.
    assign.propertyType
1122 Property Dompdf\Frame::$prev_sibling (Dompdf\Frame) does not accept null.
    assign.propertyType
1123 Property Dompdf\Frame::$parent (Dompdf\Frame) does not accept null.
    assign.propertyType

Line vendor/dompdf/dompdf/src/Frame/Factory.php
198 PHPDoc tag @var with type Dompdf\FrameDecorator\AbstractFrameDecorator is not subtype of native type
    Dompdf\FrameDecorator\Block|Dompdf\FrameDecorator\Image|Dompdf\FrameDecorator\Inline|Dompdf\FrameDecorator\ListBullet|Dompdf\
\FrameDecorator\TableRowGroup|Dompdf\FrameDecorator\Text.
    varTag.nativeType
201 Class Dompdf\FrameReflower\Block constructor invoked with 2 parameters, 1 required.
    arguments.count

[ERROR] Found 1000+ errors

! [NOTE] Result is limited to the first 1000 errors
! - Consider lowering the PHPStan level
! - Pass PHPSTAN_TABLE_ERROR_FORMATTER_FORCE_SHOW_ALL_ERRORS=1 environment variable to show all errors
! - Consider using PHPStan Pro for more comfortable error browsing
! Learn more: https://phpstan.com

(tharwat@GR)-[/opt/lampp/htdocs/asm-project]
```

Figure 83: PHP codes static testing

References

- [1] A. Papathanasiou, G. Lontos, A. Katsouras, V. Liagkou, and E. Glavas, “Cybersecurity guide for SMEs: Protecting small and medium-sized enterprises in the digital era,” *J. Inf. Secur.*, vol. 16, no. 1, pp. 1–43, 2024.
- [2] Rapid7, “Rapid7,” Rapid7. Accessed: Dec. 28, 2025. [Online]. Available: <https://www.rapid7.com/fundamentals/attack-surface-management/>
- [3] “What is Attack Surface Management? | IBM.” Accessed: Dec. 28, 2025. [Online]. Available: <https://www.ibm.com/think/topics/attack-surface-management>
- [4] M. Husák and L. Sadlek, “Attack Surface Management: State of the Art and Operational Challenges,” in *2025 IEEE 11th International Conference on Network Softwarization (NetSoft)*, IEEE, 2025, pp. 1–6. Accessed: Dec. 03, 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11080588/>
- [5] T. Ashley, S. N. G. Gourisetti, N. Brown, and C. Bonebrake, “Aggregate attack surface management for network discovery of operational technology,” *Comput. Secur.*, vol. 123, p. 102939, 2022, Accessed: Dec. 03, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404822003315>
- [6] L. Thomas, “Maple Networks CTEM Whitepaper”.
- [7] “What Is Continuous Threat Exposure Management (CTEM)?,” Palo Alto Networks. Accessed: Jan. 04, 2026. [Online]. Available: <https://www.paloaltonetworks.com/cyberpedia/ctem-continuous-threat-exposure-management>
- [8] “CISO’s Guide to Breach and Attack Simulation.” Accessed: Dec. 03, 2025. [Online]. Available: <https://www.attackiq.com/wp-content/uploads/2025/01/driving-adversarial-exposure-validation-across-ctem-stages.pdf>
- [9] University of Portsmouth, UK, School of Energy & Electronic Engineering, Anglesea Building, Anglesea Road, Portsmouth (UK), PO1 3DJ and I. Chalvatzis, “Reproducible modelling and simulating security vulnerability scanners evaluation framework towards risk management assessment of small and medium enterprises business networks,” *Indian J. Sci. Technol.*, vol. 13, no. 37, pp. 3910–3943, Oct. 2020, doi: 10.17485/IJST/v13i37.868.
- [10] A. Orebaugh and B. Pinkard, *Nmap in the enterprise: your guide to network scanning*. Burlington, MA: Syngress Publishing, 2008.
- [11] G. Jasphe Kathrine, “COMPARATIVE ANALYSIS OF SUBDOMAIN ENUMERATION TOOLS AND STATIC CODE ANALYSIS,” *J. Mech. Contin. Math. Sci.*, vol. 15, no. 6, Jun. 2020, doi: 10.26782/jmcms.2020.06.00013.
- [12] A. Palma, C. Cicimurri, and M. Angelini, “Progressive attack graph: a technique for scalable and adaptive attack graph generation,” *Int. J. Inf. Secur.*, vol. 24, no. 5, p. 212, Oct. 2025, doi: 10.1007/s10207-025-01125-w.
- [13] “The Python Language Reference,” Python documentation. Accessed: Dec. 28, 2025. [Online]. Available: <https://docs.python.org/3/reference/index.html>
- [14] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring Network Structure, Dynamics, and Function using NetworkX,” *Python Sci. Conf.*, May 2008, doi: 10.25080/TCWV9851.

- [15] “CVE: Common Vulnerabilities and Exposures.” Accessed: Dec. 28, 2025. [Online]. Available: <https://www.cve.org/>
- [16] “HTML Standard.” Accessed: Dec. 30, 2025. [Online]. Available: <https://html.spec.whatwg.org/>
- [17] “Cascading Style Sheets.” Accessed: Dec. 30, 2025. [Online]. Available: <https://www.w3.org/Style/CSS/>
- [18] “JavaScript | MDN,” MDN Web Docs. Accessed: Dec. 30, 2025. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [19] “PHP: Documentation.” Accessed: Dec. 30, 2025. [Online]. Available: <https://www.php.net/docs.php>
- [20] “MySQL :: MySQL 8.0 Reference Manual.” Accessed: Jan. 22, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [21] A. Palma and M. Angelini, “It is Time To Steer: A Scalable Framework for Analysis-Driven Attack Graph Generation,” in *Computer Security – ESORICS 2024*, vol. 14985, J. Garcia-Alfaro, R. Kozik, M. Choraś, and S. Katsikas, Eds., in Lecture Notes in Computer Science, vol. 14985. , Cham: Springer Nature Switzerland, 2024, pp. 229–250. doi: 10.1007/978-3-031-70903-6_12.
- [22] A.-M. Konsta, B. Spiga, A. L. Lafuente, and N. Dragoni, “A Survey of Automatic Generation of Attack Trees and Attack Graphs,” Sep. 25, 2023, *arXiv*: arXiv:2302.14479. doi: 10.48550/arXiv.2302.14479.
- [23] C. Schmitz, A. Sekulla, and S. Pape, “Asset-Centric Analysis and Visualisation of Attack Trees,” in *Graphical Models for Security*, vol. 12419, H. Eades Iii and O. Gadyatskaya, Eds., in Lecture Notes in Computer Science, vol. 12419. , Cham: Springer International Publishing, 2020, pp. 45–64. doi: 10.1007/978-3-030-62230-5_3.
- [24] Y. Baddi, M. A. Almaiah, O. Almomani, and Y. Maleh, *The Art of Cyber Defense: From Risk Assessment to Threat Intelligence*. CRC Press, 2024.
- [25] Y. Valdés-Rodríguez, J. Hochstetter-Diez, M. Diéguez-Rebolledo, A. Bustamante-Mora, and R. Cadena-Martínez, “Analysis of Strategies for the Integration of Security Practices in Agile Software Development: A Sustainable SME Approach,” *IEEE Access*, vol. 12, pp. 35204–35230, 2024, doi: 10.1109/ACCESS.2024.3372385.
- [26] S. Kanneh, V. Anu, S. Kanneh, and V. Anu, “Security Requirements Prioritization Techniques: A Survey and Classification Framework,” *Software*, vol. 1, no. 4, pp. 450–472, Oct. 2022, doi: 10.3390/software1040019.
- [27] P. Das *et al.*, “STRIDE-Based Cybersecurity Threat Modeling, Risk Assessment and Treatment of an In-Vehicle Infotainment System,” *Vehicles*, vol. 6, no. 3, pp. 1140–1163, Jun. 2024, doi: 10.3390/vehicles6030054.

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

► Bibliography

Match Groups

-  **166 Not Cited or Quoted** 6%
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations** 0%
Matches that are still very similar to source material
-  **11 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 5%  Publications
- 0%  Submitted works (Student Papers)

Match Groups

-  **166 Not Cited or Quoted** 6%
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations** 0%
Matches that are still very similar to source material
-  **11 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 5%  Publications
- 0%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Publication	Coulin, Chad Raymond. "A Situational Approach and Intelligent Tool for Collabora...	<1%
2	Publication	Alyaa Hisham, Aya Ahmed, Mariam Khaled, Nour Abdullatif, Sally Kassem. "Model...	<1%
3	Internet	riversecurity.eu	<1%
4	Internet	www.coursehero.com	<1%
5	Internet	doczz.net	<1%
6	Publication	Csaba Hegedűs, Pál Varga. "Co-pilots for Arrowhead-based Cyber-Physical System...	<1%
7	Internet	www.ru.is	<1%
8	Internet	www.anu.edu.au	<1%
9	Publication	Phenikaa University	<1%
10	Internet	www.mdpi.com	<1%

11	Internet	rsjp.gov.rs	<1%
12	Publication	Alphonse Binele Abana, Patrick Dany Bavoua Kenfack, Paul Salomon Ngohe Ekam...	<1%
13	Internet	etd.aau.edu.et	<1%
14	Publication	Mohammad Shahnawaz Shaikh. "Deep learning approaches for brain tumor dete...	<1%
15	Publication	Natalija Jolić, Zvonko Kavran. "Simulation Modelling in the Function of Intermoda...	<1%
16	Publication	"Computer Safety, Reliability, and Security. SAFECOMP 2025 Workshops", Springe...	<1%
17	Publication	Hassan Jalil Hadl, Muhammad Khurram Khan, Naveed Ahmad, Rehana Yasmin. "C...	<1%
18	Internet	jurnal.polbeng.ac.id	<1%
19	Publication	Xiangyu Li, Marcelo d'Amorim, Alessandro Orso. "Intent-Preserving Test Repair", ...	<1%
20	Publication	Imano Williams, Xiaohong Yuan. "Evaluating the effectiveness of Microsoft threat...	<1%
21	Internet	dspace.daffodilvarsity.edu.bd:8080	<1%
22	Publication	Hsia-Ching Chang, Sulliman Hawamdeh. "Cybersecurity for Information Professio...	<1%
23	Internet	lcsts2025.lbt.ac.in	<1%
24	Internet	papers.academic-conferences.org	<1%

25	Internet	wjarr.com	<1%
26	Publication	Andres Andreu. "The CISO Playbook", CRC Press, 2024	<1%
27	Publication	Ayman Almjnoony, Rayan Alshamrani, Jim Alves-Foss, Frederick T. Sheldon. "Brid..."	<1%
28	Publication	Poonam Nandal, Tapas Kumar, Meeta Singh, Mamta Dahliya. "Progressive Compu..."	<1%
29	Internet	eprints.utar.edu.my	<1%
30	Internet	www.academicedgepress.co.uk	<1%
31	Internet	www.oreilly.com	<1%
32	Publication	James Curbo, Gregory Falco. "Testable Cyber Requirements for Space Flight Softw..."	<1%
33	Publication	Mohammad Rezaul Karim, Sohag Kabir, Ci Lei, Raluca Lefticaru. "A Combined App..."	<1%
34	Internet	adminhub.dev	<1%
35	Internet	www.utrc2.org	<1%
36	Publication	James Ransome, Anmol Misra. "Core Software Security - Security at the Source", A...	<1%
37	Publication	Alvi, Aleem Khalid. "Security Pattern Detection in Software Architectures", Queen'...	<1%
38	Internet	llc-jku.github.io	<1%

39	Internet	vdocuments.site	<1%
40	Internet	www3.imperial.ac.uk	<1%
41	Internet	activism.worldslargestlesson.globalgoals.org	<1%
42	Internet	dbnetworks.com	<1%
43	Internet	json-server.dev	<1%
44	Internet	matlab-project-codes.blogspot.com	<1%
45	Internet	www.exploitone.com	<1%
46	Internet	arxiv.org	<1%
47	Internet	download.bibis.ir	<1%
48	Internet	fourcore.io	<1%
49	Internet	www.tfzr.rs	<1%
50	Publication	"New Challenges in Software Engineering", Springer Science and Business Media ...	<1%
51	Publication	Bonomo, Flavia, Javier Marengo, Daniela Saban, and Nicol��s E. Stier-Moses. "A po...	<1%
52	Publication	Jose, Crispin R.. "Exploring Security Process Improvements for Integrating Securit...	<1%

53	Publication	Mark S. Merkow. "Secure, Resilient, and Agile Software Development", CRC Press, ...	<1%
54	Internet	drive.uqu.edu.sa	<1%
55	Internet	edoc.sub.uni-hamburg.de	<1%
56	Internet	fddocuments.us	<1%
57	Internet	univendspace.univen.ac.za	<1%
58	Internet	www.paloaltonetworks.com	<1%
59	Publication	"Advances in Ubiquitous Networking 2", Springer Science and Business Media LL...	<1%
60	Publication	"Cyber Security", Springer Science and Business Media LLC, 2026	<1%
61	Publication	"Graphical Models for Security", Springer Science and Business Media LLC, 2020	<1%
62	Publication	Asta Slotkienė, Jolanta Millauskaitė, Rasa Karbauskaitė. "Fuzzy-based Security Re...	<1%
63	Publication	Chengliang Gao, Jing Qiu, Yifeng Sun, Du Cheng, Lihua Yin. "Scalable logical attack ...	<1%
64	Publication	Jalaluddin, Arooj Zafar. "An Exploration of Countermeasures to Defend Against W...	<1%
65	Publication	Martinez Medina, Gonzalo. "Infrastructure, and Security: Advancing Smart Manuf...	<1%
66	Publication	Meenakshi Kandpal, Veena Goswami, Yash Pritwani, Rabindra K. Barik, Manob Jyo...	<1%

67	Publication	Mingsheng Yang, Yan Jia, Yangyang Mei, Jie Yang, Weihong Han, Jiawei Zhang, Zh...	<1%
68	Publication	Phillip A. Laplante, Chris Laplante. "Technical Writing - A Practical Guide for Engin...	<1%
69	Internet	alnafl.com	<1%
70	Internet	data.mitsgwallor.in	<1%
71	Internet	dirros.openscience.si	<1%
72	Internet	dokumen.pub	<1%
73	Internet	ljcrt.org	<1%
74	Internet	iris.polito.it	<1%
75	Internet	open.library.ubc.ca	<1%
76	Internet	par.nsf.gov	<1%
77	Internet	recipp.lpp.pt	<1%
78	Internet	repository.effatuniversity.edu.sa	<1%
79	Internet	www.kamatera.com	<1%
80	Internet	www.orblsresearch.com	<1%

81	Internet	www.ostl.gov	<1%
82	Internet	www.rewardheat.eu	<1%
83	Publication	Alessandro Palma, Silvia Bonomi. "Behind the scenes of attack graphs: Vulnerabl...	<1%
84	Publication	Anastasiya Kurylets. "Analysis of information security improvement approaches f...	<1%
85	Publication	Taghreed Bagles. "Classifying software security requirements into confidentiality...	<1%
86	Publication	Yousuf, Mohamed Faisal. "Analysis of Risk Reduction Using Threat Modeling Tech...	<1%
87	Publication	Yusuf Gür, Tuğba Taşkaya Temizel, Banu Günel Kılıç. "Implicit security requireme...	<1%